ECE2002
DIGITAL LOGIC DESIGN

Module 2 – Lecture 1

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

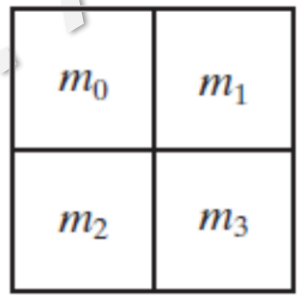
Karnaugh Map

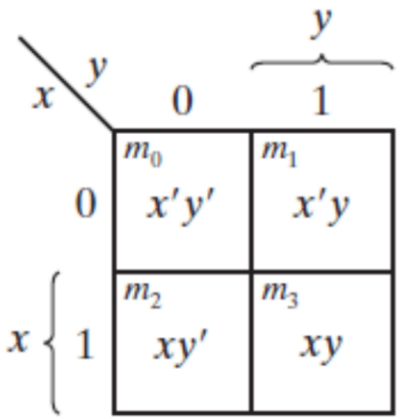
- *Gate-level minimization* is the design task of finding an optimal gate-level implementation of the Boolean functions describing a digital circuit.

- The **Karnaugh map** or **K-map** method provides a simple, straightforward procedure for minimizing Boolean functions in a pictorial form of a truth table.

Two-variable K-map

- There are four minterms for two variables; hence, the map consists of four squares, one for each minterm.
- The 0 and 1 marked in each row and column designate the values of variables.





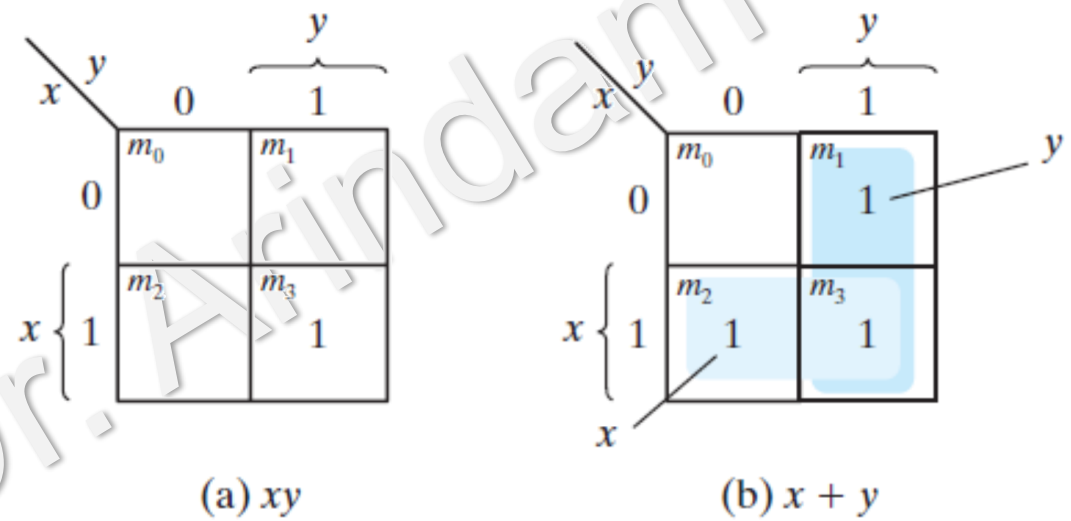
Dr. Arindam Dutta

Karnaugh Map

As an example, the function xy is shown in Fig. (a). Since xy is equal to m_3, a 1 is placed inside the square that belongs to m_3.

Similarly, the function $x + y$ is represented in the map of Fig. (b) by three squares marked with 1's. These squares are found from the minterms of the function:

$$m_1 + m_2 + m_3 = x'y + xy' + xy = x + y$$



(a) xy (b) $x + y$

The three squares could also have been determined from the intersection of variable x in the second row and variable y in the second column, which encloses the area belonging to x or y . In each example, the minterms at which the function is asserted are marked with a 1.

Dr. Arindam Dutta

Karnaugh Map

Three-variable K-map

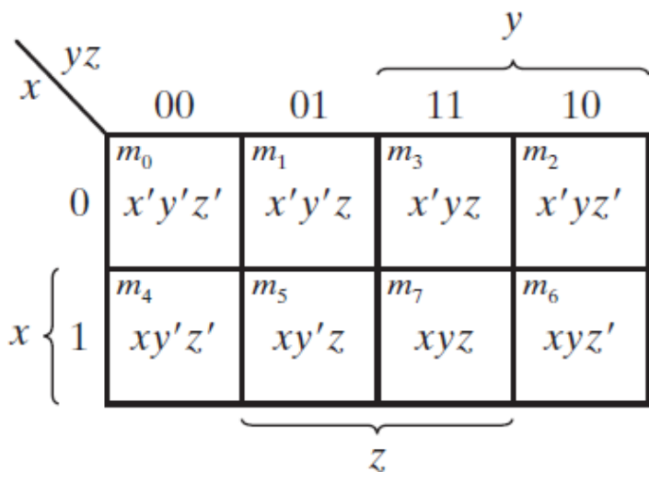
- There are eight minterms for three binary variables; therefore, the map consists of eight squares. The characteristic of this sequence is that **only one bit changes in value from one adjacent column to the next.**

- For example, the square assigned to m_5 corresponds to row 1 and column 01. For example, the square assigned to m_5 corresponds to row 1 and column 01. When these two numbers are concatenated, they give the binary number 101, whose decimal equivalent is 5.

- Each cell of the map corresponds to a unique minterm, so another way of looking at square $m_5 = xy'z$ is to consider it to be in the row marked x and the column belonging to $y'z$ (column 01).

- **Any two adjacent squares in the map differ by only one variable,** which is primed in one square and unprimed in the other.

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6



Dr. Arindam Dutta

Karnaugh Map

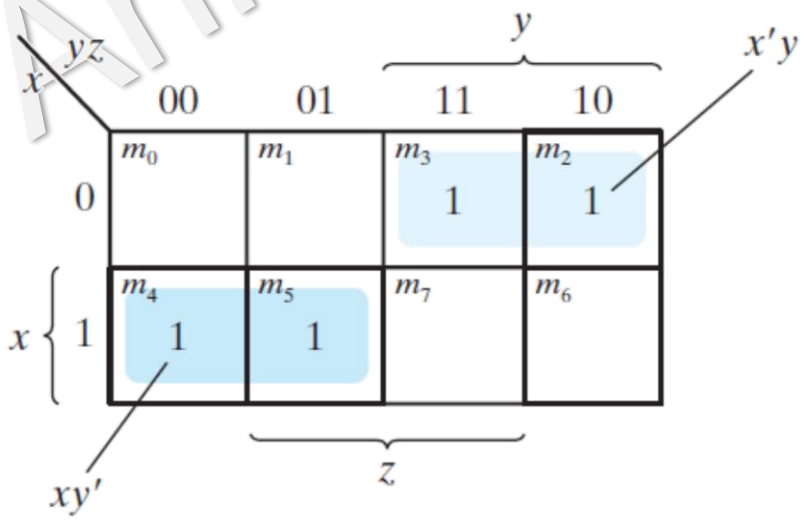
Example: Simplify the Boolean function $F(x, y, z) = \Sigma(2, 3, 4, 5)$.

First, a 1 is marked in each minterm square that represents the function.

The next step is to find possible adjacent squares. These are indicated in the map by two shaded rectangles, each enclosing two 1's. The upper right rectangle represents the area enclosed by $x'y$. This area is determined by observing that the two-square area is in row 0, corresponding to x', and the last two columns, corresponding to y.

Similarly, the lower left rectangle represents the product term xy'. The sum of four minterms can be replaced by a sum of only two product terms. The logical sum of these two product terms gives the simplified expression

$$F = x'y + xy'$$



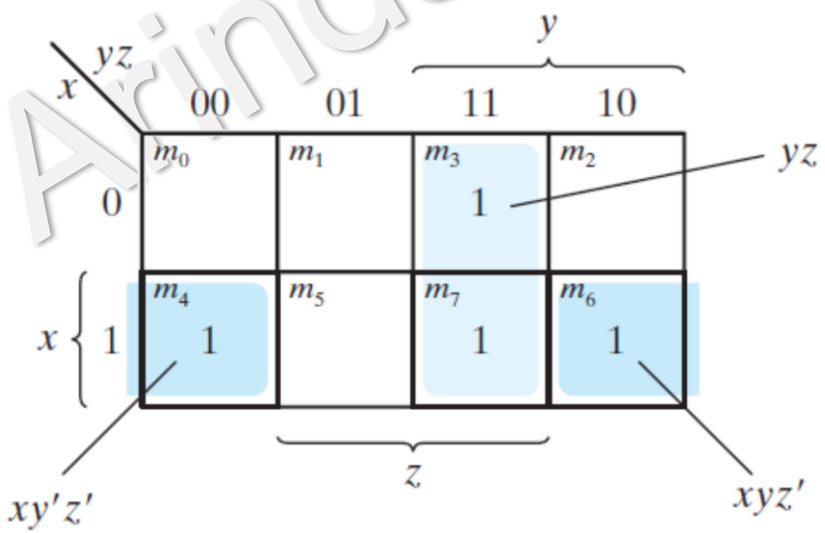
Dr. Arindam Dutta

Karnaugh Map

Example: Simplify the Boolean function $F(x, y, z) = \Sigma(3, 4, 6, 7)$.

There are four squares marked with 1's, one for each minterm of the function. Two adjacent squares are combined in the third column to give a two-literal term *yz*. The remaining two squares with 1's are also adjacent by the new definition. These two squares, when combined, give the two-literal term *xz'*. The simplified function then becomes

$$F = yz + xz'$$



Note: $xy'z' + xyz' = xz'$

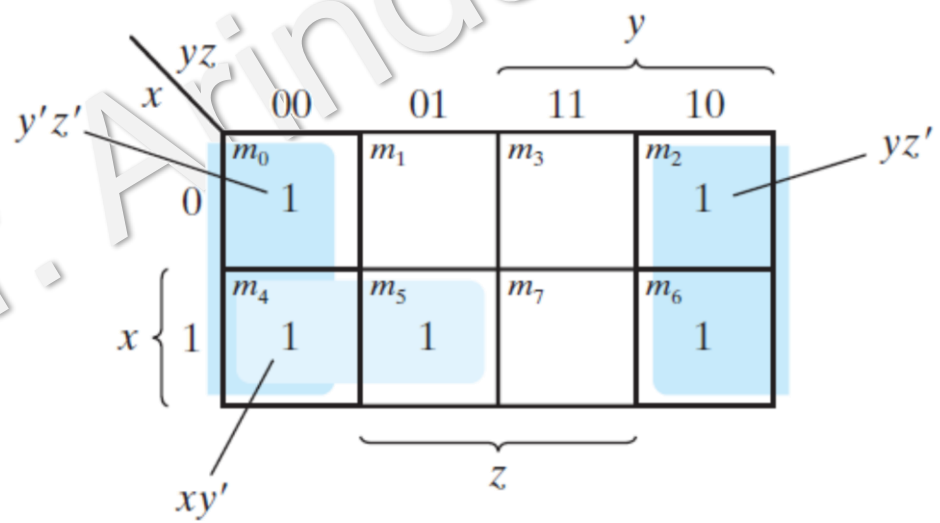
Dr. Arindam Dutta

Karnaugh Map

Example: Simplify the Boolean function $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$.

First, we combine the four adjacent squares in the first and last columns to give the single literal term z'. The remaining single square, representing minterm 5, is combined with an adjacent square that has already been used once. This is not only permissible, but rather desirable, because the two adjacent squares give the two-literal term xy' and the single square represents the three-literal minterm $xy'z$. The simplified function is

$$F = z' + xy'$$



Note: $y'z' + yz' = z'$

Dr. Arindam Dutta

Karnaugh Map

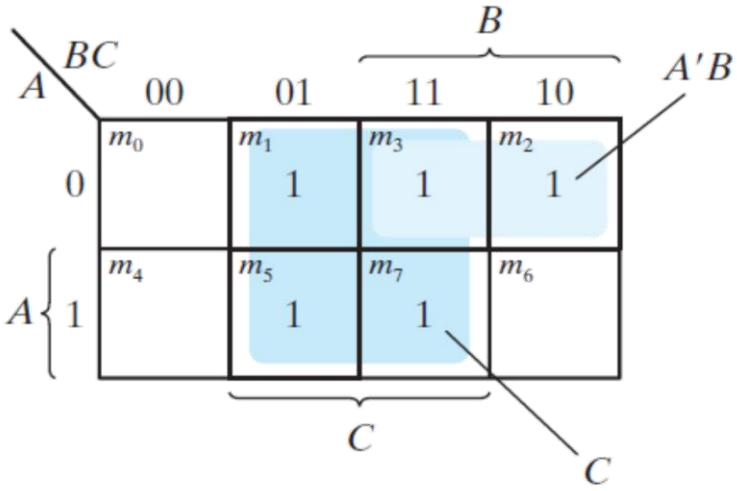
Example: For the Boolean function $F = A'C + A'B + AB'C + BC$, (a) express this function as a sum of minterms and (b) find the minimal sum-of-products expression.

Note that F is a sum of products. Three product terms in the expression have two literals and are represented in a three-variable map by two squares each. The two squares corresponding to the first term, $A'C$, are found from the coincidence of A' (first row) and C (two middle columns) to give squares 001 and 011.

Note that, in marking 1's in the squares, it is possible to find a 1 already placed there from a preceding term. This happens with the second term, $A'B$, which has 1's in squares 011 and 010. Square 011 is common with the first term, $A'C$, though, so only one 1 is marked in it.

Continuing in this fashion, we determine that the term $AB'C$ belongs in square 101, corresponding to minterm 5, and the term BC has two 1's in squares 011 and 111. The function has a total of five minterms, as read directly from the map to be 1, 2, 3, 5, and 7. It can be expressed in sum-of-minterms form as

$$F(A, B, C) = \Sigma(1, 2, 3, 5, 7)$$



The sum-of-products expression, as originally given, has too many terms. It can be simplified, as shown in the map, to an expression with only two terms:

$$F = C + A'B$$

Dr. Arindam Dutta

Karnaugh Map

Four-variable K-map:

- Let the four binary variables be w, x, y, and z. There are16 minterms.
- The rows and columns are numbered such that only one digit changes value between two adjacent rows or columns. The minterm corresponding to each square can be obtained from the concatenation of the row number with the column number. For example, the numbers of the third row (11) and the second column (01), when concatenated, give the binary number 1101, the binary equivalent of decimal 13. Thus, the square in the third row and second column represents minterm m_{13}.

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

wx \ yz	00	01	11	10
00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$

y (00, 01, 11, 10); x (01, 11); w (11, 10); z

Dr. Arindam Dutta

Karnaugh Map

- The map minimization of four-variable Boolean functions is similar to the method used to minimize three-variable functions.

- The combination of adjacent squares that is useful during the simplification process is easily determined from inspection of the four-variable map:

  - One square represents one minterm, giving a term with four literals.
  - Two adjacent squares represent a term with three literals.
  - Four adjacent squares represent a term with two literals.
  - Eight adjacent squares represent a term with one literal.
  - Sixteen adjacent squares produce a function that is always equal to 1.

Dr. Arindam Dutta

Karnaugh Map

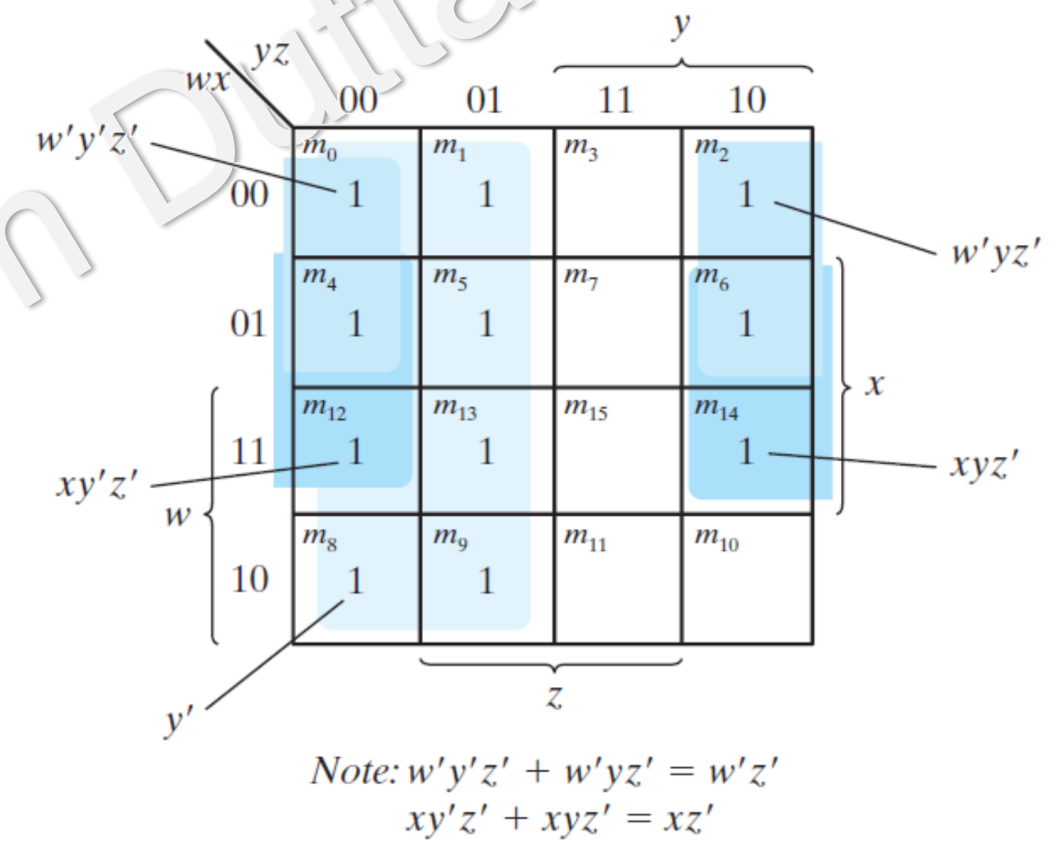
Example: Simplify the Boolean function $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$.

Eight adjacent squares marked with 1's can be combined to form the one literal term *y'*.

The remaining three 1's on the right cannot be combined to give a simplified term; they must be combined as two or four adjacent squares. The top two 1's on the right are combined with the top two 1's on the left to give the term *w'z'*. *(We can use the same square more than once)*.

We are now left with a square marked by 1 in the third row and fourth column (square 1110). Instead of taking this square alone (which will give a term with four literals), we combine it with squares already used to form an area of four adjacent squares. These squares make up the two middle rows and the two end columns, giving the term *xz'*.

The simplified function is $F = y' + w'z' + xz'$



Note: $w'y'z' + w'yz' = w'z'$
$xy'z' + xyz' = xz'$

Dr. Arindam Dutta

Karnaugh Map

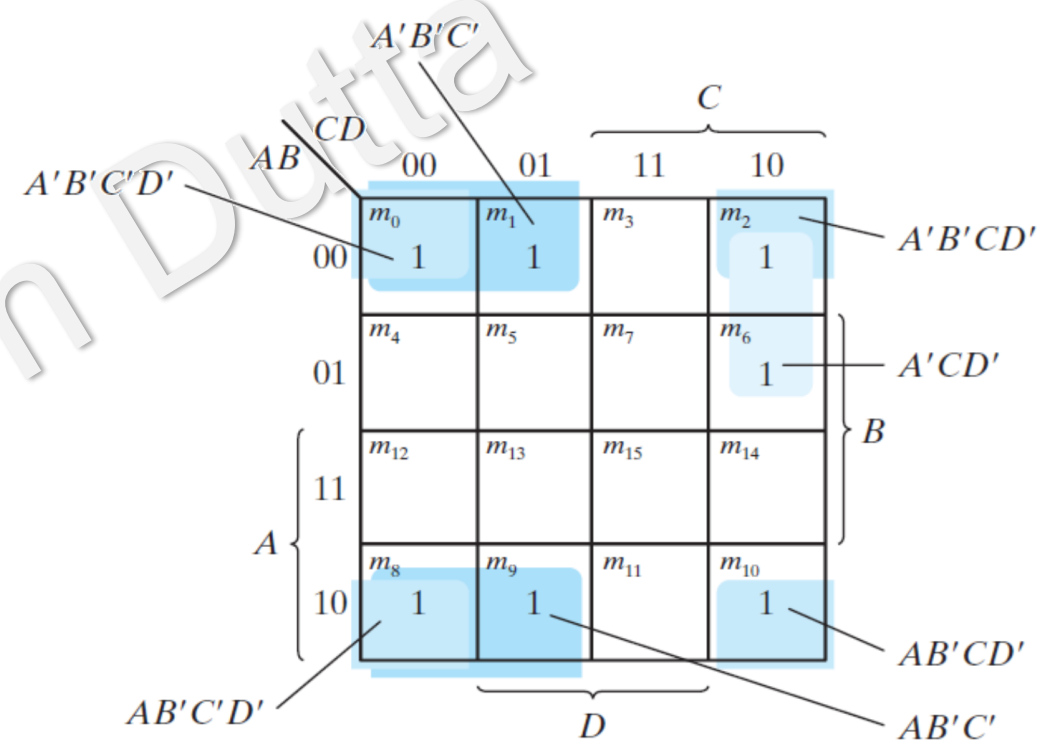
Example: Simplify the Boolean function *F = A'B'C' + B'CD' + A'BCD' + AB'C'*.

The function has four variables and consists of three terms with three literals each and one term with four literals. Each term with three literals is represented in the map by two squares. For example, *A'B'C'* is represented in squares 0000 and 0001.

The function can be simplified in the map by taking the 1's in the four corners to give the term *B'D'*. This is possible because these four squares are adjacent when the map is drawn in a surface with top and bottom edges, as well as left and right edges, touching one another.

The two left-hand 1's in the top row are combined with the two 1's in the bottom row to give the term *B'C'*. The remaining 1 may be combined in a two square area to give the term *A'CD'*.

The simplified function is ***F = B'D' + B'C' + A'CD'***



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
$AB'C'D' + AB'CD' = AB'D'$
$A'B'D' + AB'D' = B'D'$
$A'B'C' + AB'C' = B'C'$

Dr Arindam Dutta

Karnaugh Map

- The minimized Boolean functions in all previous examples were in **sum-of-products (SOP)** form.
- The minimized function can also be obtained in **product-of-sums (POS)** form.
- The 1's placed in the squares of the map represent the minterms of the function. The minterms not included in the standard SOP form of a function denote the complement of the function.
- Thus, the complement of a function is represented in the map by the squares not marked by 1's.
- If we mark the empty squares by 0's and combine them into valid adjacent squares, we obtain a simplified SOP expression of the complement of the function (i.e., of *F'*).
- The complement of *F'* gives us back the function *F* in POS form (a consequence of DeMorgan's theorem).

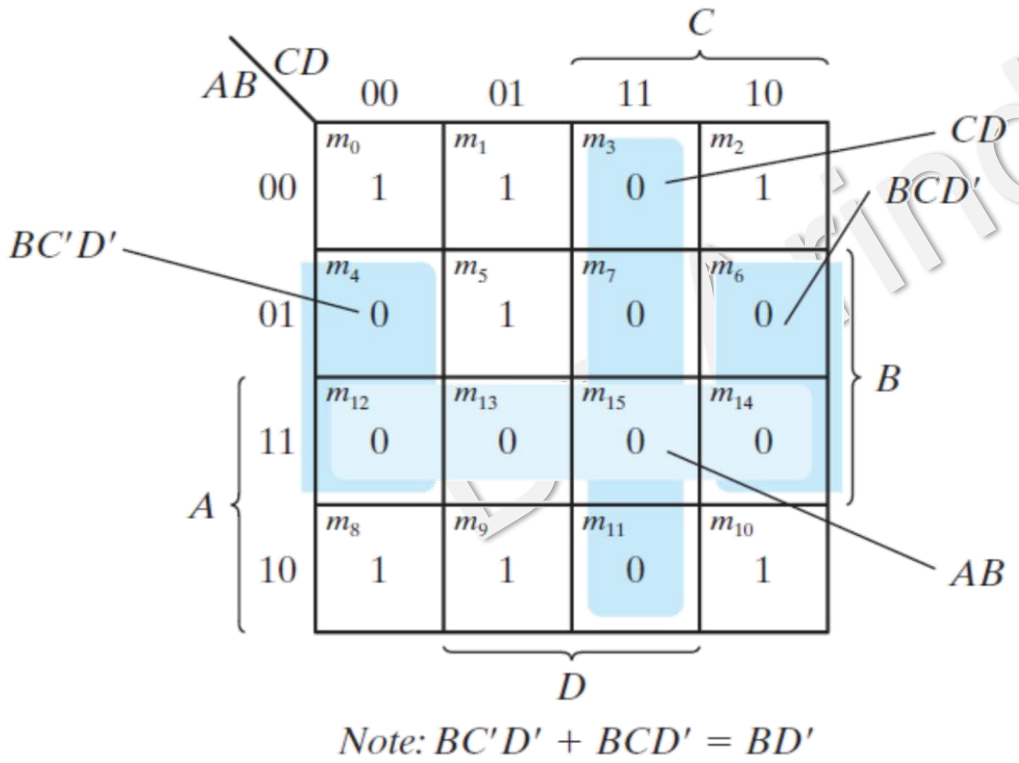
Dr. Arindam Dutta

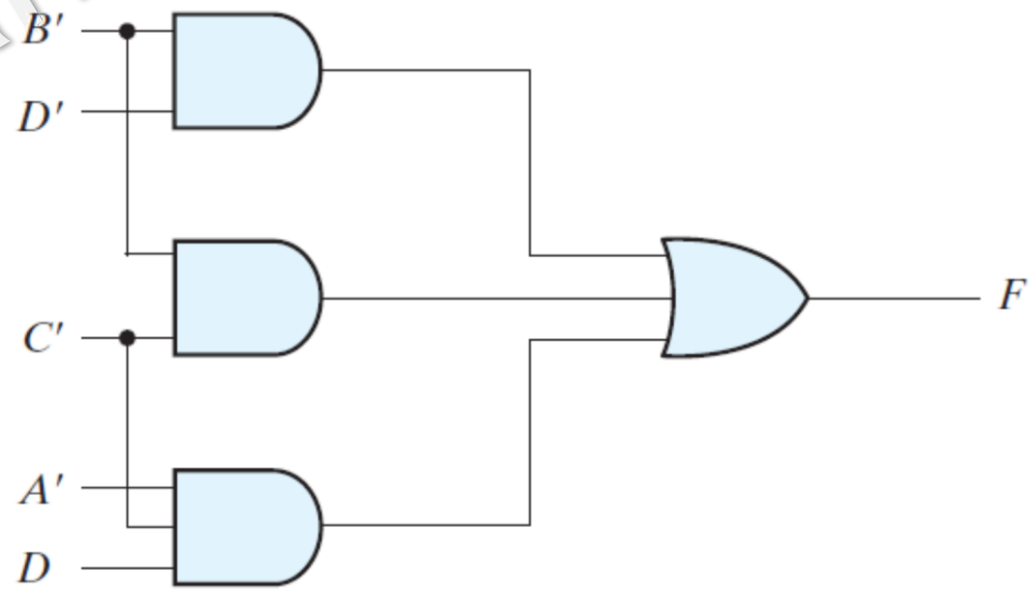
Karnaugh Map

Example: Simplify the Boolean function into (a) SOP and (b) POS forms: $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$

(a) The 1's marked in the map represent all the minterms of the function. The squares marked with 0's represent the minterms not included in F and therefore denote F'. Combining the squares with 1's gives the simplified function in SOP form:

$$F = B'D' + B'C' + A'C'D$$





Dr. Nilanjan Dutta

Karnaugh Map

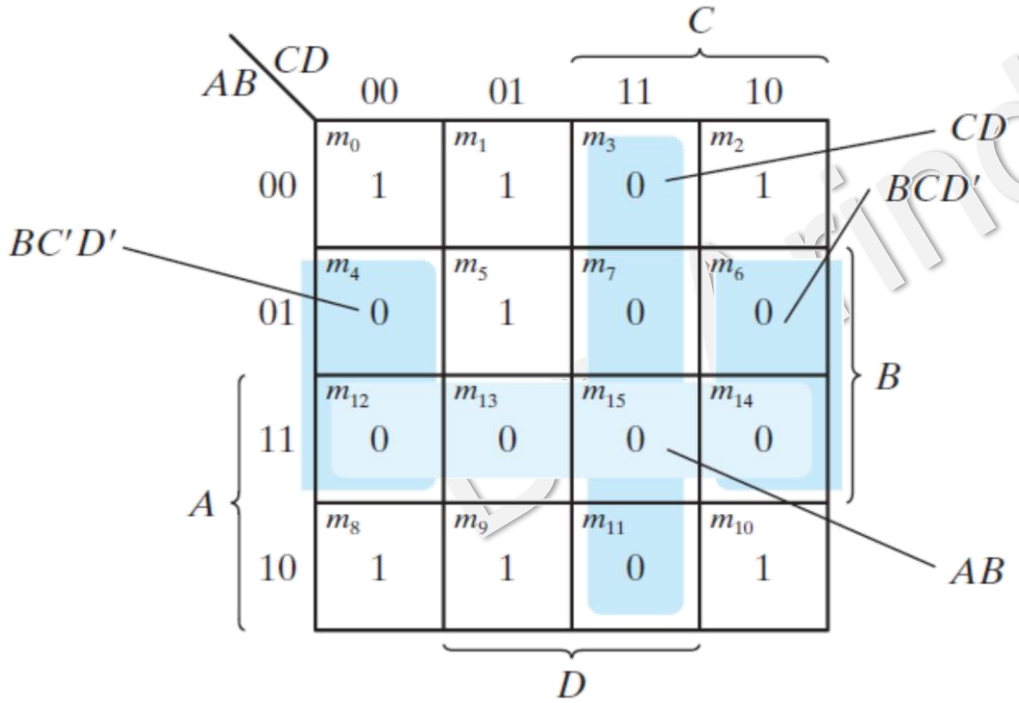
(b) From (a) we have: $F = B'D' + B'C' + A'C'D$

If the squares marked with 0's are combined, we obtain the simplified complemented function:

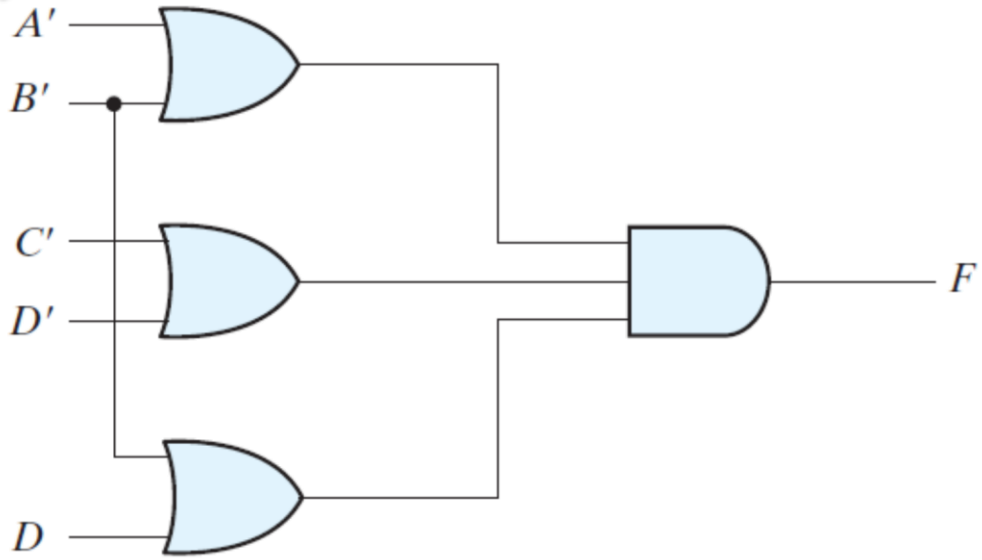
$$F' = AB + CD + BD'$$

By taking the dual and complementing each literal, we obtain the simplified function in POS form:

$$\boldsymbol{F = (A' + B')\,(C' + D')\,(B' + D)}$$



Note: $BC'D' + BCD' = BD'$



Nandan Dutta

Karnaugh Map

In sum-of-minterms form, the function F in the truth table is expressed as

$$F(x, y, z) = \Sigma(1, 3, 4, 6)$$

In product-of-maxterms form, it is expressed as

$$F(x, y, z) = \Pi(0, 2, 5, 7)$$

1's of the function represent the minterms and 0's represent the maxterms. First mark the 1's for each minterm that the function is a 1. The remaining squares are marked by 0's. If, instead, the product of maxterms is initially given, first mark 0's in those squares listed in the function; the remaining squares are then marked by 1's. Now, the function can be simplified in either one of the standard forms. For SOP, we combine the 1's to obtain

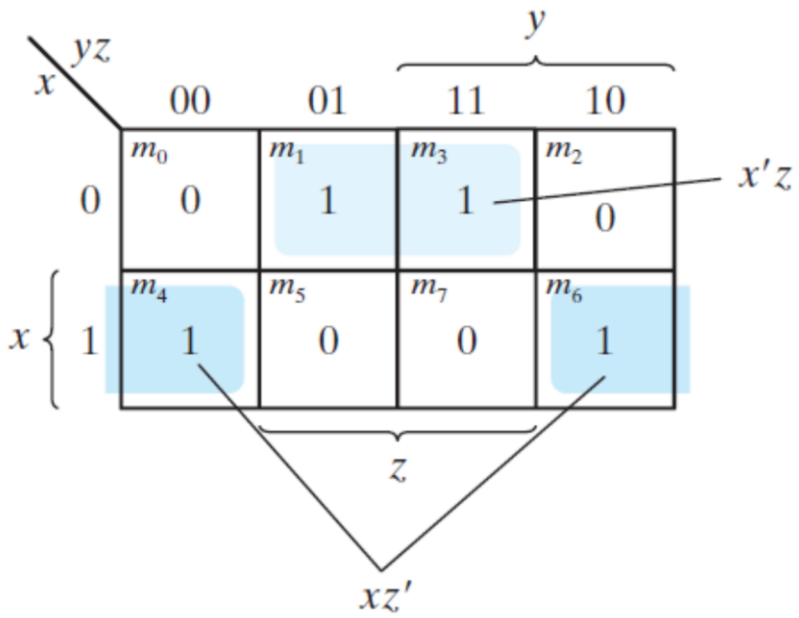
$$F = x'z + xz'$$

For POS, we combine the 0's to obtain the complemented function

$$F' = xz + x'z'$$

Taking the complement of F, the POS form is: $F = (x' + z')(x + z)$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0



Dr. Arindam Dutta

Karnaugh Map

Don’t-Care Conditions

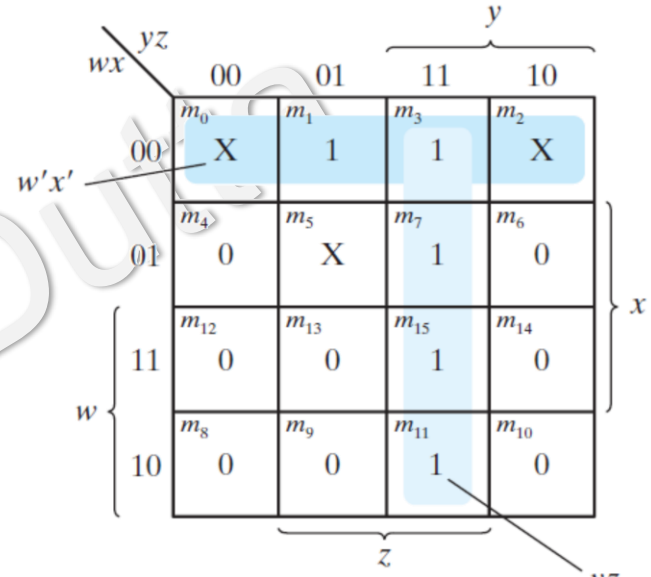
- In some applications the function is not specified for certain combinations of the variables.
- Such functions are called *incompletely specified functions*.
- In most applications, we simply don’t care what value (0 or 1) is assumed by the function for the unspecified minterms. These unspecified minterms are called *don’t-care conditions*.
- These don’t-care conditions can be used on a map to provide further simplification of the Boolean expression.
- A don’t-care minterm is a combination of variables whose logical value is not specified.
- To distinguish the don’t-care condition from 1’s and 0’s, an X is used.
- In choosing adjacent squares to simplify the function in a map, the don’t-care minterms may be assumed to be either 0 or 1.
- When simplifying the function, we can choose to include each don’t-care minterm with either the 1’s or the 0’s, depending on which combination gives the simplest expression.

Dr. Arindam Dutta

Karnaugh Map

Example: Simplify the Boolean function $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$ which has the don't-care conditions $d(w, x, y, z) = \Sigma(0, 2, 5)$.

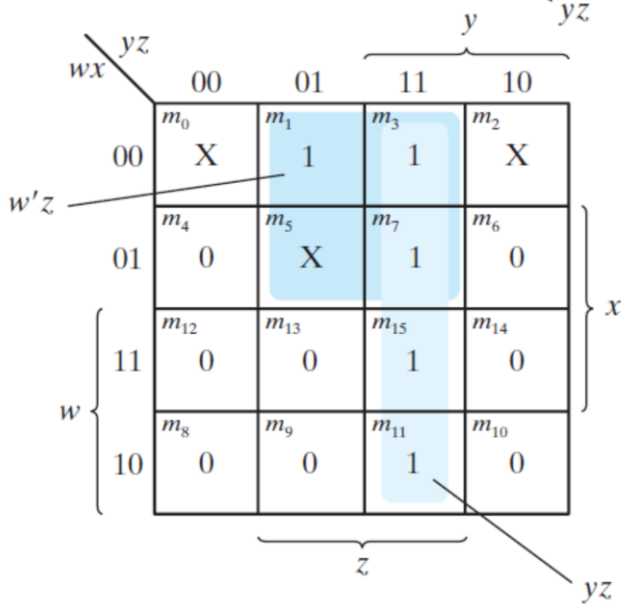
The minterms of F are marked by 1's, those of d are marked by X's, and the remaining are filled with 0's. For SOP, we must include all five 1's in the map, but we may or may not include any of the X's, depending on the way the function is simplified. The term yz covers the four minterms in the third column. The remaining minterm, m_1, can be combined with m_3 to give the three-literal term $w'x'z$. However, by including one or two adjacent X's we can combine four adjacent squares to give a two-literal term.



(a)

In Fig. (a), don't-care minterms 0 and 2 are included with the 1's, resulting in the simplified function

$$F = yz + w'x'$$



(b)

In Fig. (b), don't-care minterm 5 is included with the 1's, and the simplified function is now

$$F = yz + w'z$$

Dr. Arindam Dutta

Karnaugh Map

The only way to obtain a simplified POS expression for the same function in the previous slide is to combine the 0's by including don't-care minterms 0 and 2 with the 0's to give a simplified complemented function:

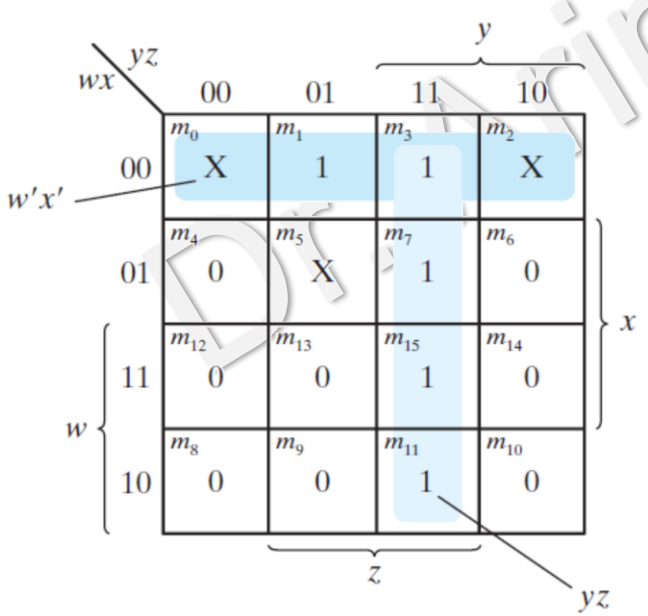
$$F' = z' + wy'$$

Taking the complement of F' gives the simplified expression in product-of-sums form:

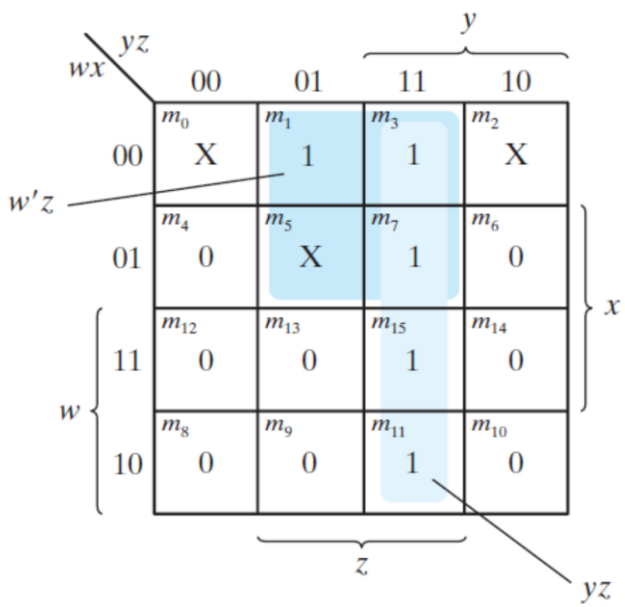
$$F(w, x, y, z) = z(w' + y) = \Sigma(1, 3, 5, 7, 11, 15)$$

In this case, we include minterms 0 and 2 with the 0's and minterm 5 with the 1's.

(a)



(b)



Dr. Arindam Dutta

Karnaugh Map

NAND Implementation

- The NAND gate is said to be a *universal* gate because any logic circuit can be implemented with it.
- A convenient way to implement a Boolean function with NAND gates is to obtain the simplified Boolean function in terms of Boolean operators and then convert the function to NAND logic.
- The implementation of Boolean functions with NAND gates requires that the function be in SOP form.
- The procedure for obtaining the logic diagram from a Boolean function is as follows:
  - Simplify the function and express it in sum-of-products form.
  - Draw a NAND gate for each product term of the expression that has at least two literals. The inputs to each NAND gate are the literals of the term. This procedure produces a group of first-level gates.
  - Draw a single NAND gate in the second level, with inputs coming from outputs of first-level gates.
  - A term with a single literal can be complemented (if not already in primed form) and connected directly to an input of the second-level NAND gate.

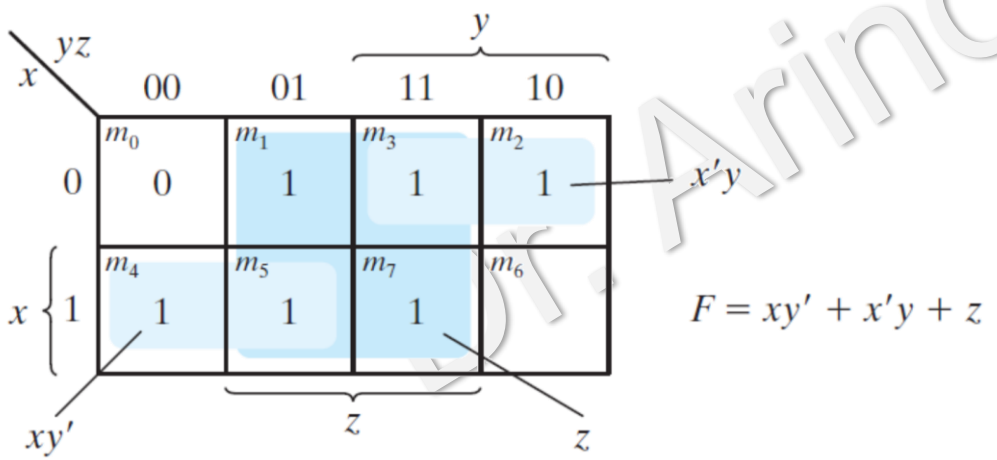
Dr. Arindam Dutta

Karnaugh Map

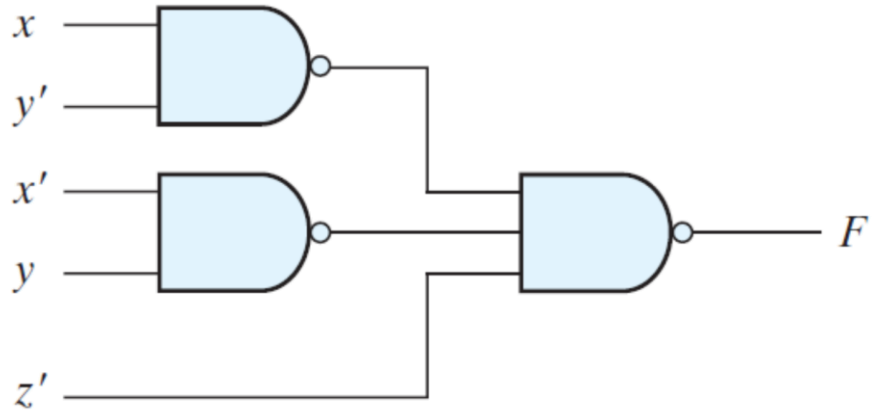
Example: Implement the following Boolean function with NAND gates: $F(x, y, z) = (1, 2, 3, 4, 5, 7)$.

The first step is to simplify the function into SOP form. This is done by K-map, from which the simplified function obtained is:

$$F = xy' + x'y + z$$



$F = xy' + x'y + z$

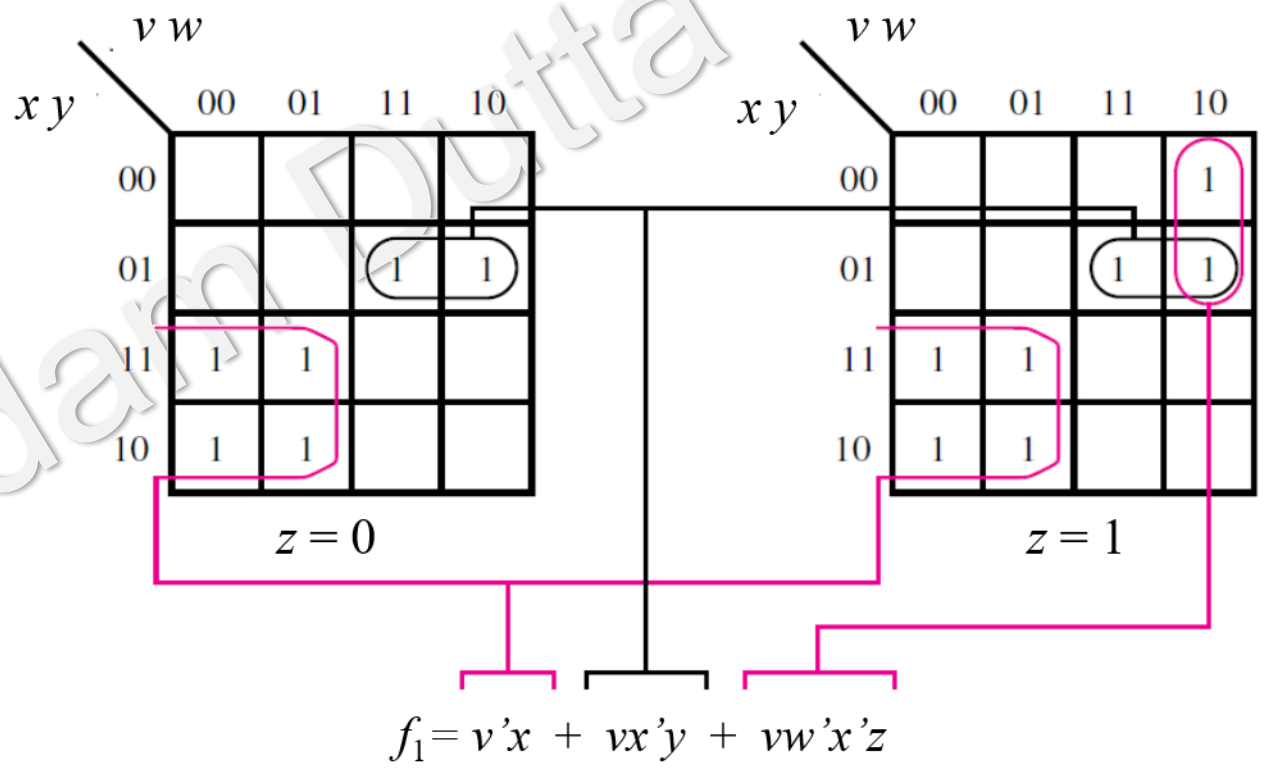


Dr. Arindam Dutta

Karnaugh Map

Five-variable K-map

- We can use 2 four-variable maps to construct a five-variable map.
- One map is imagined to be directly behind the other, and are distinguished by $z = 0$ for one map and $z = 1$ for the other, and place the two maps side by side.
- In this example, two groups of four 1's appear in the same place in both four-variable maps; hence their realization does not depend on the value of z.
- Same is true for two groups of two 1's in second row.
- The 1 in top-right corner appears only in the right map, where $z = 1$; it is a part of the group of two 1's realized by the term $vw'x'z$.



Dr. Arindam Dutta

- Note that in this map we left blank those cells for which $f = 0$ just like in some of the previous examples.

ECE2002
DIGITAL LOGIC DESIGN

Module 2 – Lecture 2

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

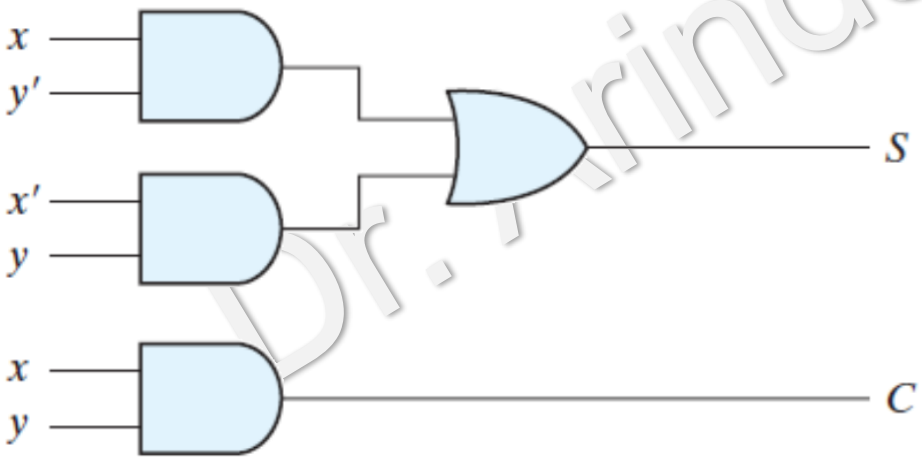
Arithmetic Circuits

Half Adder: A combinational circuit that performs the addition of two bits.

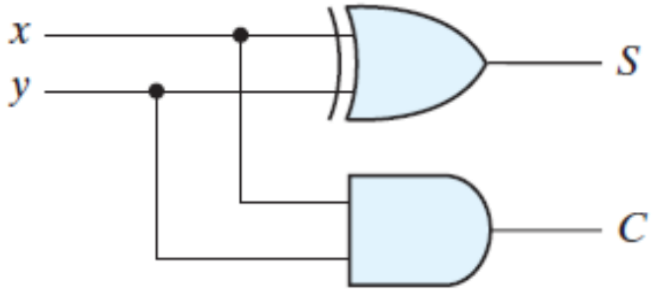
x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Sum, $S = x'y + xy' = x \oplus y$

Carry, $C = x\,y$



(a) $S = xy' + x'y$
$C = xy$

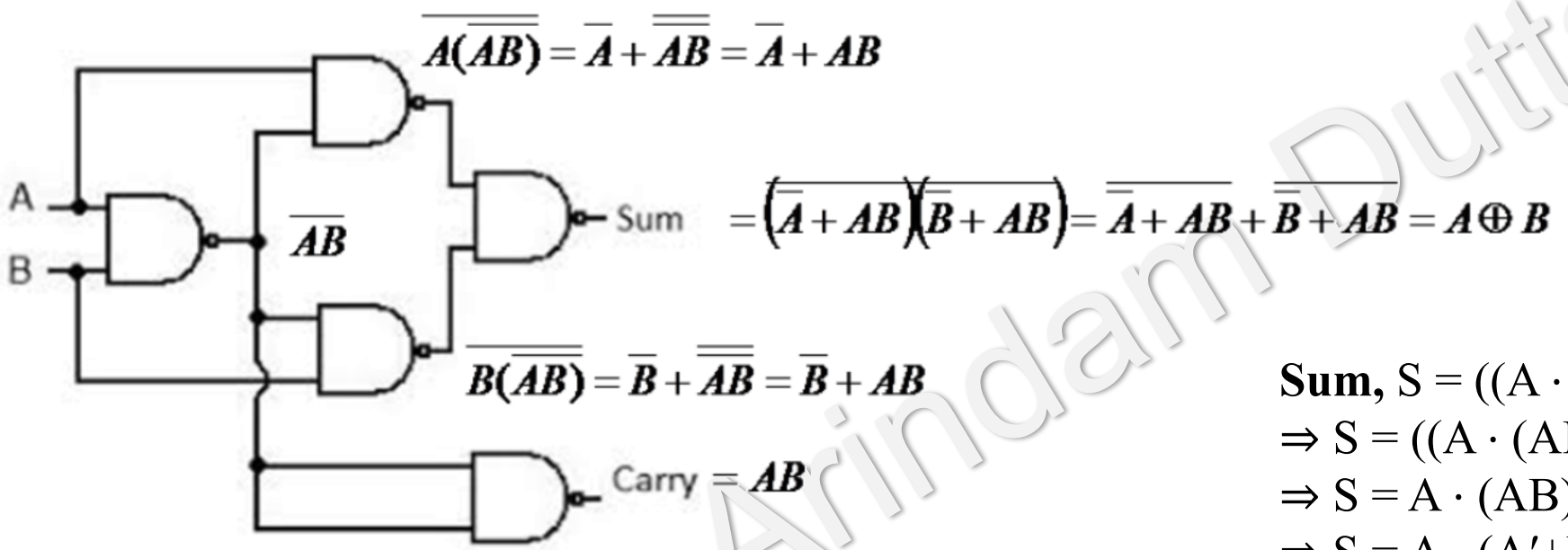


(b) $S = x \oplus y$
$C = xy$

Dr. Arindam Dutta

Arithmetic Circuits

Half Adder Using NAND Gates



Dr. Arindam Dutta

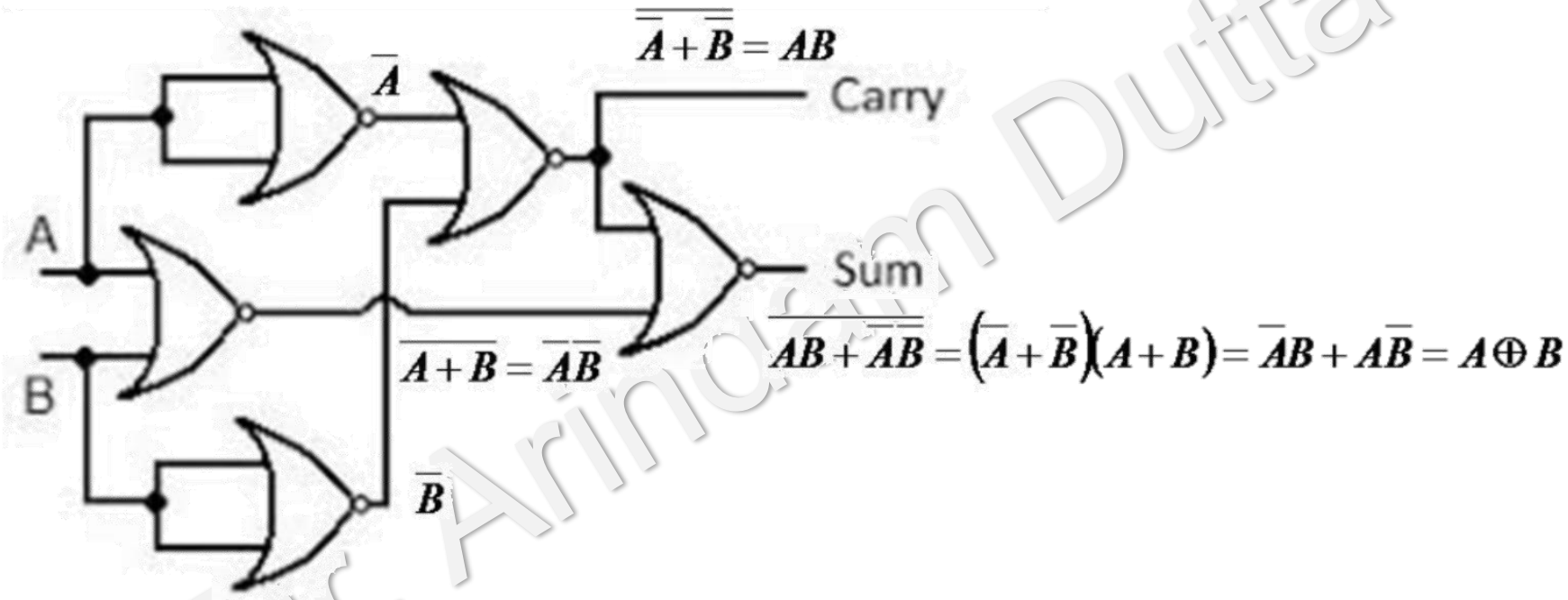
Sum, S = ((A · (AB)′)′ · (B · (AB)′)′)′
⇒ S = ((A · (AB)′)′)′ + ((B · (AB)′)′)′
⇒ S = A · (AB)′ + B · (AB)′
⇒ S = A · (A′+B′) + B · (A′+B′)
⇒ S=AA′ + AB′ + A′B + BB′
∴ S = AB′ + A′B = A⊕B

Similarly,

Carry, C = ((AB)′)′ = AB

Arithmetic Circuits

Half Adder Using NOR Gates

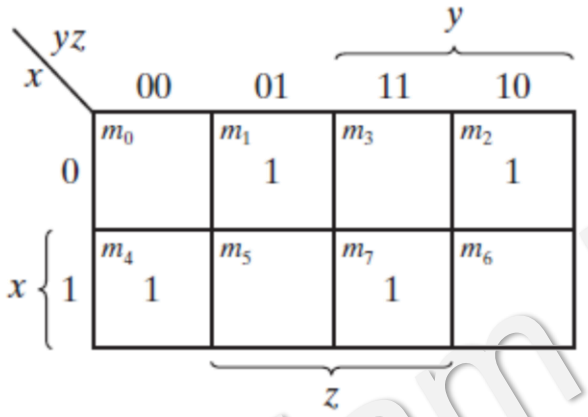


Dr. Arindam Dutta

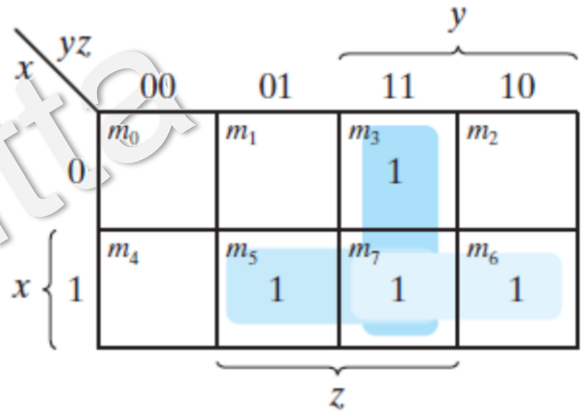
Arithmetic Circuits

Full Adder: A combinational circuit that performs the addition of three bits.

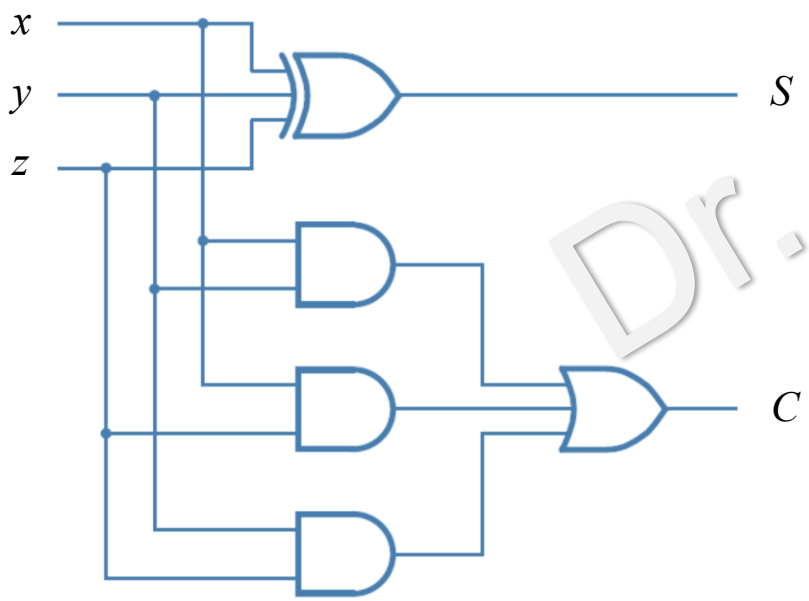
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

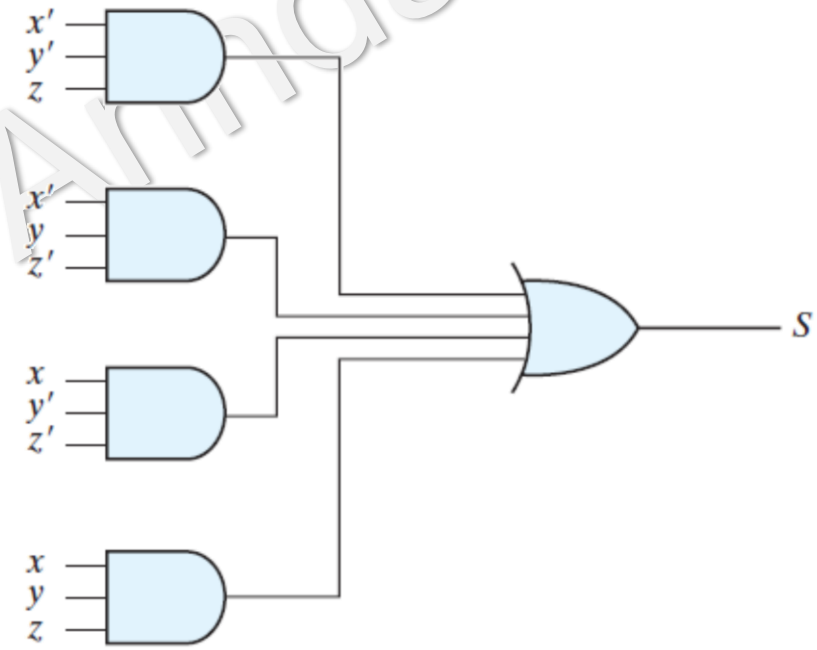


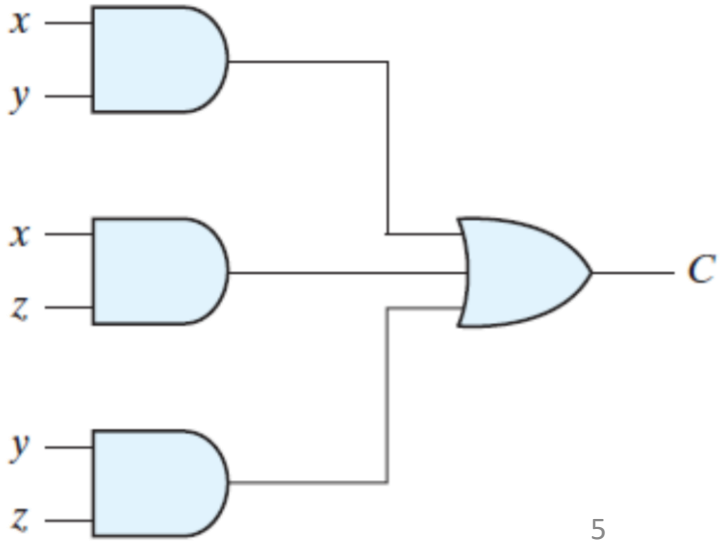
Sum, $S = x'y'z + x'yz' + xy'z' + xyz$



Carry, $C = xy + xz + yz$







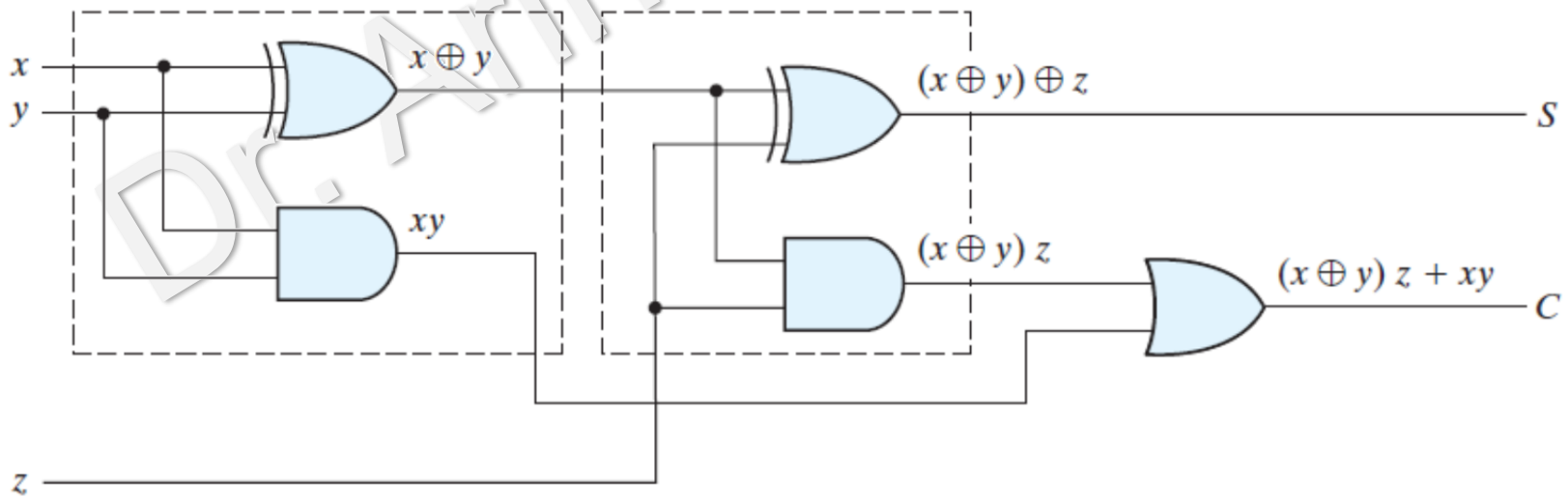
Dr. Arnab Dutta

Arithmetic Circuits

A full-adder can also be implemented with two half adders and one OR gate. The *S* output from the second half adder is the exclusive-OR of *z* and the output of the first half adder, giving:

$$
\begin{aligned}
S &= z \oplus (x \oplus y) \\
&= z'(xy' + x'y) + z(xy' + x'y)' \\
&= z'(xy' + x'y) + z(xy + x'y') \\
&= xy'z' + x'yz' + xyz + x'y'z
\end{aligned}
$$

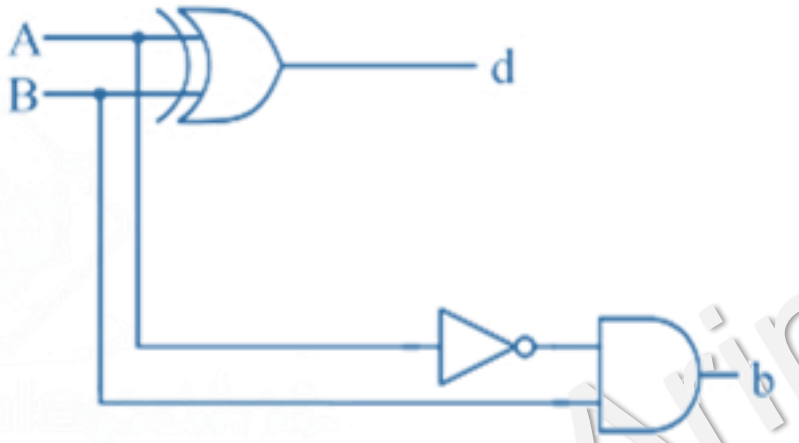
The carry output is $C = x'yz + xy'z + xyz' + xyz$ (minterms) $= (x'y + xy')z + xy(z + z') = (x \oplus y)z + xy$



Dr. Arindam Dutta

Arithmetic Circuits

Half Subtractor: A combinational logic circuit that performs the subtraction of two bits yielding two outputs (difference and borrow).



Inputs		Outputs	
A	**B**	**d (Difference)**	**b (Borrow)**
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

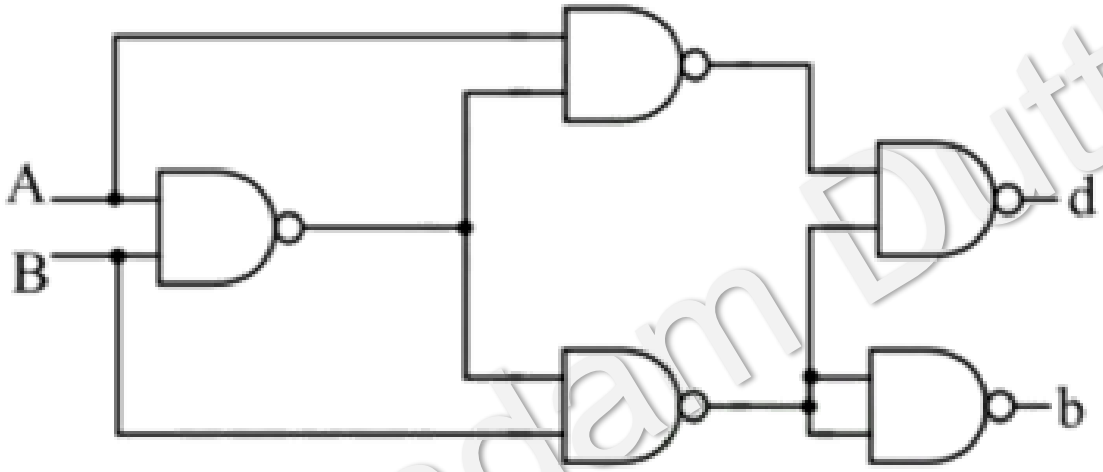
$$d = A' B + A B' = A \oplus B$$

$$b = A' B$$

Dr. Arindam Dutta

Arithmetic Circuits

Half Subtractor Using NAND Gates



Dr. Arindam Dutta

Difference bit (d)

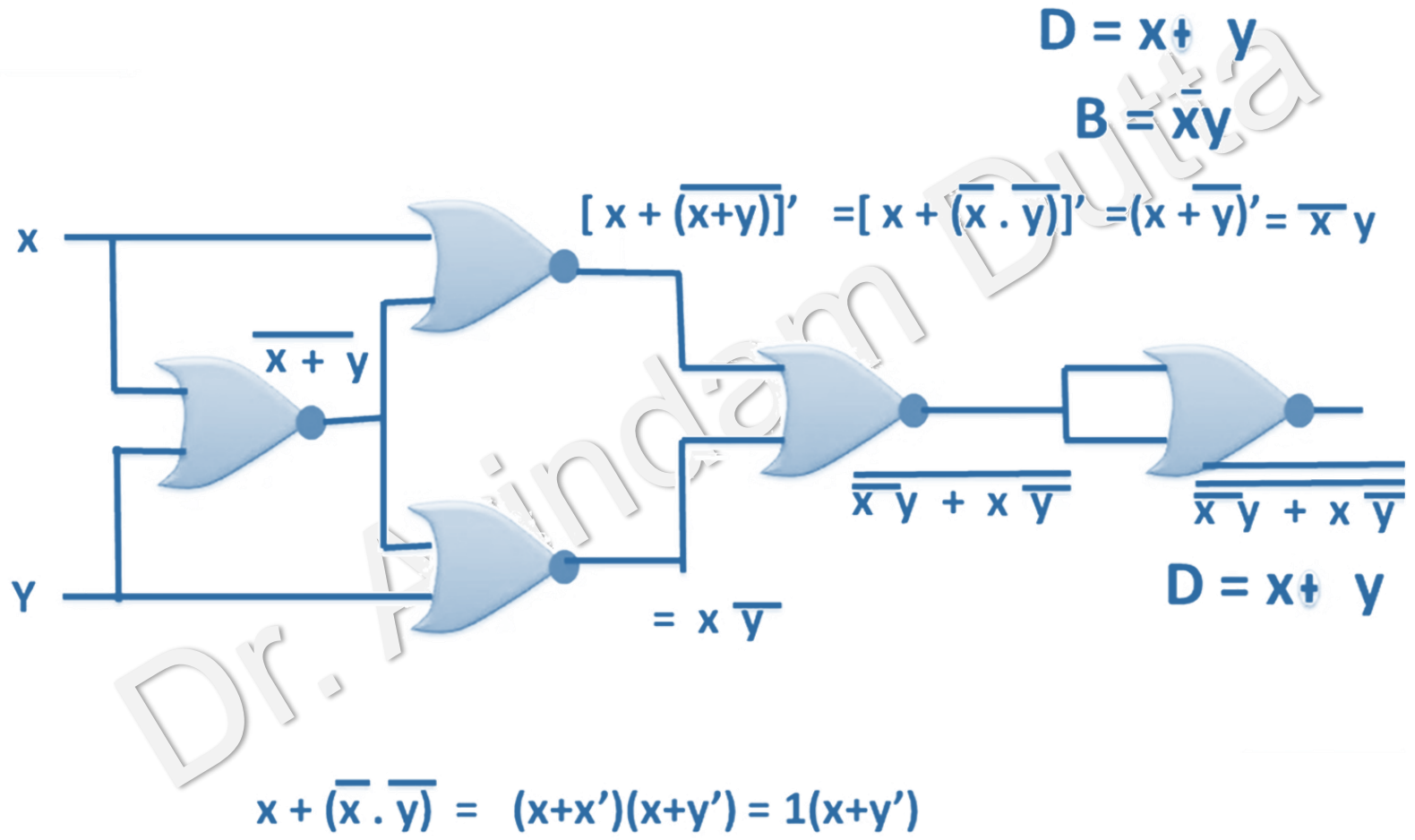
$$\text{Difference, d} = \overline{\overline{A \cdot \overline{AB}} \cdot \overline{B \cdot \overline{AB}}} = A \oplus B$$

Borrow bit (b)

$$\text{Borrow, b} = \overline{\overline{B \cdot \overline{AB}}} = \overline{A}\,B$$

Arithmetic Circuits

Half Subtractor Using NOR Gates



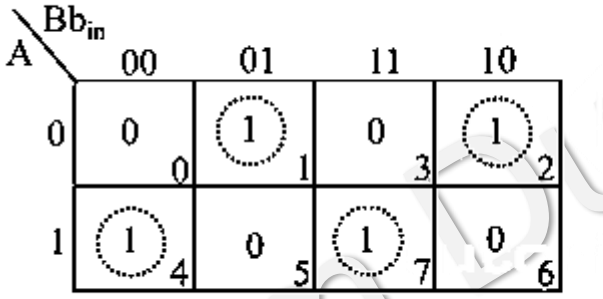
Dr. Arindam Dutta

Arithmetic Circuits

Full Subtractor: A combinational logic circuit that performs subtraction on three binary bits.

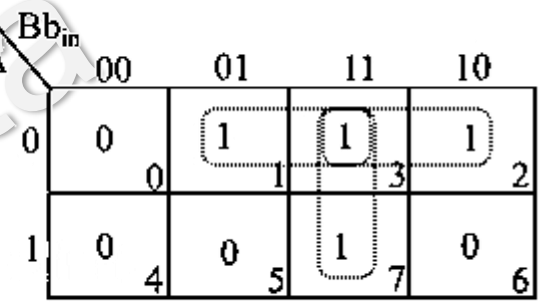
Inputs			Outputs	
A	**B**	**b_{in}**	**d (Difference)**	**b (Borrow)**
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

For Difference (d)

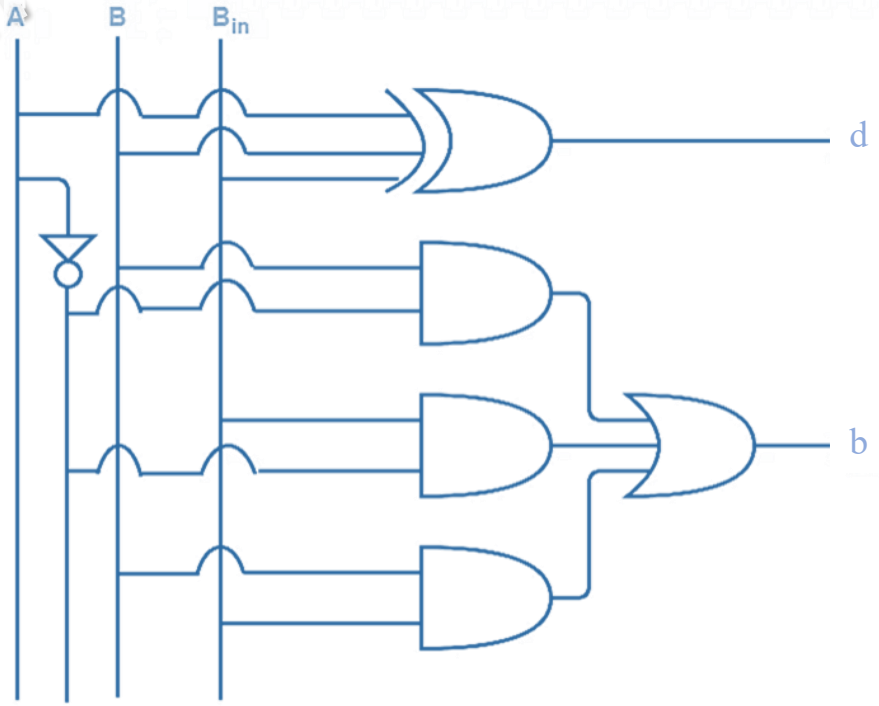


$d = A'B'b_{in} + A'Bb'_{in} + AB'b'_{in} + ABb_{in}$

For Borrow (b)



$b = A'b_{in} + A'B + Bb_{in}$



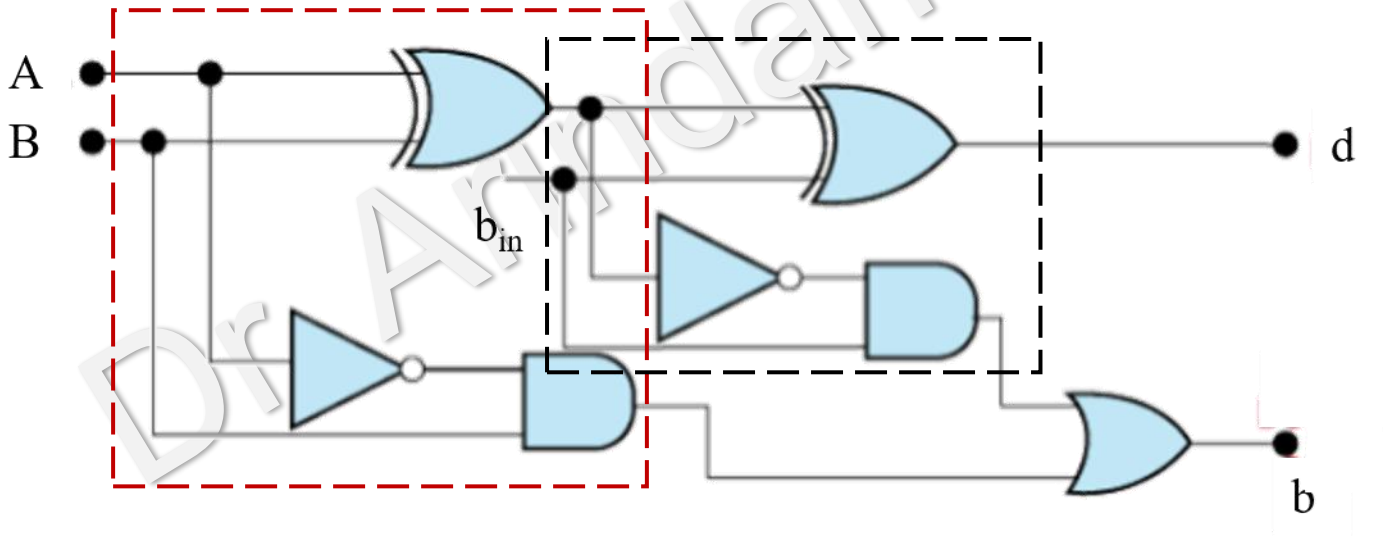
Dr. Arindam Dutta

Arithmetic Circuits

A full-subtractor can also be implemented with two half subtractors and one OR gate.

$d = A'B'b_{in} + AB'b_{in}' + A'Bb_{in}' + ABb_{in} = (A \oplus B) \oplus b_{in}$

$b = A'B'b_{in} + A'Bb_{in}' + A'Bb_{in} + ABb_{in}$ (minterms)
$= A'B\,(b_{in} + b_{in}') + (AB + A'B')\,b_{in}$
$= A'B + (A \oplus B)'\,b_{in}$



Dr Arindam Dutta

Arithmetic Circuits

Binary-Coded Decimal

- Converting decimal numbers to binary, performing arithmetic calculations in binary, and converting binary results back to decimal require storing decimal numbers in the computer so that they can be converted to binary.
- The decimal digits are represented by means of a code that contains 1's and 0's.
- Different binary codes can be obtained by arranging four bits into 10 distinct combinations.
- One such scheme is called *binary-coded decimal (BCD)* and is commonly referred to as BCD.
- A number with k decimal digits will require $4k$ bits in BCD.
- $(396)_{10}$ is represented in BCD with 12 bits as 001110010110, with each group of 4 bits representing one decimal digit.

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Dr. Arindam Dutta

Arithmetic Circuits

- A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9.
- A BCD number greater than 10 looks different from its equivalent binary number, even though both contain 1's and 0's.
- Moreover, **the binary combinations 1010 through 1111 are not used and have no meaning in BCD.**
- Consider decimal 185 and its corresponding value in BCD and binary:

$$(185)_{10} = (0001\ 1000\ 0101)_{BCD} = (10111001)_2$$

- The BCD value has 12 bits to encode the characters of the decimal value, but the equivalent binary number needs only 8 bits. It is obvious that the representation of a BCD number needs more bits than its equivalent binary value.
- However, there is an advantage in the use of decimal numbers, because computer input and output data are generated by people who use the decimal system.

Dr. Arindam Dutta

Arithmetic Circuits

BCD Adder:

- Consider the arithmetic addition of two decimal digits in BCD, together with an input carry from a previous stage. Since each input digit does not exceed 9, the output sum cannot be greater than 9 + 9 + 1 = 19, the 1 in the sum being an input carry.

- If we apply two BCD digits to a four-bit binary adder, the adder will form the sum in *binary* and produce a result that ranges from 0 through 19 (table in next slide).

- These binary numbers may be labeled by symbols K, Z_8, Z_4, Z_2, and Z_1 K is the carry, and the subscripts under the letter Z represent the weights 8, 4, 2, and 1 that can be assigned to the four bits in the BCD code.

- The columns under the binary sum list the binary value that appears in the outputs of the four-bit binary adder. The output sum of two decimal digits must be represented in BCD and should appear in the form listed in the columns under "BCD Sum."

- When the binary sum is equal to or less than 1001, the corresponding BCD number is identical, and therefore no conversion is needed (see table).

- When the binary sum is greater than 1001, addition of binary 6 (0110) to the binary sum converts it to the correct BCD representation and produces an output carry as required.

Dr. Arindam Dutta

Arithmetic Circuits

BCD Adder:

Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

Dr. Arindam Dutta

Arithmetic Circuits

Logic Circuit of BCD Adder: A correction is needed when the binary sum has an output carry $K = 1$. The other six combinations from 1010 through 1111 that need a correction have a 1 in position Z_8. To distinguish them from binary 1000 and 1001, which also have a 1 in position Z_8, we specify further that either Z_4 or Z_2 must have a 1. The condition for a correction and an output carry can be expressed by the Boolean function

$$C = K + Z_8 Z_4 + Z_8 Z_2$$

When $C = 1$, it is necessary to add 0110 to the binary sum and provide an output carry for the next stage.

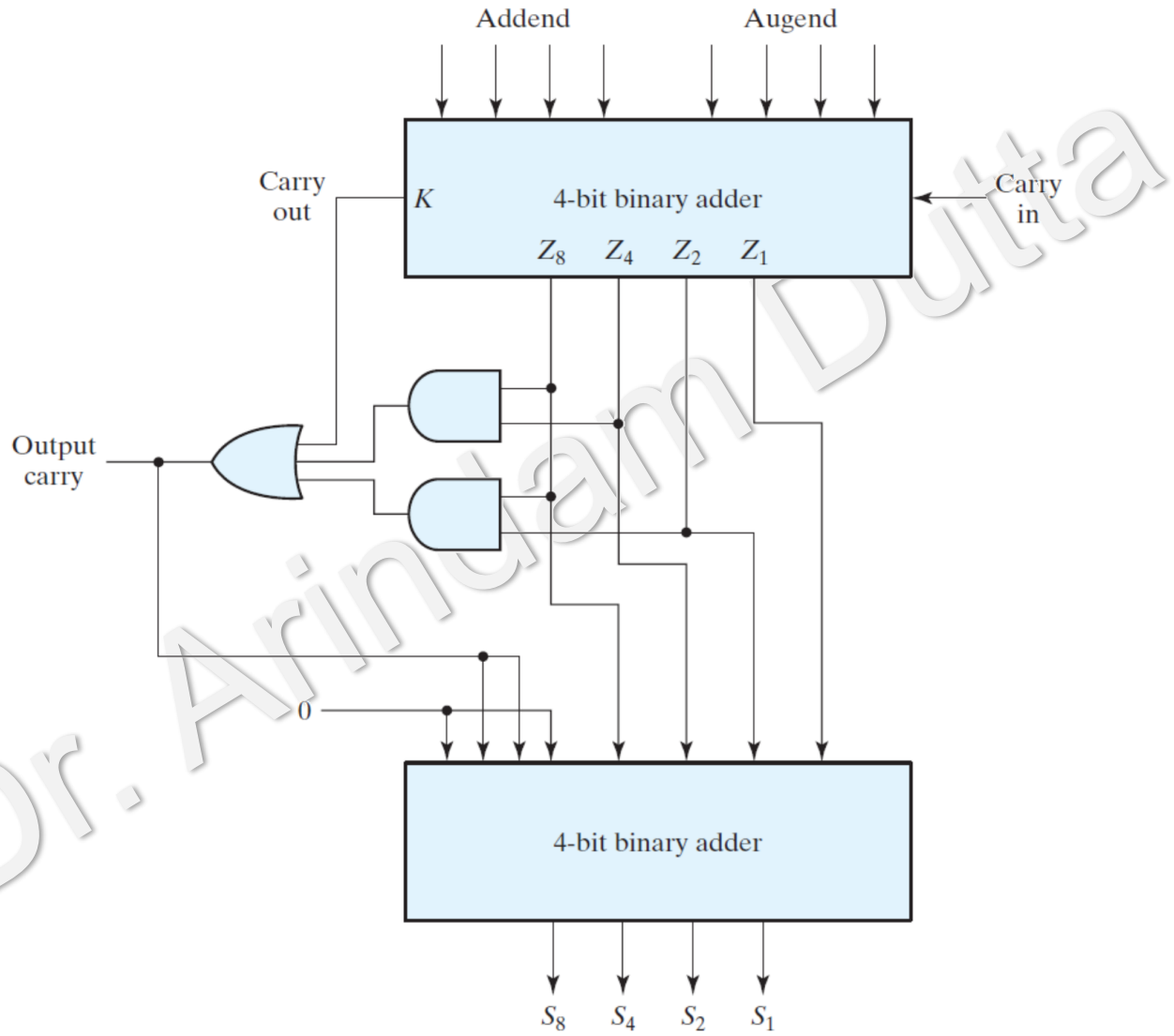
For the logic diagram (next slide):

- The two decimal digits, together with the input carry, are first added in the top four-bit adder to produce the binary sum.
- When the output carry is equal to 0, nothing is added to the binary sum. When it is equal to 1, binary 0110 is added to the binary sum through the bottom four-bit adder.
- The output carry generated from the bottom adder can be ignored, since it supplies information already available at the output carry terminal.
- A decimal parallel adder that adds n decimal digits needs n BCD adder stages. The output carry from one stage must be connected to the input carry of the next higher order stage.

Dr. Arindam Dutta

Arithmetic Circuits

Logic Circuit of BCD Adder:



Dr. Arindam Dutta

A BCD adder that adds two BCD digits and produces a sum digit in BCD

Arithmetic Circuits

Multiplier: Multiplication of binary numbers is performed in the same way as that of decimal numbers. The multiplicand is multiplied by each bit of the multiplier, starting from the LSB. Each such multiplication forms a partial product. Successive partial products are shifted one position to the left. The final product is obtained from the sum of the partial products.

- **For the multiplication of two 2-bit numbers**, the multiplicand bits are B_1 and B_0, the multiplier bits are A_1 and A_0, and the product is $C_3C_2C_1C_0$.
- The first partial product is formed by multiplying B_1B_0 by A_0. The multiplication of two bits such as A_0 and B_0 produces a 1 if both bits are 1; otherwise, it produces a 0. This is identical to an AND operation.
- The second partial product is formed by multiplying B_1B_0 by A_1 and shifting one position to the left.
- The two partial products are added with two half-adder (HA) circuits.
- The LSB of the product does not have to go through an adder, since it is formed by the output of the first AND gate.

Inputs		Output
Multiplicand (*A*)	Multiplier (*B*)	(*A.B*)
0	0	0
0	1	0
1	0	0
1	1	1

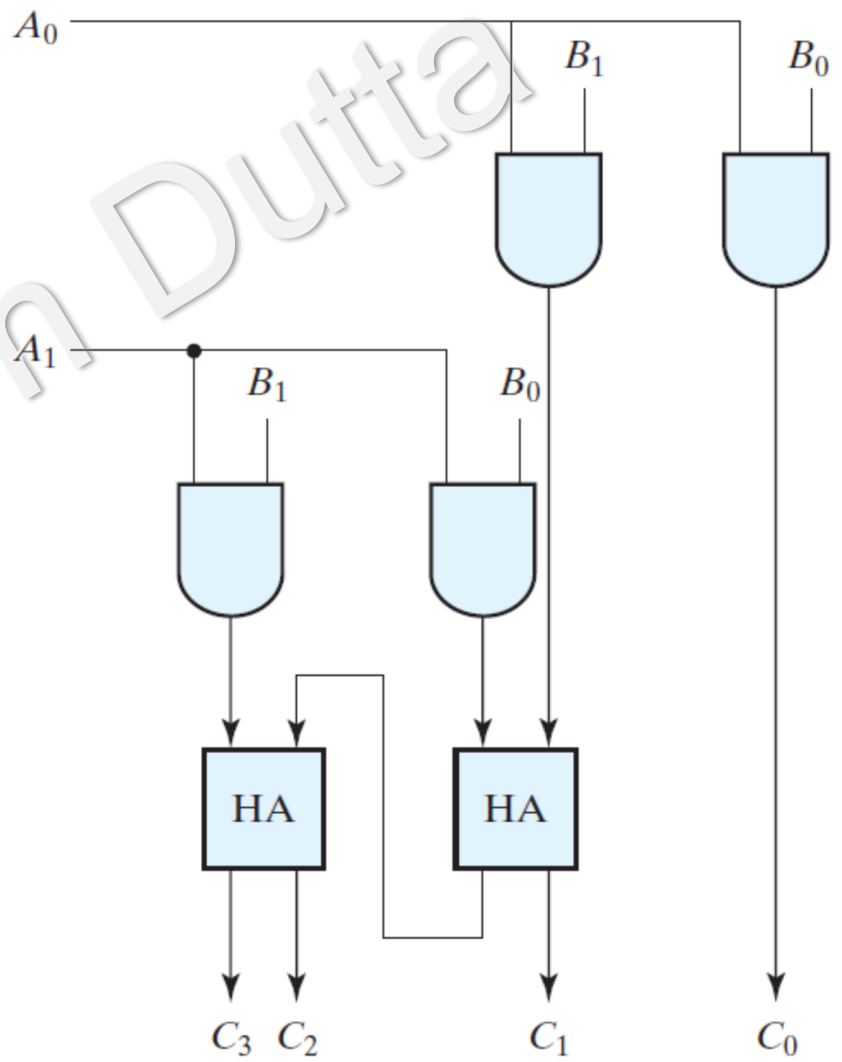
$$\begin{array}{cccc} & & B_1 & B_0 \\ & & A_1 & A_0 \\ \hline & & A_0B_1 & A_0B_0 \\ & A_1B_1 & A_1B_0 & \\ \hline C_3 & C_2 & C_1 & C_0 \end{array}$$

Dr. Arindam Dutta

Arithmetic Circuits

2-Bit Multiplier:

Inputs				Output			
Multiplicand		Multiplier					
A_1	A_0	B_1	B_0	C_3	C_2	C_1	C_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1



Dr. Arindam Dutta

ECE2002
DIGITAL LOGIC DESIGN

Module 2 – Lecture 3

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

Code Converters

Gray Code

- Continuous or analog information can be converted into digital form by means of an analog-to-digital converter by using the Gray code to represent digital data that have been converted from analog data.

- The advantage of Gray code is that only one bit in the code group changes in going from one number to the next. For example, from 7 to 8, the Gray code changes from 0100 to 1100. Only the first bit changes, from 0 to 1. With binary numbers the change from 7 to 8 will be from 0111 to 1000, which causes all four bits to change.

- If binary numbers are used, a change, for example, from 0111 to 1000 may produce an intermediate erroneous number 1001 if the value of the rightmost bit takes longer to change than do the values of the other three bits.

- The Gray code eliminates this problem, since only one bit changes its value during transition between two numbers.

Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

Dr. Arindam Dutta

Code Converters

Binary to Gray Code Converter

Decimal Numbers	Binary Code				Gray Code			
	A	B	C	D	G_3	G_2	G_1	G_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

Dr. Arindam Dutta

$$G_0 = F_1(A, B, C, D) = \Sigma m(1,2,5,6,9,10,13,14)$$

$$G_1 = F_2(A, B, C, D) = \Sigma m(2,3,4,5,10,11,12,13)$$

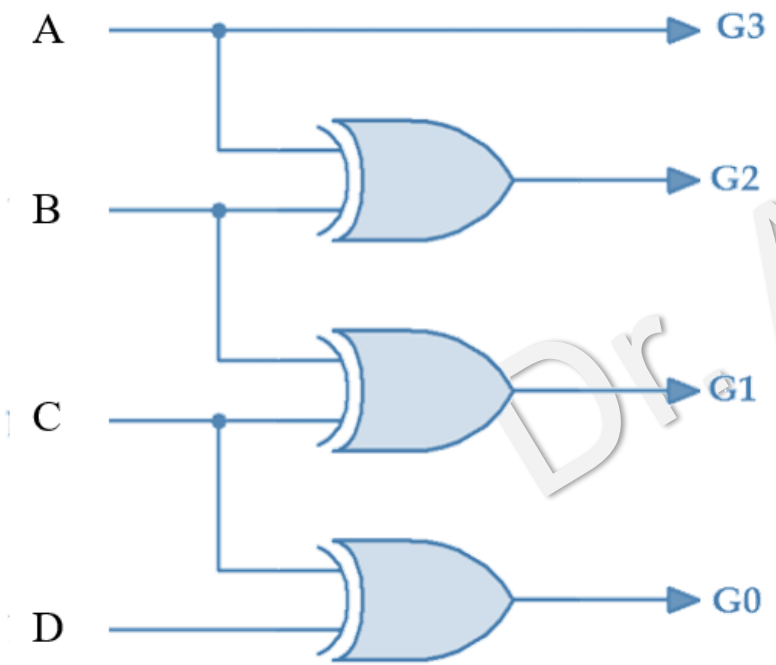
$$G_2 = F_3(A, B, C, D) = \Sigma m(4,5,6,7,8,9,10,11)$$

$$G_3 = F_4(A, B, C, D) = \Sigma m(8,9,10,11,12,13,14,15)$$

Code Converters

Binary to Gray Code Converter

$$G_0 = F_1(A, B, C, D) = \Sigma m(1,2,5,6,9,10,13,14)$$
$$G_1 = F_2(A, B, C, D) = \Sigma m(2,3,4,5,10,11,12,13)$$
$$G_2 = F_3(A, B, C, D) = \Sigma m(4,5,6,7,8,9,10,11)$$
$$G_3 = F_4(A, B, C, D) = \Sigma m(8,9,10,11,12,13,14,15)$$



AB \ CD	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

$G_0 = C'D + CD' = C \oplus D$

AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	1	1	0	0
10	0	0	1	1

$G_1 = B'C + BC' = B \oplus C$

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	1

$G_2 = A'B + AB' = A \oplus B$

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

$G_3 = A$

Dr Arindam Dutta

Code Converters

Excess-3 Code

- Excess-3 is an unweighted code in which each coded combination is obtained from the corresponding binary value plus 3.
- It is an example of self-complementing code.
- Such codes have the property that the 9's complement of a decimal number is obtained directly by changing 1's to 0's and 0's to 1's (i.e., by complementing each bit in the pattern).
- For example, decimal 395 is represented in the excess-3 code as 0110 1100 1000. The 9's complement of 395 (999 – 395 = 604) is represented as 1001 0011 0111, which is obtained simply by complementing each bit of the code (just like 1's complement of binary numbers).

Decimal Digit	Excess-3
0	0011
1	0100
2	0101
3	0110
4	0111
5	1000
6	1001
7	1010
8	1011
9	1100
Unused bit combinations	0000
	0001
	0010
	1101
	1110
	1111

Dr. Arindam Dutta

Code Converters

BCD to Excess-3 Code Converter

- Since each code (BCD and excess-3) uses four bits to represent a decimal digit, there must be four input variables and four output variables.
- The four-input variables are designated by the symbols *A, B, C,* and *D,* and the four output variables by *w, x, y* , and *z*, say.
- Out of the 16 combinations, the 6 combinations not listed for the input variables are don't-care combinations.

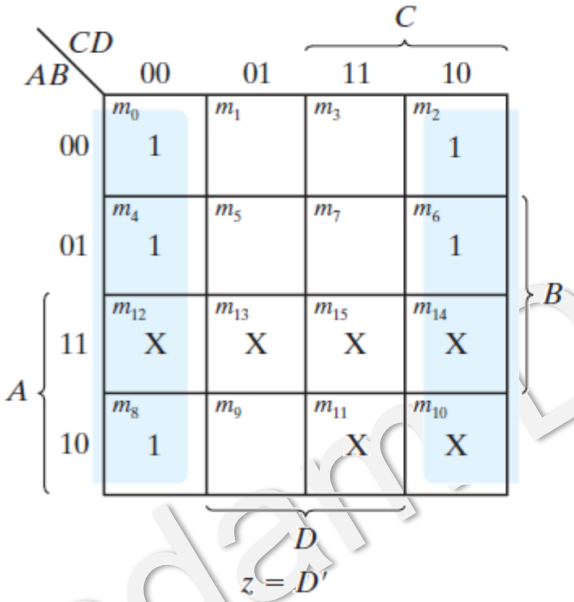
Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Dr. Arindam Dutta

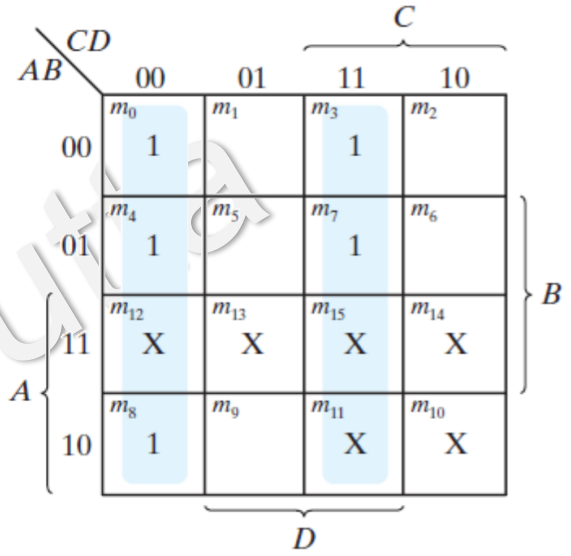
Code Converters

BCD to Excess-3 Code Converter

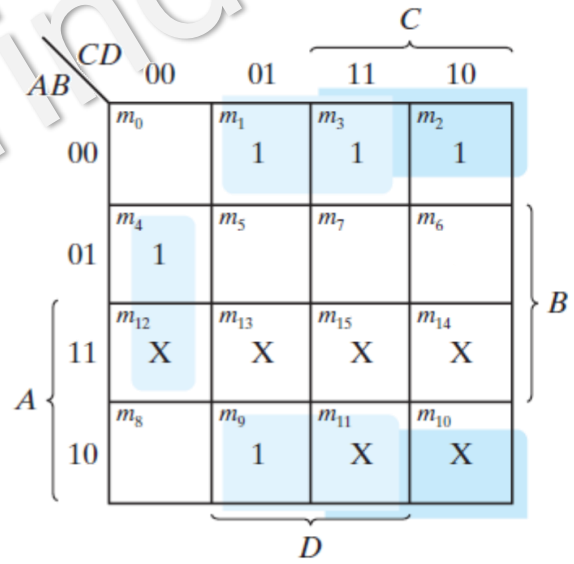
The six don't-care minterms 10 through 15 are marked with an X in the K-maps representing the four outputs



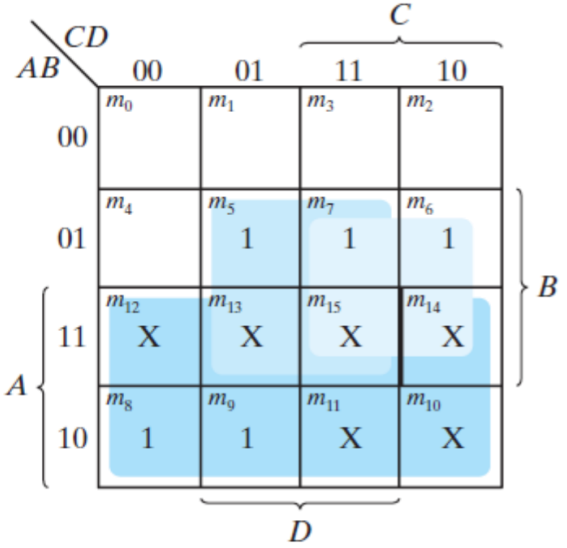
$z = D'$



$y = CD + C'D'$



$x = B'C + B'D + BC'D'$



$w = A + BC + BD$

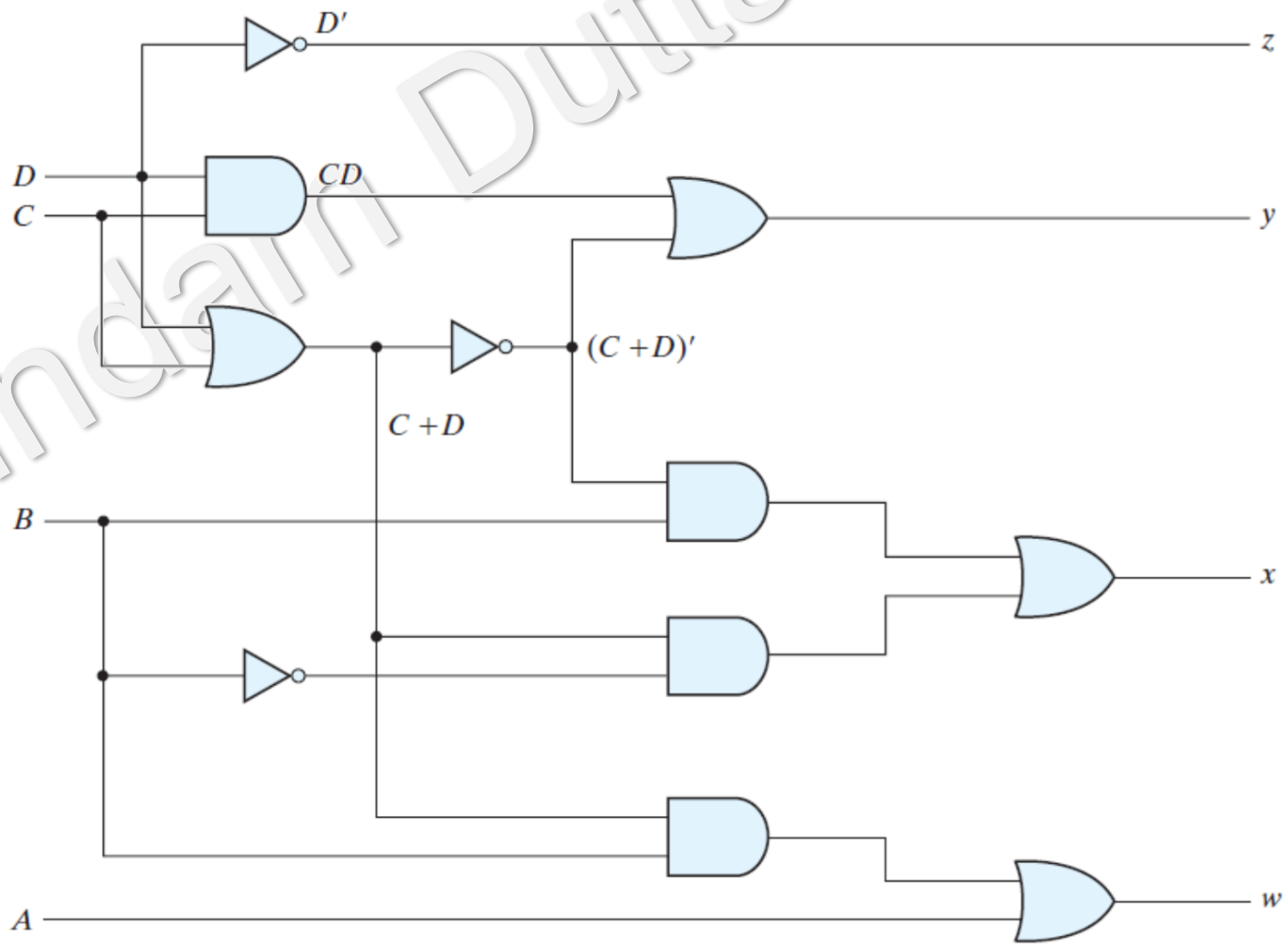
Dr. Arindam Dutta

Code Converters

BCD to Excess-3 Code Converter

There are various possibilities for a logic diagram that implements this circuit. The expressions obtained may be manipulated algebraically for the purpose of using common gates for two or more outputs.

$$z = D'$$
$$y = CD + C'D' = CD + (C + D)'$$
$$x = B'C + B'D + BC'D' = B'(C + D) + BC'D'$$
$$= B'(C + D) + B(C + D)'$$
$$w = A + BC + BD = A + B(C + D)$$



Dr. Arindam Dutta

Parity Generation and Checking

- A parity bit is an extra bit included with a binary message to make the number of 1's either odd or even.
- A parity bit is used for the purpose of detecting errors during the transmission of binary information.
- The message, including the parity bit, is transmitted and then checked at the receiving end for errors.
- An error is detected if the checked parity does not correspond with the one transmitted.
- The circuit that generates the parity bit in the transmitter is called a *parity generator.*
- The circuit that checks the parity in the receiver is called a *parity checker.*

Dr. Arindam Dutta

Parity Generation and Checking

Parity Generation

- Consider a three-bit message to be transmitted together with an even-parity bit.
- The three bits— x, y, and z —constitute the message and are the inputs to the circuit.
- The parity bit P is the output. For even parity, the bit P must be generated to make the total number of 1's (including P) even.
- From the truth table, we see that P constitutes an odd function because it is equal to 1 for those minterms whose numerical values have an odd number of 1's.

Three-Bit Message			Parity Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

- Therefore, P can be expressed as a three-variable XOR function (simplification in next slide):

$$P = x \oplus y \oplus z$$

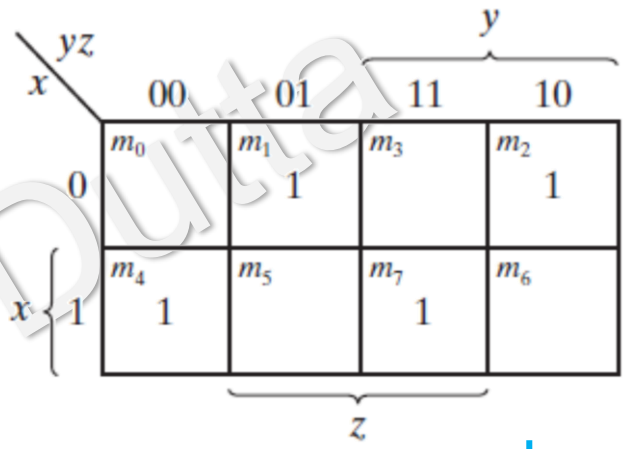
N.B. *A single XOR operation (whether with two or more inputs) always produces an* ***odd function*** *because it gives a high (1) output only when an odd number of its inputs are high.*

Dr. Arindam Dutta

Parity Generation and Checking

Parity Generation

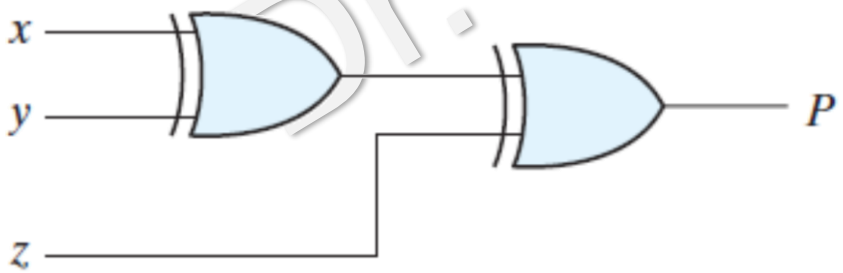
Three-Bit Message			Parity Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



$$P = x \oplus y \oplus z$$

Since, $P = z \oplus (x \oplus y)$

$$= z'(xy' + x'y) + z(xy' + x'y)'$$
$$= z'(xy' + x'y) + z(xy + x'y')$$
$$= xy'z' + x'yz' + xyz + x'y'z$$



3-bit parity generator

Dr. Arindam Dutta

Parity Generation and Checking

Parity Checking

- The three bits, along with the parity bit, are transmitted to the destination, where a parity-checker checks for possible errors in the transmission.
- Since the information was transmitted with even parity, the four bits received must have an even number of 1's.
- An error occurs in transmission if the four bits received have an odd number of 1's, meaning one bit has changed in value during transmission.
- The output of the parity checker (C) will be equal to 1 if an error occurs—that is, if the four bits received have an odd number of 1's.
- From the truth table, we see that the function C consists of eight minterms with odd number of 1's, obviously representing an odd function.
- The parity checker can be implemented with XOR gates as: $C = x \oplus y \oplus z \oplus P$

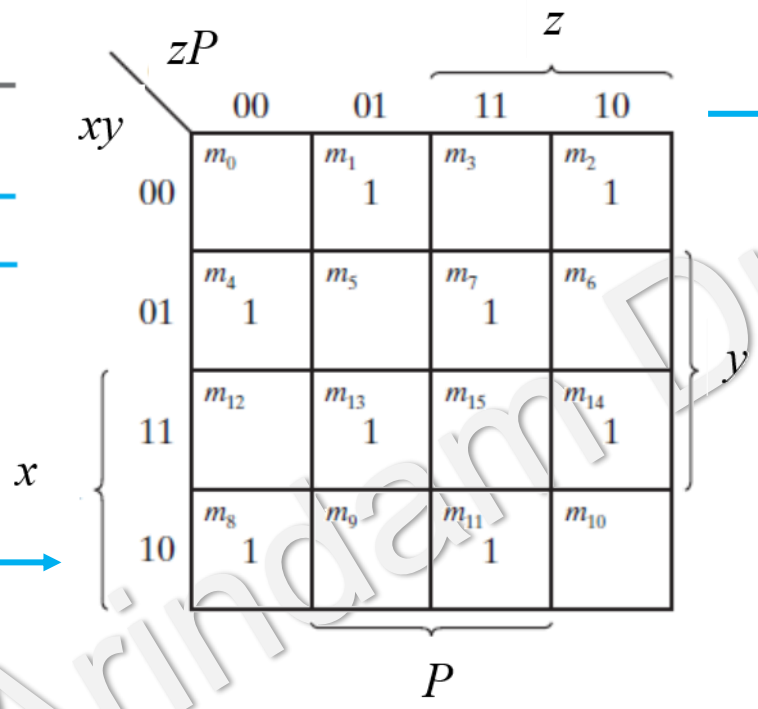
Four Bits Received				Parity Error Check
x	y	z	P	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Dr. Arindam Dutta

Parity Generation and Checking

Parity Checking

Four Bits Received				Parity Error Check
x	y	z	P	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

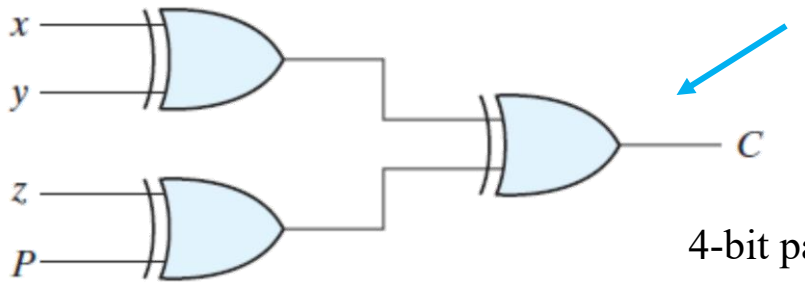


$x \oplus y \oplus z \oplus P = (xy' + x'y) \oplus (zP' + z'P)$

$= (xy' + x'y)(zP + z'P') + (xy + x'y')(zP' + z'P)$

$= xy'zP + xy'z'P' + x'yzP + x'yz'P' + xyzP' + xyz'P + x'y'zP' + x'y'z'P$

$= \Sigma (1, 2, 4, 7, 8, 11, 13, 14)$



4-bit parity checker

The parity generator can be implemented with the circuit of parity checker if the input P is connected to logic 0, because $z \oplus 0 = z$. Thus, the same circuit can be used for both parity generation and checking.

Dr. Arindam Dutta

ECE2002
DIGITAL LOGIC DESIGN

Module 2 – Lecture 4

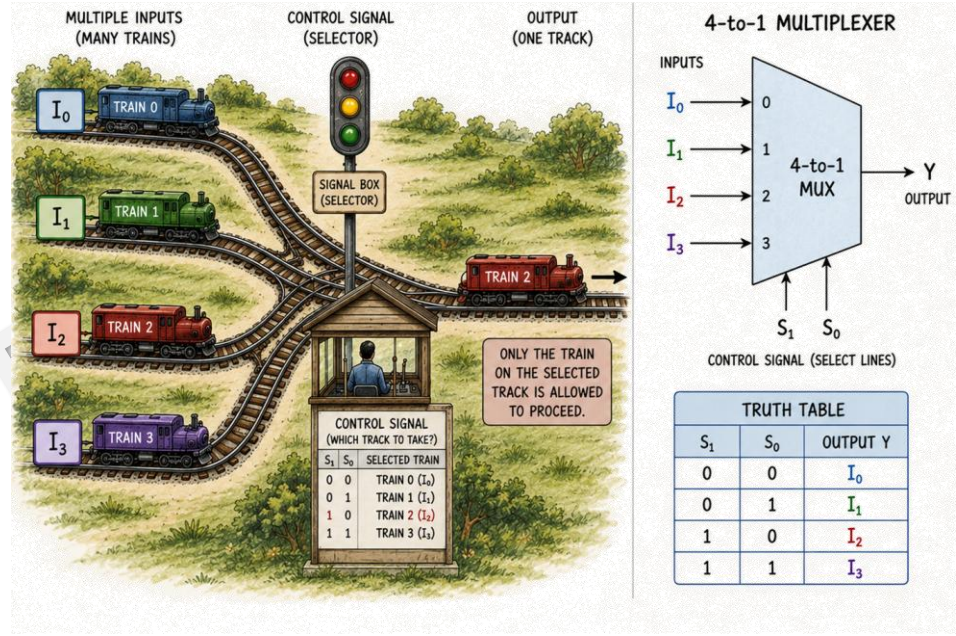
Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

Multiplexer and Demultiplexer

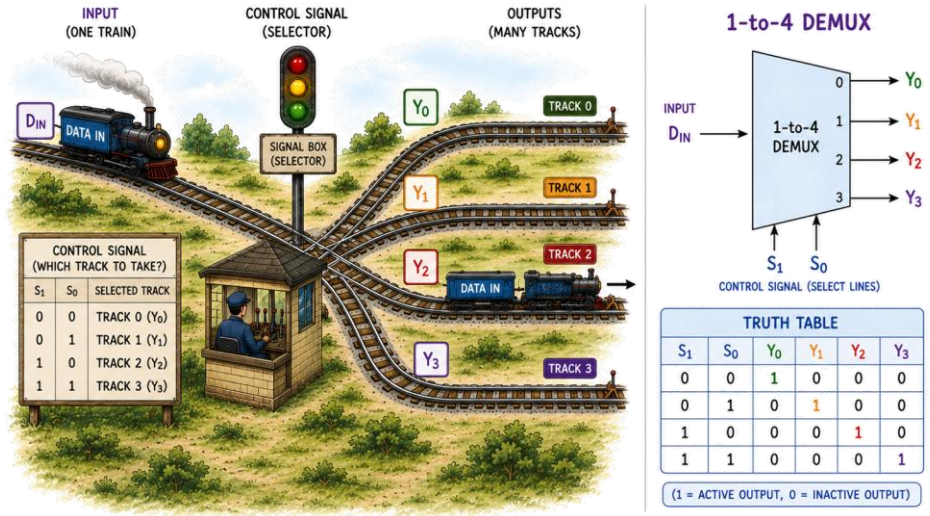
Multiplexer (MUX)

- Allows one of many input signals to be selected and transmitted to a single output line (data selector)
- Like a railway junction where multiple tracks converge. The signal controls which track the train (data) will follow



Demultiplexer (DEMUX)

- Opposite of MUX. It takes a single input signal and routes it to one of many output lines based on a control signal (data distributor)
- Like a dispatcher that sends a single train (data) to one of several possible tracks, depending on the signal



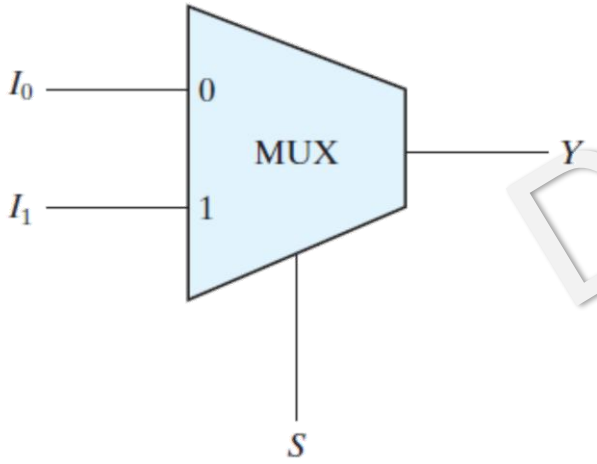
Applications: communication systems, data-acquisition systems, computer networks, etc.

Dr. Arindam

Multiplexer

- The selection of input is determined by a set of control signals or select lines
- The number of select lines (n) determines how many inputs (2^n) can be selected
- Control signals are binary signals that control which input is selected. For e.g., for 1 control line, we can select from 2 different inputs ($2^1 = 2$)

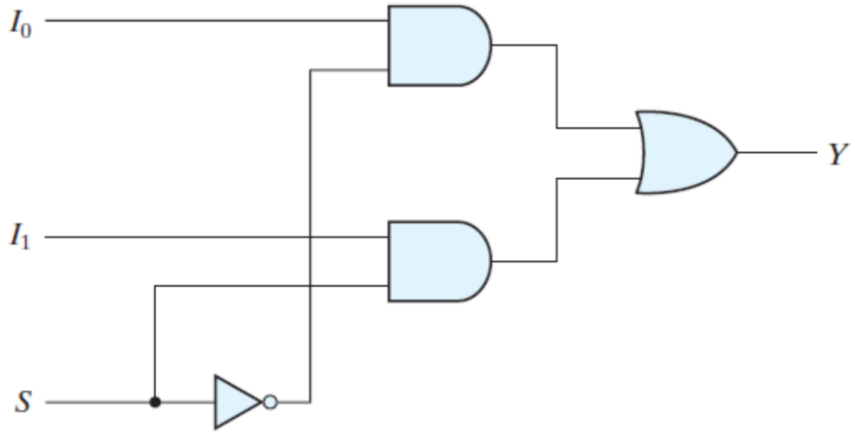
2:1 Multiplexer: If the inputs are I_0 and I_1, and the control signal is S, the output (Y) will be one of these inputs depending on S.



Truth table

S	Y
0	I_0
1	I_1

$$\mathbf{Y = \bar{S}I_0 + SI_1}$$

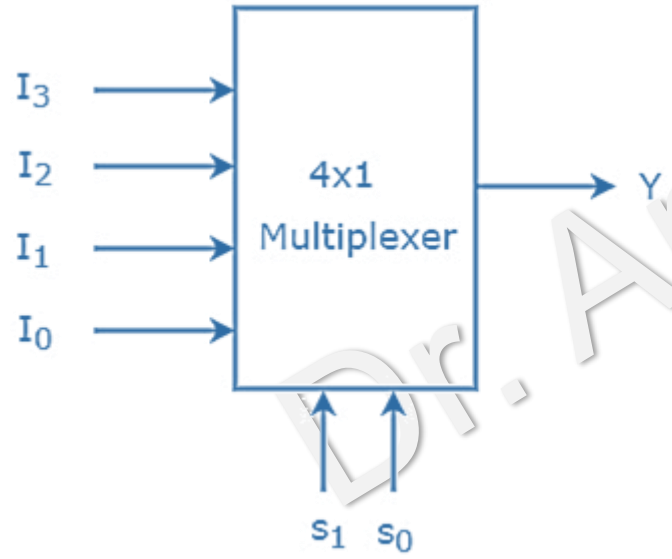


Dr. Arindam Dutta

Multiplexer

4:1 Multiplexer

- 2 control lines, allowing selection from 4 different inputs ($2^2 = 4$)
- If the inputs are I_0, I_1, I_2, and I_3, and the control signals are S_1 and S_0, the output (Y) will be one of these inputs depending on the combination of S_1 and S_0



Truth table

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Y = \overline{S_1}\,\overline{S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$

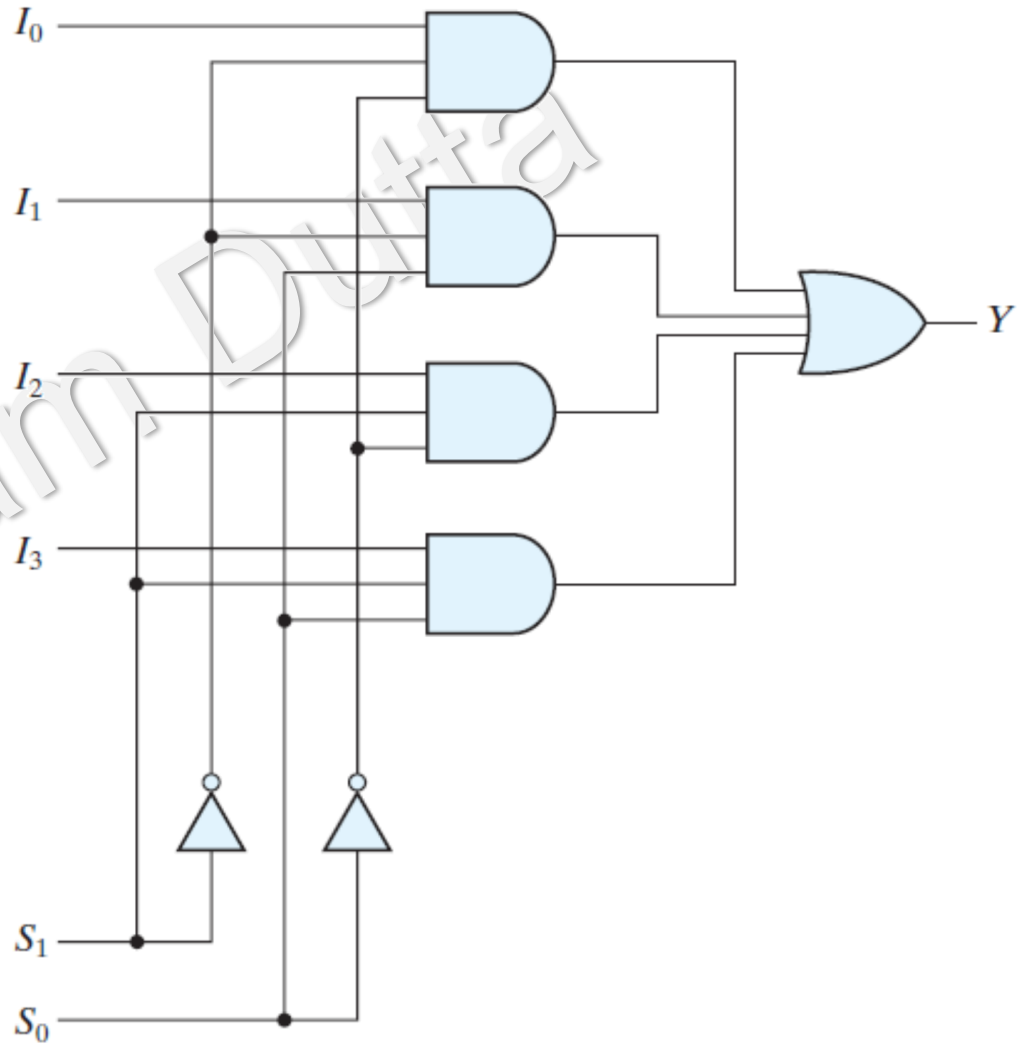
Dr. Arindam Dutta

Multiplexer

Logic Circuit of 4:1 MUX

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

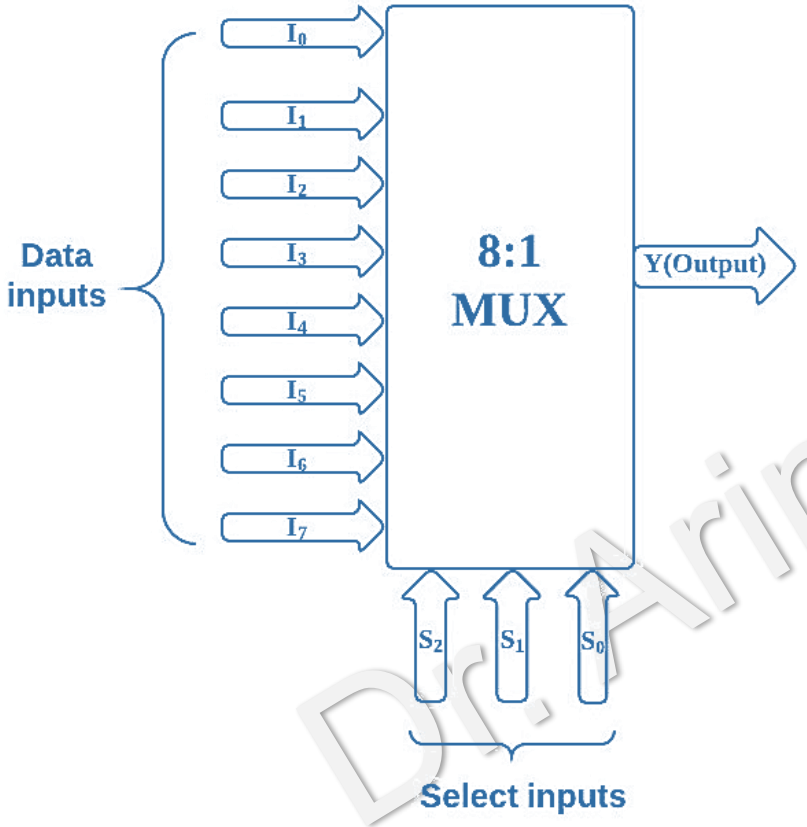
$$Y = \overline{S_1}\,\overline{S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$



Dr. Arindam Dutta

Multiplexer

8:1 Multiplexer: 3 control lines, allowing selection from 8 different inputs ($2^3 = 8$).



Truth table

S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

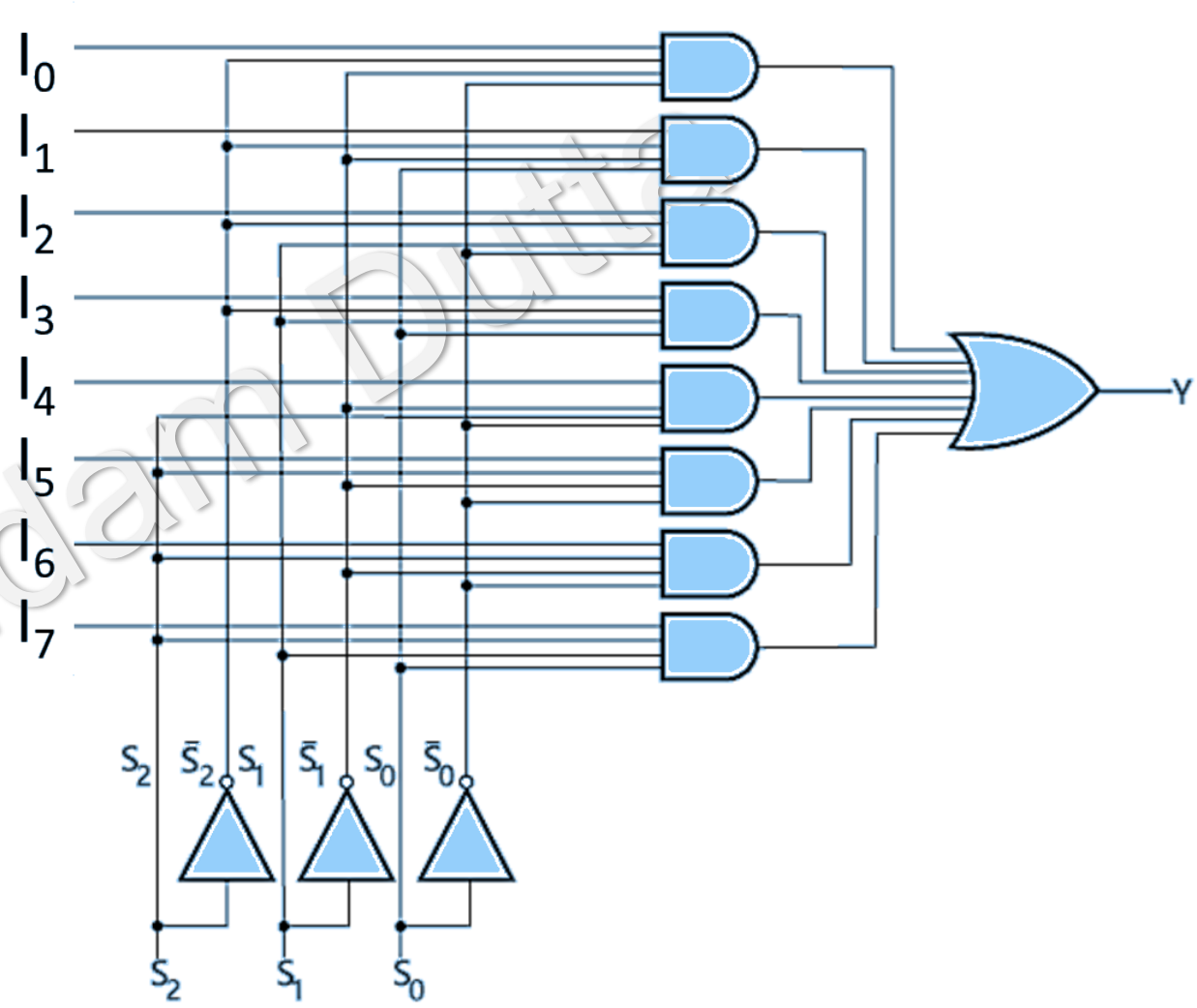
Dr. Arindam Dutta

$$Y = \overline{S_2}\,\overline{S_1}\,\overline{S_0}I_0 + \overline{S_2}\,\overline{S_1}S_0I_1 + \overline{S_2}S_1\overline{S_0}I_2 + \overline{S_2}S_1S_0I_3 + S_2\overline{S_1}\,\overline{S_0}I_4 + S_2\overline{S_1}S_0I_5 + S_2S_1\overline{S_0}I_6 + S_2S_1S_0I_7$$

Multiplexer

Logic Circuit of 8:1 MUX

S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7



Dr. Arindam Dutta

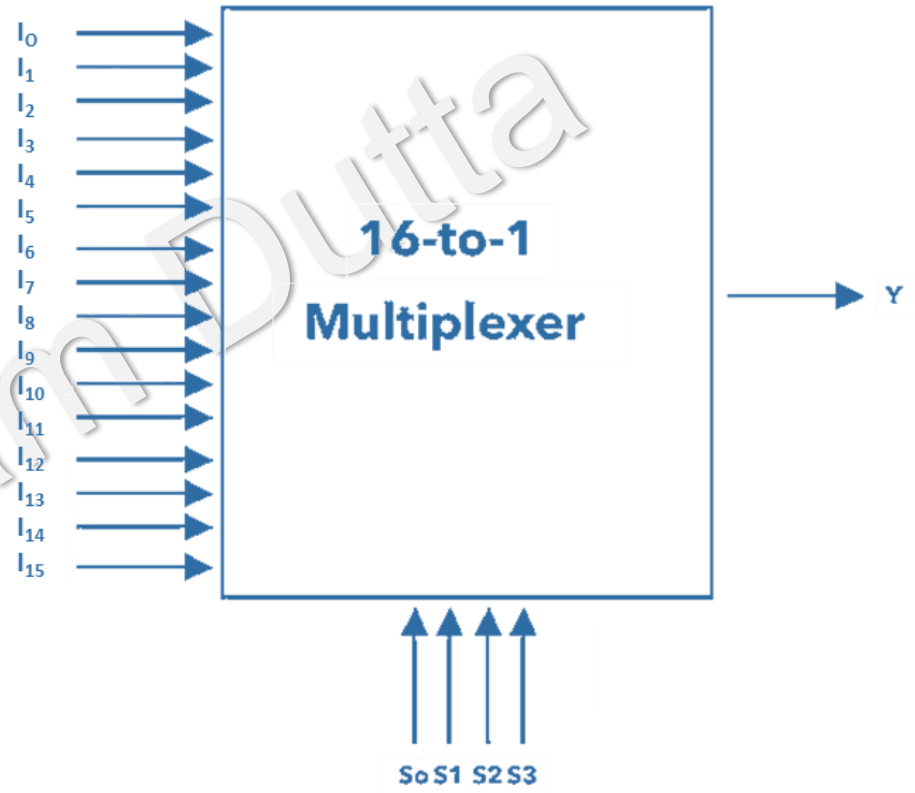
$$Y = \overline{S_2}\,\overline{S_1}\,\overline{S_0}I_0 + \overline{S_2}\,\overline{S_1}S_0I_1 + \overline{S_2}S_1\overline{S_0}I_2 + \overline{S_2}S_1S_0I_3 + S_2\overline{S_1}\,\overline{S_0}I_4 + S_2\overline{S_1}S_0I_5 + S_2S_1\overline{S_0}I_6 + S_2S_1S_0I_7$$

Multiplexer

16:1 Multiplexer: 4 control lines, allowing selection from 16 different inputs ($2^4 = 16$)

Truth table

S_3	S_2	S_1	S_0	Y
0	0	0	0	I_0
0	0	0	1	I_1
0	0	1	0	I_2
0	0	1	1	I_3
0	1	0	0	I_4
0	1	0	1	I_5
0	1	1	0	I_6
0	1	1	1	I_7
1	0	0	0	I_8
1	0	0	1	I_9
1	0	1	0	I_{10}
1	0	1	1	I_{11}
1	1	0	0	I_{12}
1	1	0	1	I_{13}
1	1	1	0	I_{14}
1	1	1	1	I_{15}

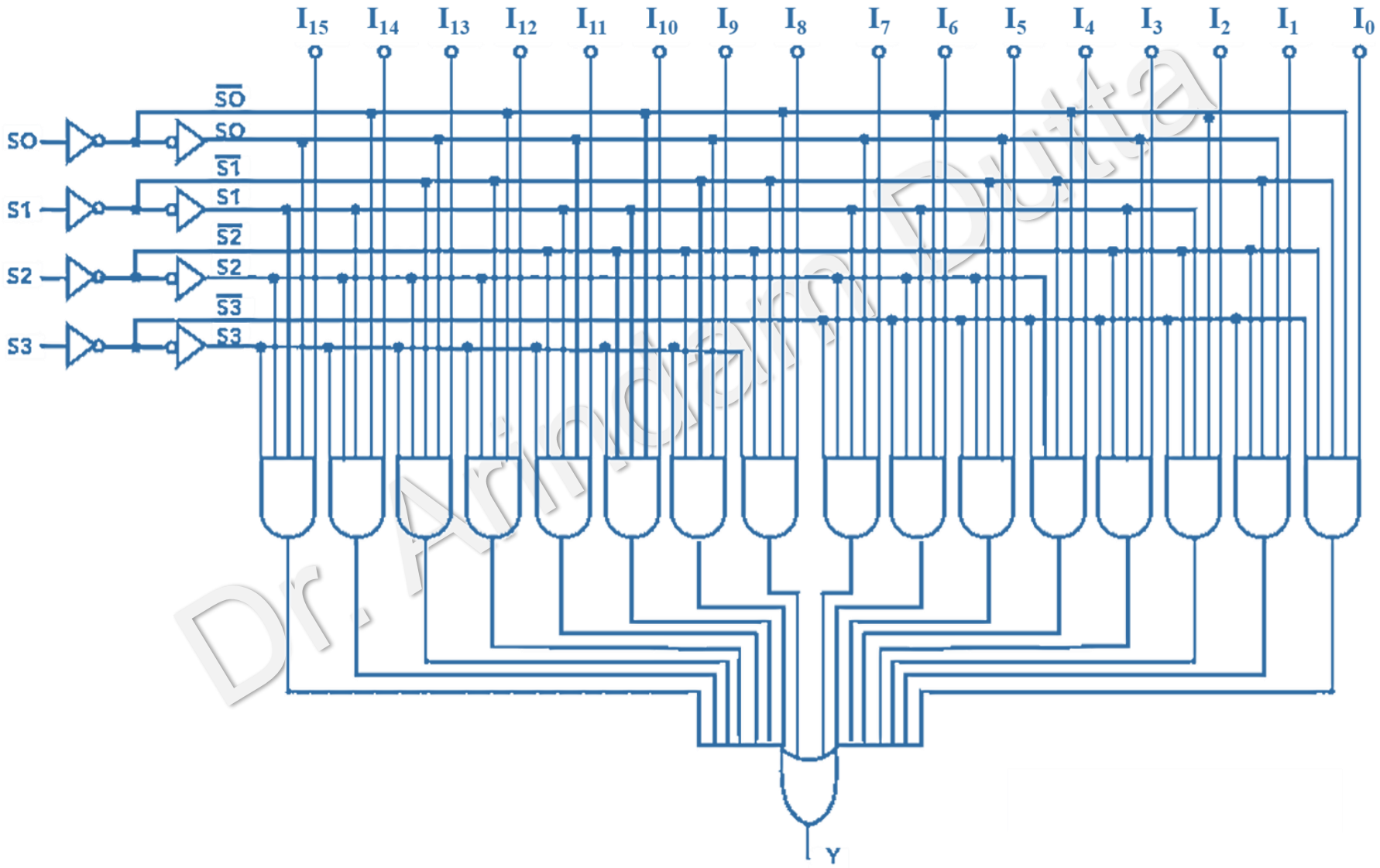


$$Y = S_0' S_1' S_2' S_3' I_0 + S_0 S_1' S_2' S_3' I_1 + S_0' S_1 S_2' S_3' I_2 + S_0 S_1 S_2' S_3' I_3 + S_0' S_1' S_2 S_3' I_4 + S_0 S_1' S_2 S_3' I_5 + S_0' S_1 S_2 S_3' I_6 + S_0 S_1 S_2 S_3' I_7 + S_0' S_1' S_2' S_3 I_8 + S_0 S_1' S_2' S_3 I_9 + S_0' S_1 S_2' S_3 I_{10} + S_0 S_1 S_2' S_3 I_{11} + S_0' S_1' S_2 S_3 I_{12} + S_0 S_1' S_2 S_3 I_{13} + S_0' S_1 S_2 S_3 I_{14} + S_0 S_1 S_2 S_3 I_{15}$$

Dr. Arindam Dutta

Multiplexer

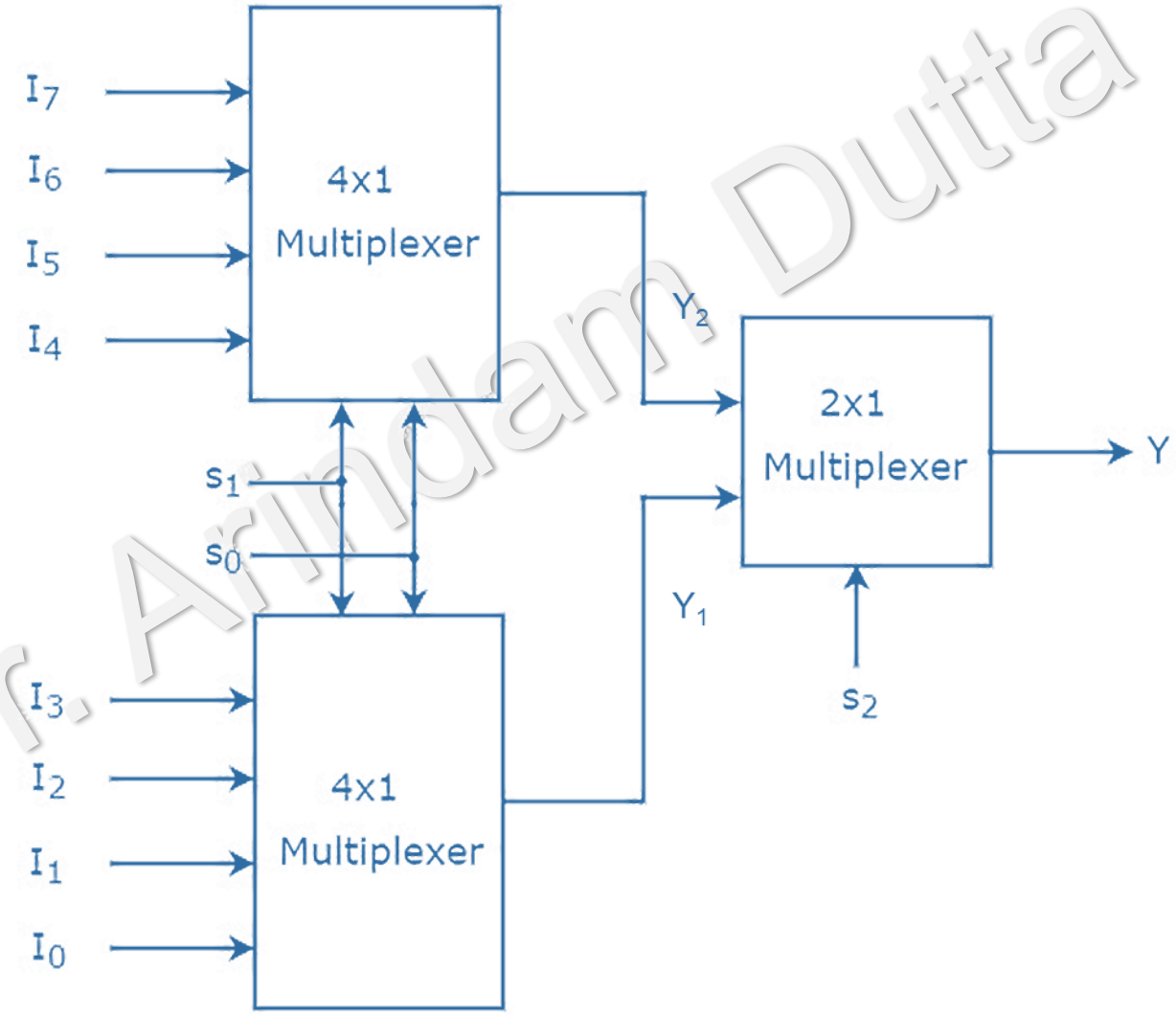
Logic Circuit of 16:1 MUX



Dr. Arindam Dutta

Multiplexer

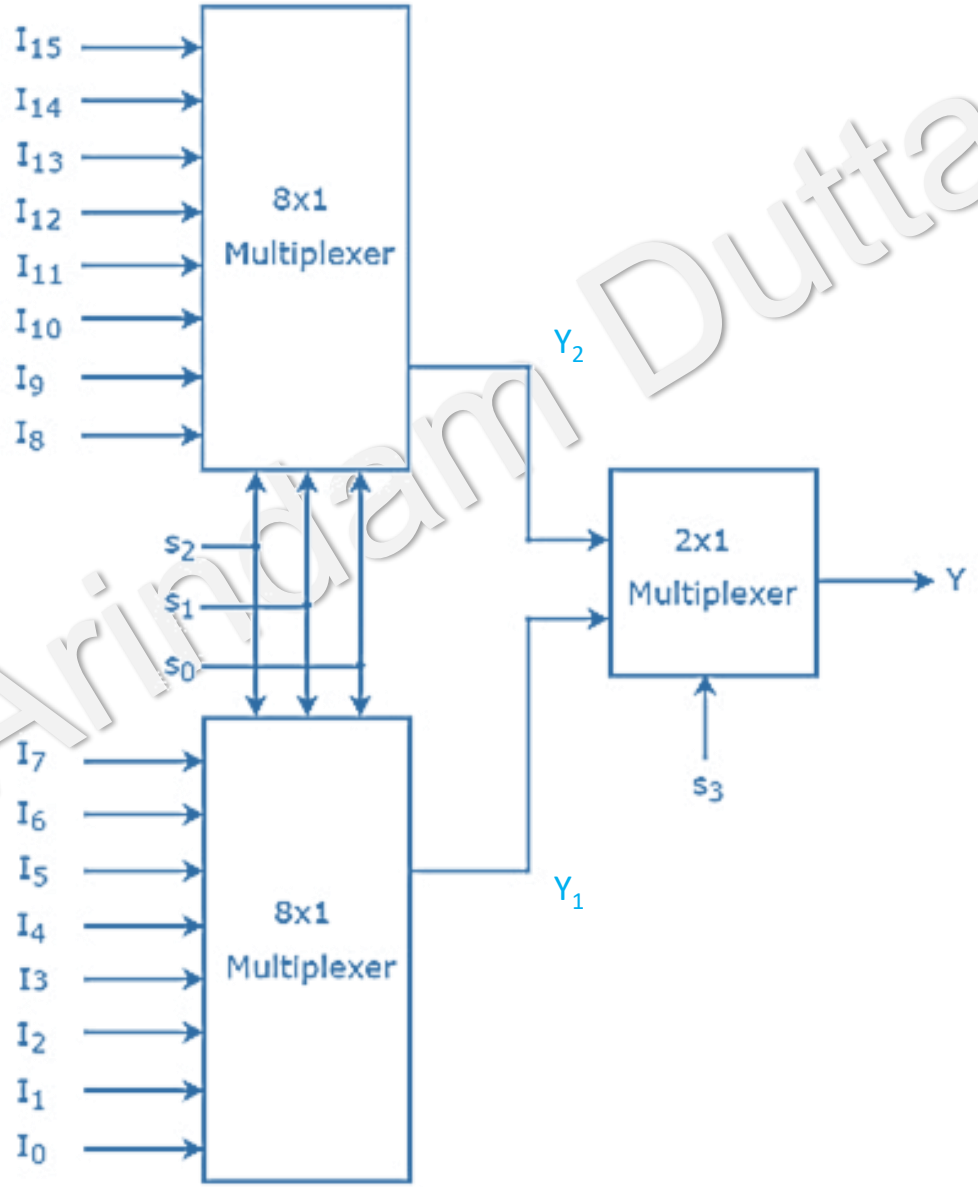
8:1 MUX Using 4:1 MUX and 2:1 MUX



Dr. Arindam Dutta

Multiplexer

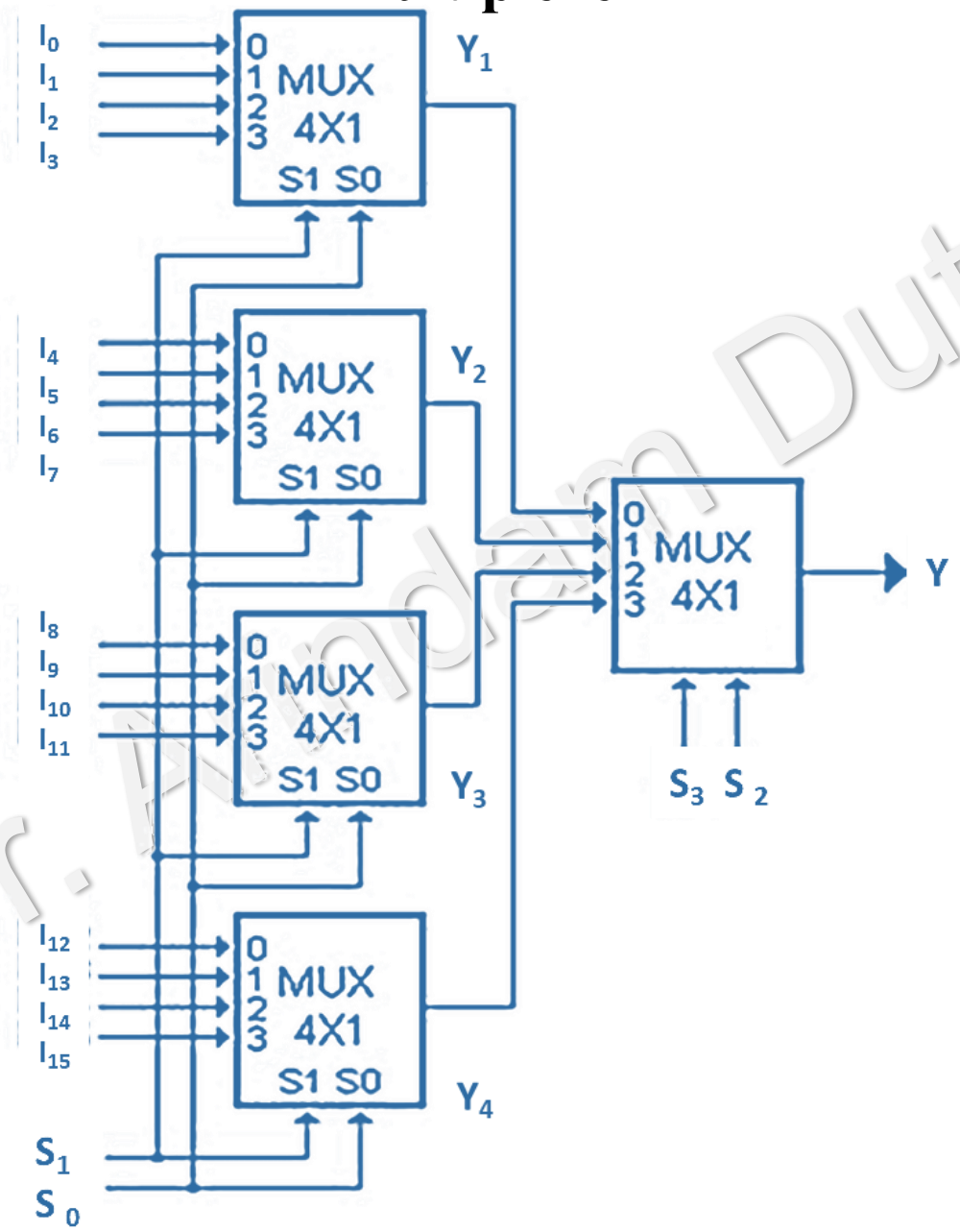
16:1 MUX Using 8:1 MUX and 2:1 MUX



Dr. Arindam Dutta

Multiplexer

16:1 MUX Using 4:1 MUX

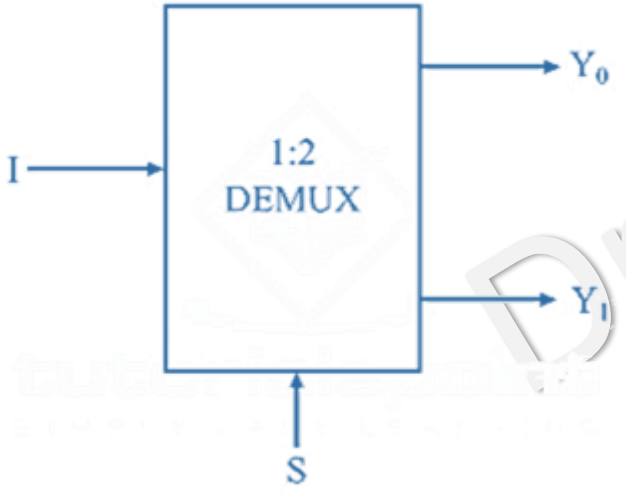


Dr. Anindam Dutta

Demultiplexer

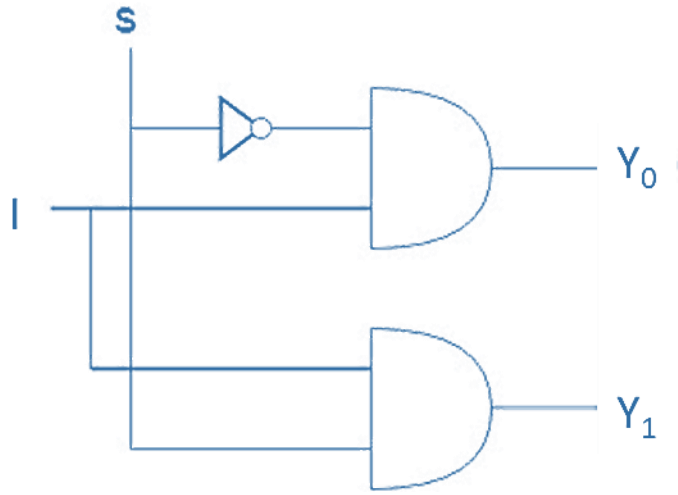
- The output line that receives the input signal is determined by the control signals or select lines
- The number of select lines (n) determines the number of outputs (2^n)

1: 2 DEMUX: For one input (I) and two outputs Y_0 and Y_1, and the control signal S, the input will be directed to one of the outputs depending on S



Truth table

S	Y_1	Y_0
0	0	I
1	I	0

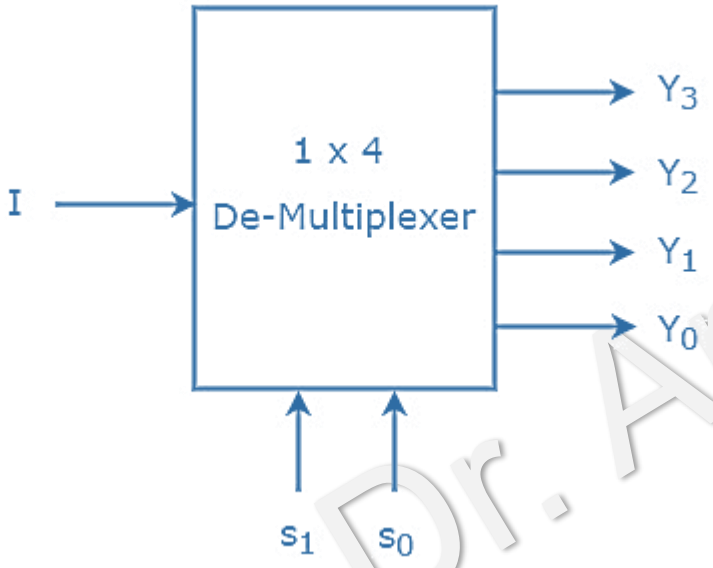


$$Y_0 = \bar{S}\,I \qquad Y_1 = S\,I$$

Dr. Arindam Dutta

Demultiplexer

1: 4 DEMUX: For one input (I) and four outputs Y_0, Y_1, Y_2, and Y_3, and the control signals S_1 and S_0, the input will be directed to one of the outputs depending on the combination of S_1 and S_0



Truth table

S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

$$Y_0 = \overline{S_1}\,\overline{S_0}I \qquad Y_1 = \overline{S_1}S_0I$$

$$Y_2 = S_1\overline{S_0}I \qquad Y_3 = S_1S_0I$$

Dr. Arindam Dutta

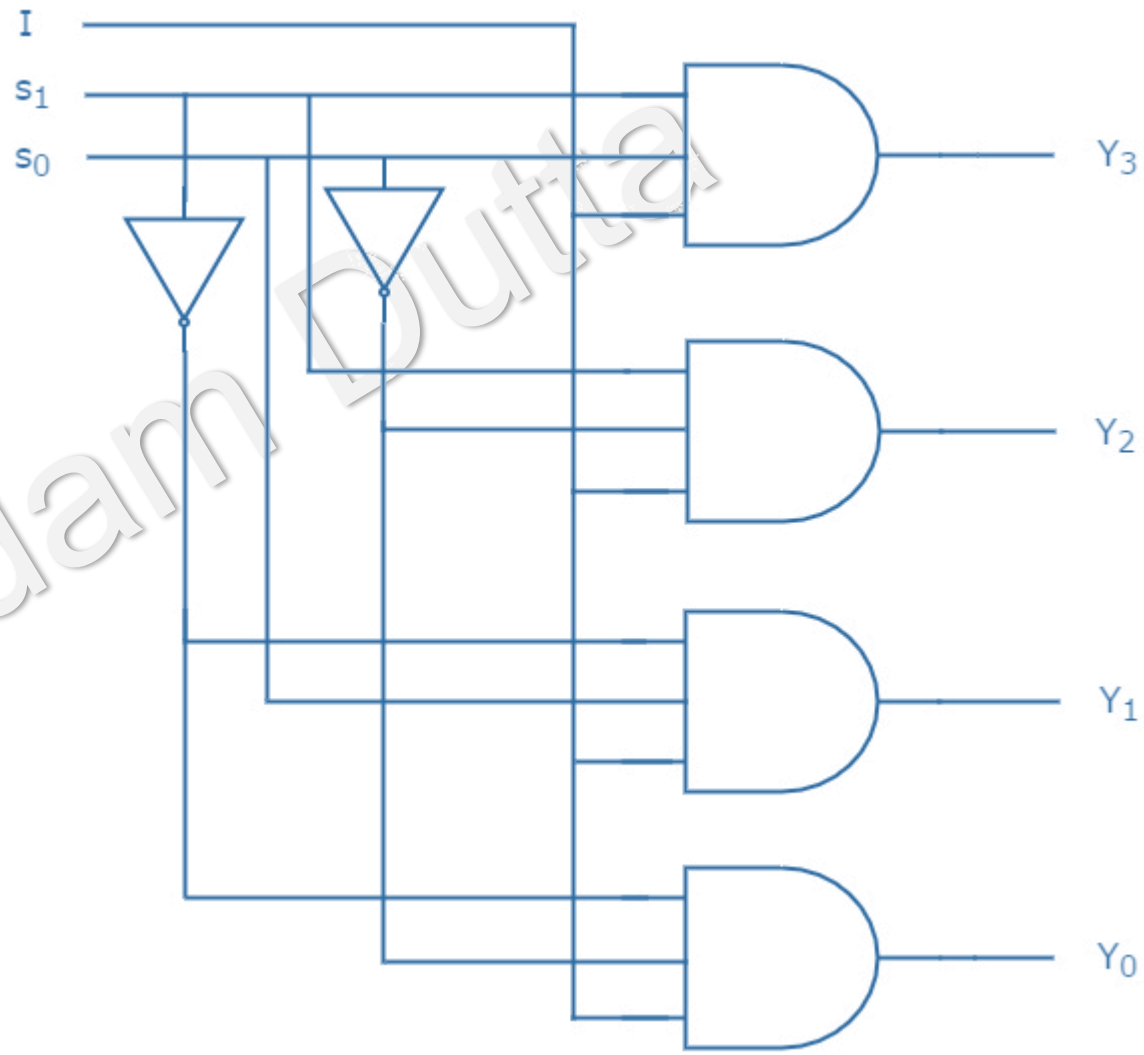
Demultiplexer

Logic Circuit of 1:4 DEMUX

S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

$$Y_0 = \overline{S_1}\,\overline{S_0}I \qquad Y_1 = \overline{S_1}S_0I$$

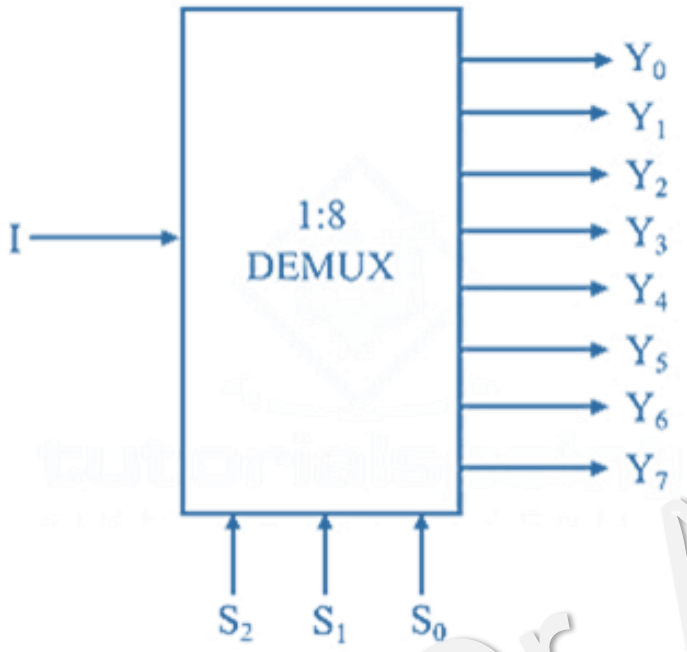
$$Y_2 = S_1\overline{S_0}I \qquad Y_3 = S_1S_0I$$



Dr. Arindam Dutta

Demultiplexer

1: 8 DEMUX: $2^3 = 8$; 3 select lines and 8 outputs



Truth table

S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	I
0	0	1	0	0	0	0	0	0	I	0
0	1	0	0	0	0	0	0	I	0	0
0	1	1	0	0	0	0	I	0	0	0
1	0	0	0	0	0	I	0	0	0	0
1	0	1	0	0	I	0	0	0	0	0
1	1	0	0	I	0	0	0	0	0	0
1	1	1	I	0	0	0	0	0	0	0

$$Y_0 = \overline{S_2}\,\overline{S_1}\,\overline{S_0}I \quad Y_2 = \overline{S_2}S_1\overline{S_0}I \quad Y_4 = S_2\overline{S_1}\,\overline{S_0}I \quad Y_6 = S_2S_1\overline{S_0}I$$

$$Y_1 = \overline{S_2}\,\overline{S_1}S_0I \quad Y_3 = \overline{S_2}S_1S_0I \quad Y_5 = S_2\overline{S_1}S_0I \quad Y_7 = S_2S_1S_0I$$

Dr. Arindam Dutta

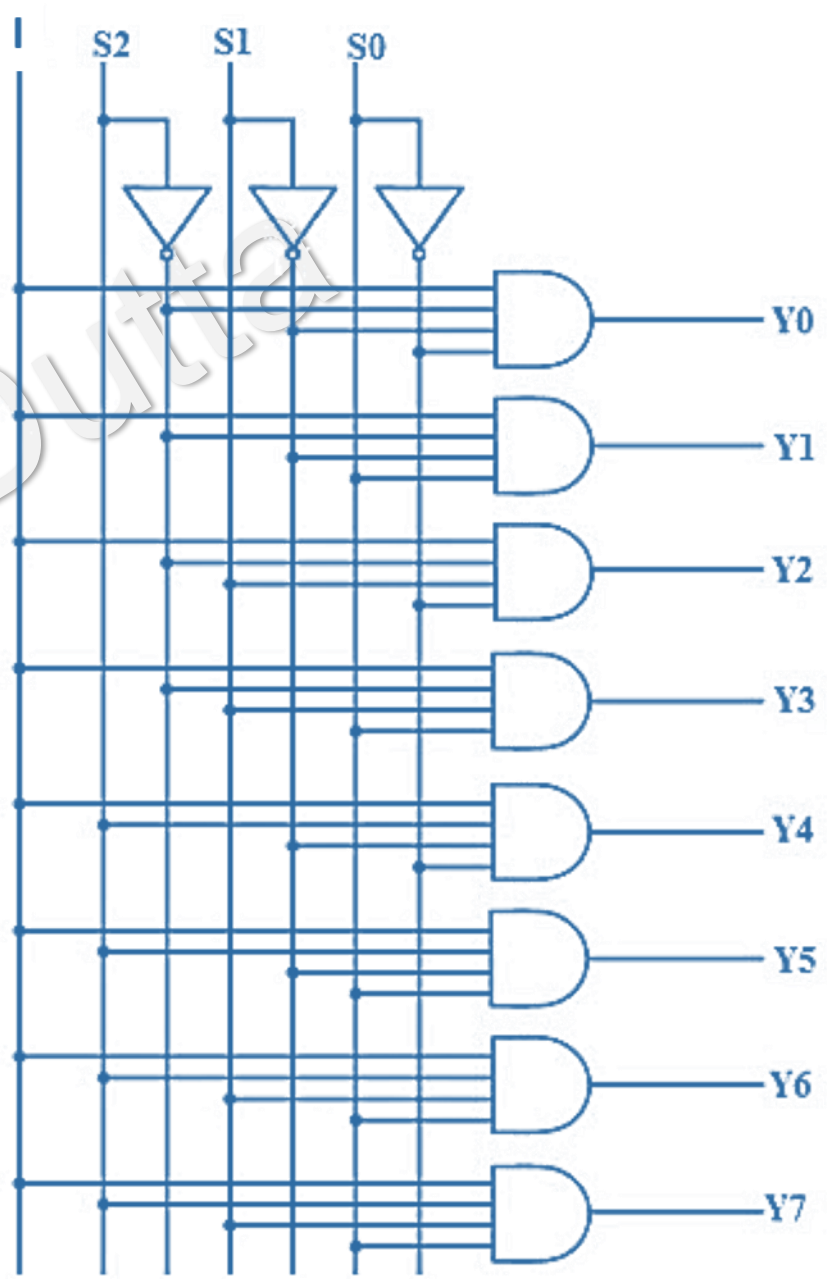
Demultiplexer

Logic Circuit of 1:8 DEMUX

S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	I
0	0	1	0	0	0	0	0	0	I	0
0	1	0	0	0	0	0	0	I	0	0
0	1	1	0	0	0	0	I	0	0	0
1	0	0	0	0	0	I	0	0	0	0
1	0	1	0	0	I	0	0	0	0	0
1	1	0	0	I	0	0	0	0	0	0
1	1	1	I	0	0	0	0	0	0	0

$$Y_0 = \overline{S_2}\,\overline{S_1}\,\overline{S_0}I \quad Y_2 = \overline{S_2}S_1\overline{S_0}I \quad Y_4 = S_2\overline{S_1}\,\overline{S_0}I \quad Y_6 = S_2S_1\overline{S_0}I$$

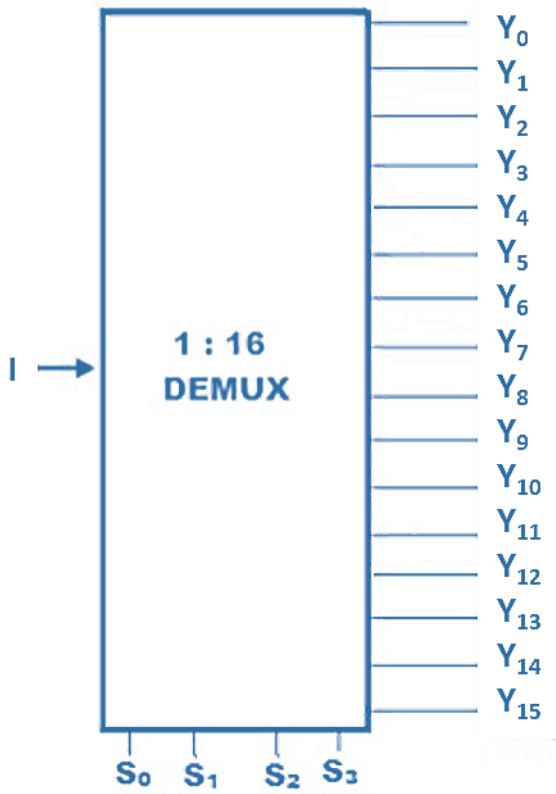
$$Y_1 = \overline{S_2}\,\overline{S_1}S_0I \quad Y_3 = \overline{S_2}S_1S_0I \quad Y_5 = S_2\overline{S_1}S_0I \quad Y_7 = S_2S_1S_0I$$



Dr. Arindam Dutta

Demultiplexer

1:16 DEMUX: $2^4 = 16$; 4 select lines and 16 outputs



INPUTS				OUTPUTS															
S_3	S_2	S_1	S_0	Y_{15}	Y_{14}	Y_{13}	Y_{12}	Y_{11}	Y_{10}	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

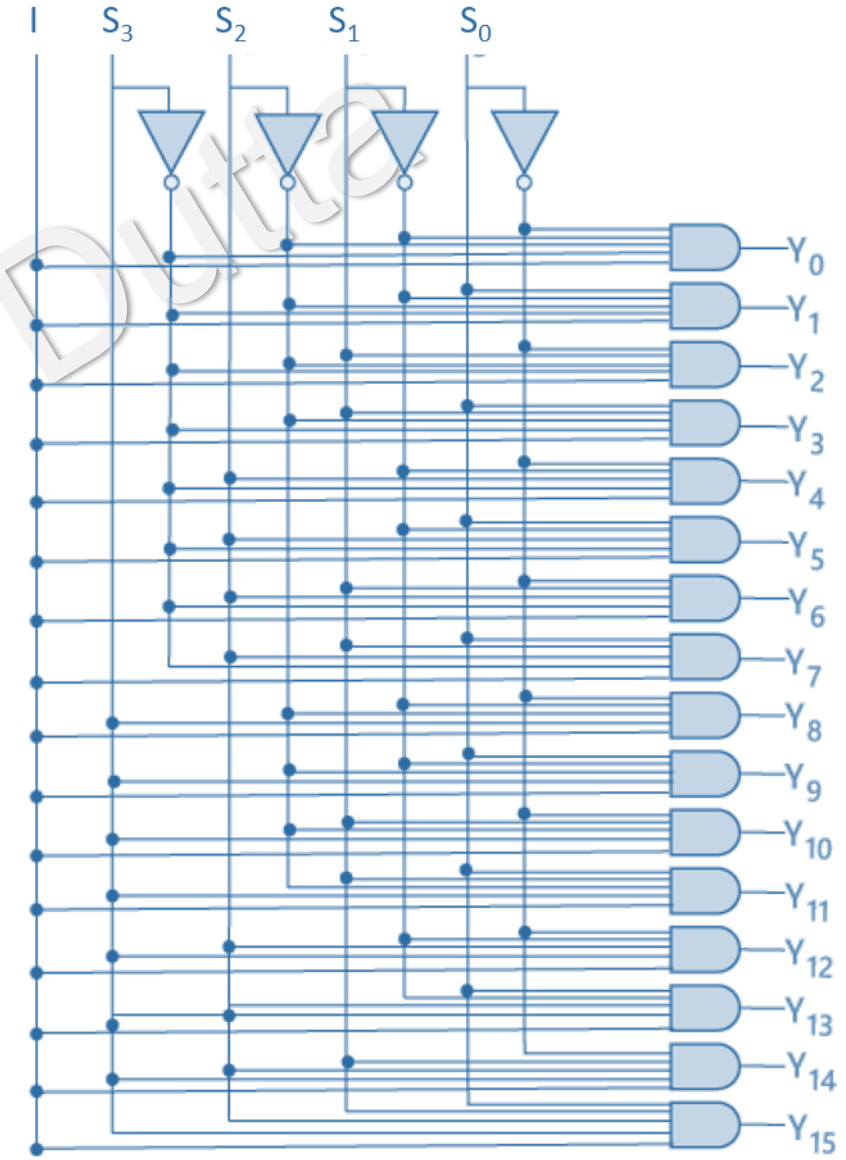
$Y_0 = I.S_0'.S_1'.S_2'.S_3'$
$Y_1 = I.S_0.S_1'.S_2'.S_3'$
$Y_2 = I.S_0'.S_1.S_2'.S_3'$
$Y_3 = I.S_0.S_1.S_2'.S_3'$
$Y_4 = I.S_0'.S_1'.S_2.S_3'$
$Y_5 = I.S_0.S_1'.S_2.S_3'$
$Y_6 = I.S_0'.S_1.S_2.S_3'$
$Y_7 = I.S_0.S_1.S_2.S_3'$
$Y_8 = I.S_0'.S_1'.S_2'.S_3$
$Y_9 = I.S_0.S_1'.S_2'.S_3$
$Y_{10} = I.S_0'.S_1.S_2'.S_3$
$Y_{11} = I.S_0.S_1.S_2'.S_3$
$Y_{12} = I.S_0'.S_1'.S_2.S_3$
$Y_{13} = I.S_0.S_1'.S_2.S_3$
$Y_{14} = I.S_0'.S_1.S_2.S_3$
$Y_{15} = I.S_0.S_1.S_2.S_3$

Dr. Arindam Dutta

Demultiplexer

Logic Circuit of 1:16 DEMUX

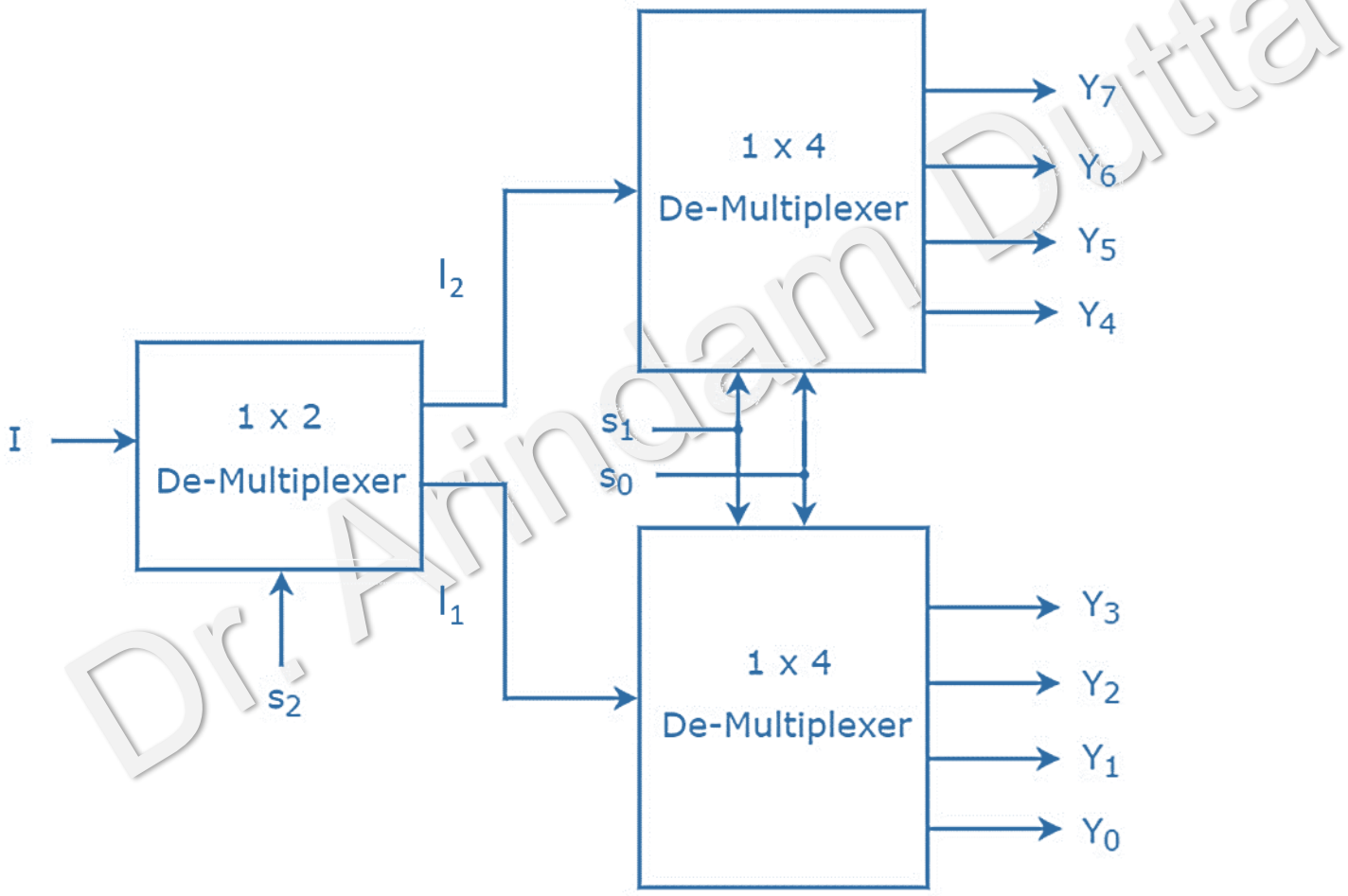
INPUTS				OUTPUTS															
S_3	S_2	S_1	S_0	Y_{15}	Y_{14}	Y_{13}	Y_{12}	Y_{11}	Y_{10}	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	I
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	I	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	I	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	I	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	I	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	I	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	I	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	I	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	I	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	I	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	I	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	I	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	I	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	I	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	I	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	I	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0



Dr. Arindam Dutta

Demultiplexer

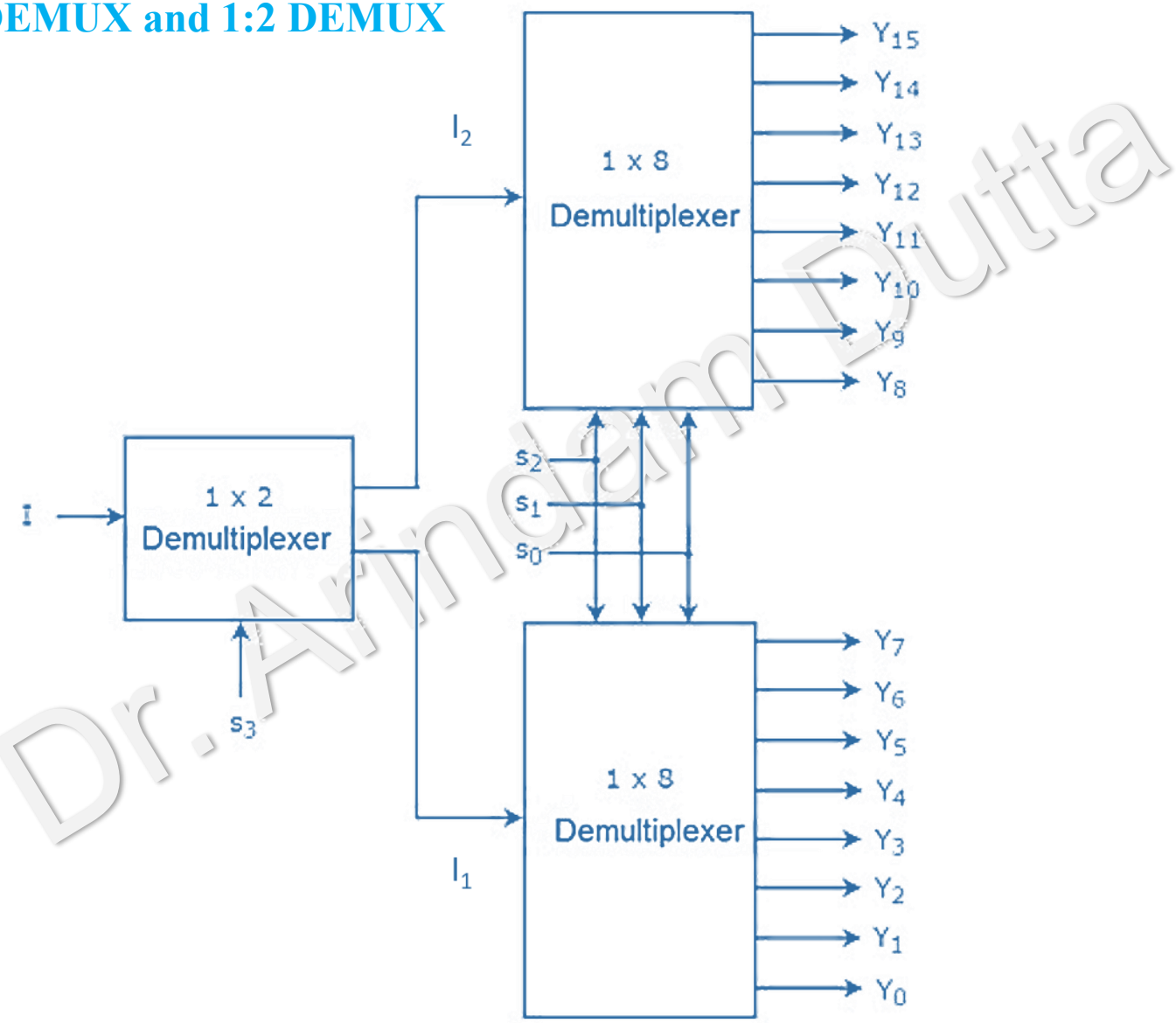
1:8 DEMUX Using 1:4 DEMUX and 1:2 DEMUX



Dr. Arindam Dutta

Demultiplexer

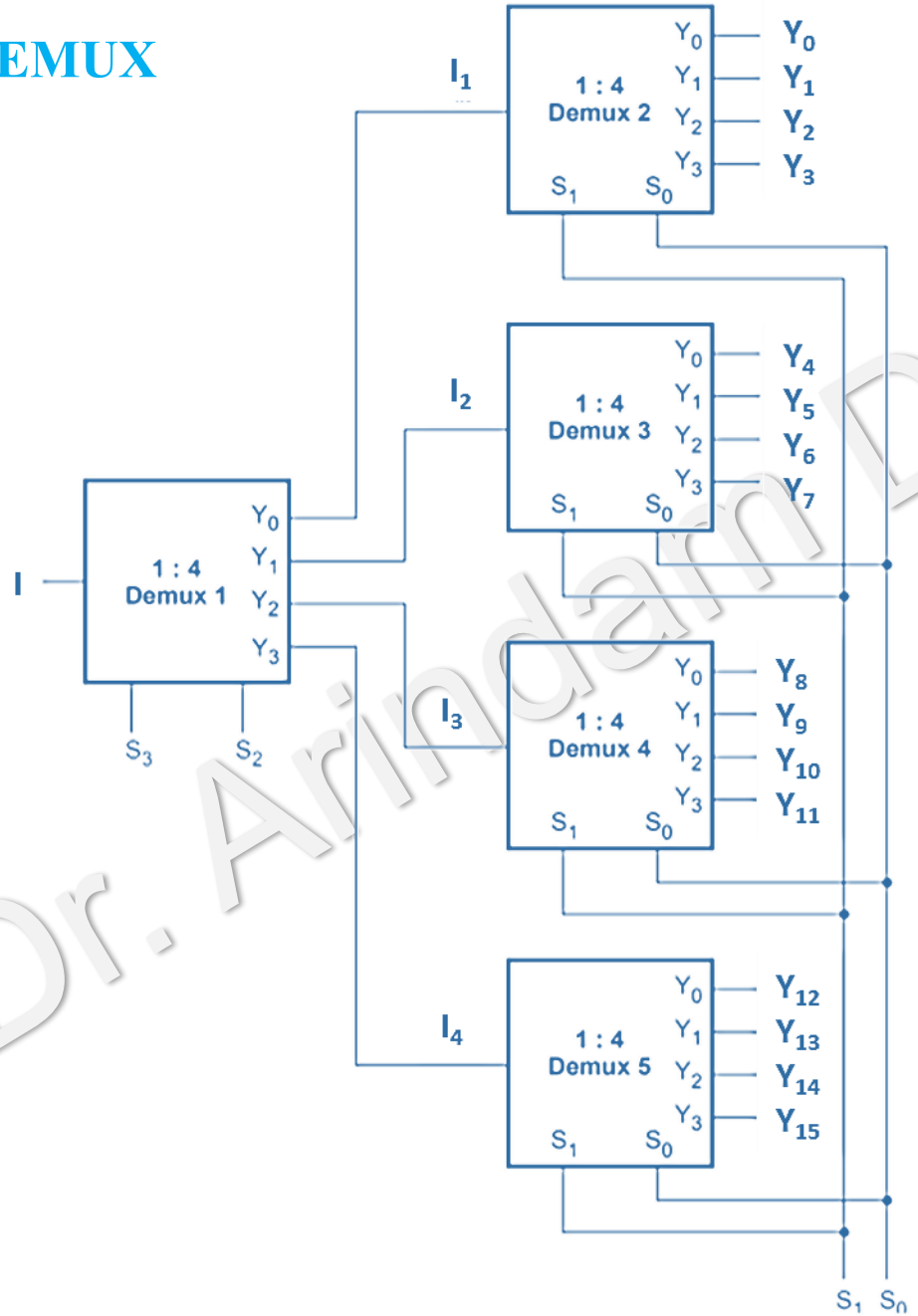
1:16 DEMUX Using 1:8 DEMUX and 1:2 DEMUX



Dr. Arindam Dutta

Demultiplexer

1:16 DEMUX Using 1:4 DEMUX

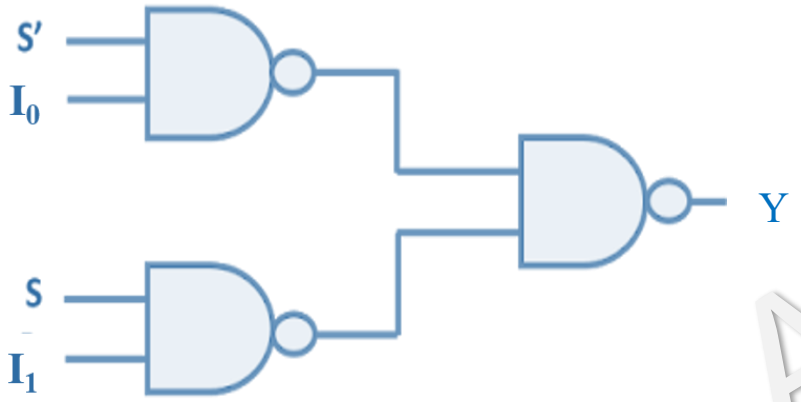


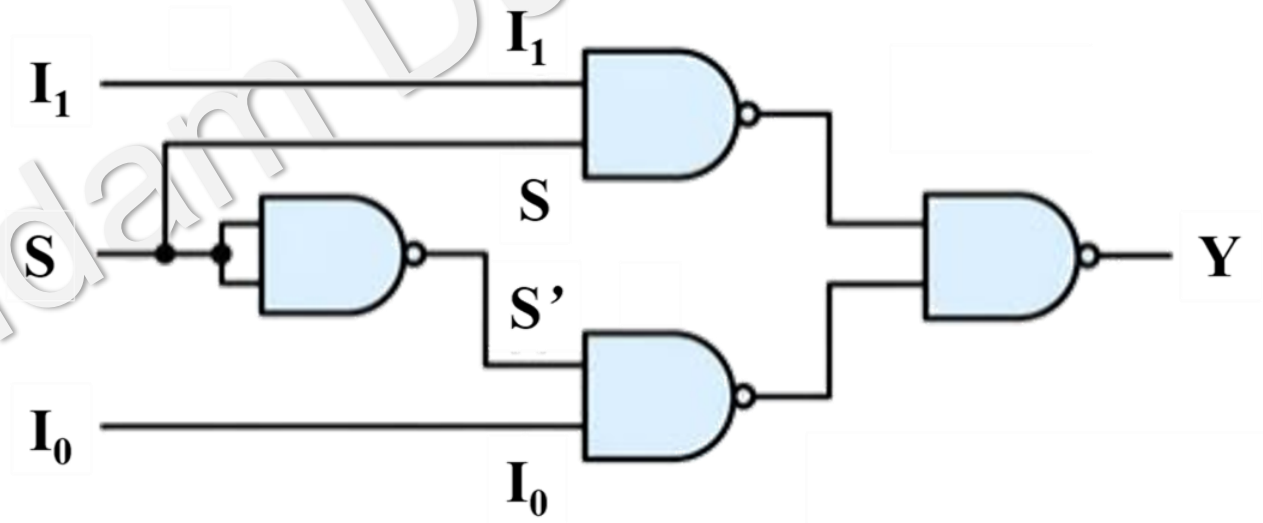
Dr. Arindam Dutta

Problem

Example: **Design a 2:1 MUX using only NAND gates.**

$$Y = \bar{S}I_0 + SI_1$$

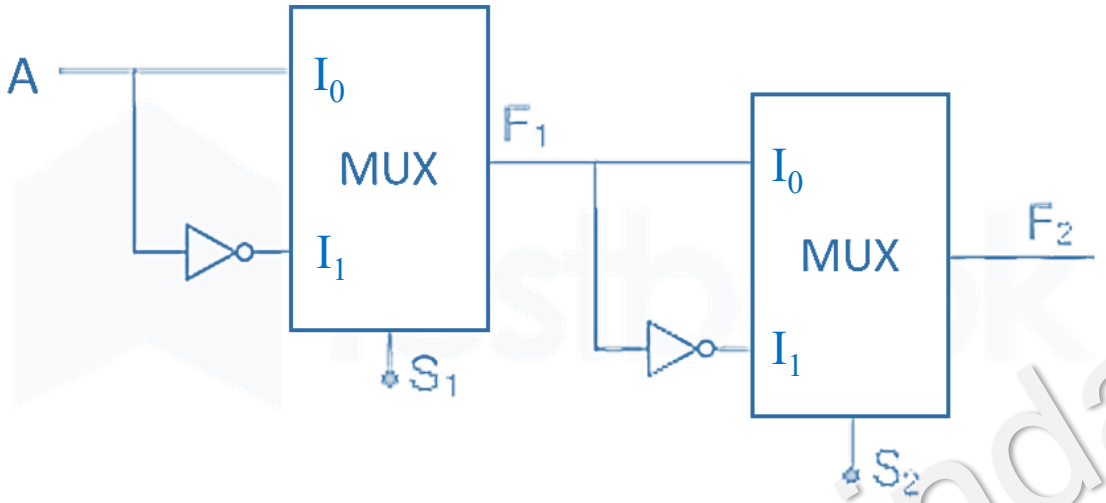




Dr. Arindam Dutta

Problem

Example: **Find the output F_2**



For a 2:1 MUX, we know:

Output, $Y = S'I_0 + SI_1$

Therefore,

$$\mathbf{F_1} = \overline{S_1}A + S_1\overline{A} = S_1 \oplus A$$

$$\mathbf{F_2} = \overline{S_2}F_1 + S_2\,\overline{F_1}$$

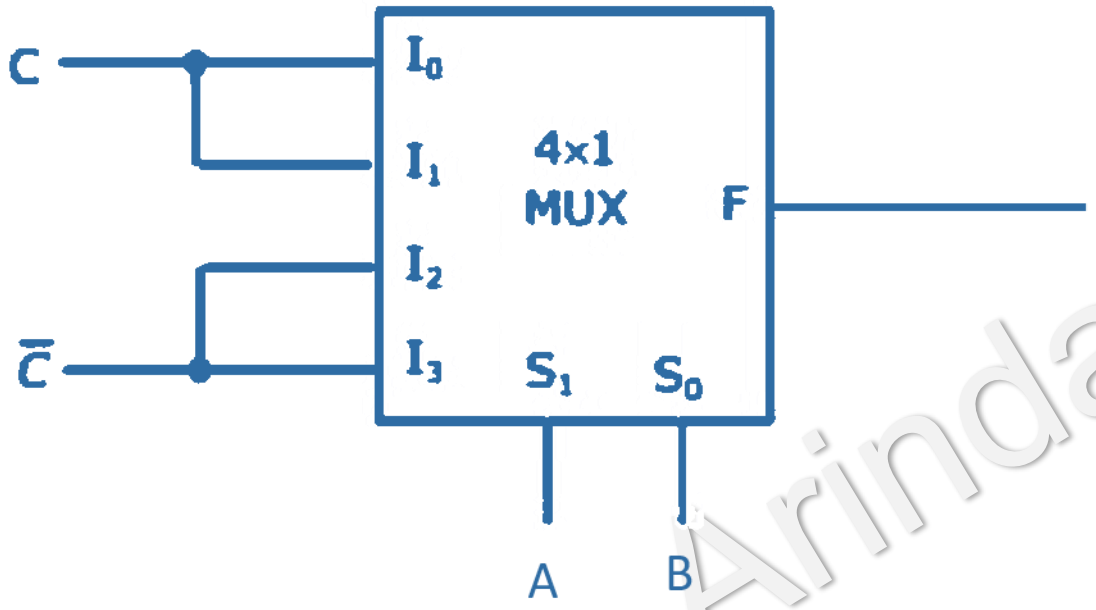
$$= S_2 \oplus F_1$$

$$= S_2 \oplus S_1 \oplus A$$

Dr. Arindam Dutta

Problem

Example: **Find the output F.**



Dr. Arindam Dutta

$$F = \bar{S}_1\bar{S}_0I_0 + \bar{S}_1S_0I_1 + S_1\bar{S}_0I_2 + S_1S_0I_3$$

$$= \bar{A}\,\bar{B}\,C + \bar{A}\,B\,C + A\bar{B}\,\bar{C} + AB\,\bar{C}$$

$$= \bar{A}C\,(\bar{B}+B) + A\bar{C}\,(\bar{B}+B)$$

(Distributive law)

$$= \bar{A}C + A\bar{C}$$

$$F = A \oplus C$$

Problem

Example: Design an AND gate using 2:1 MUX.

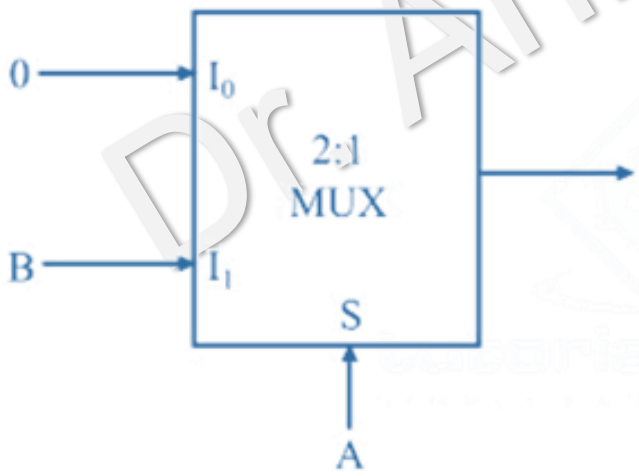
Taking A as the select line and B as the input variable:



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Y = 0 when A = 0

Y = B when A = 1



$Y = A'.0 + A \cdot B = AB$

Dr Arindam Dutta

Problem

Example: Design a NAND gate using 2:1 MUX.

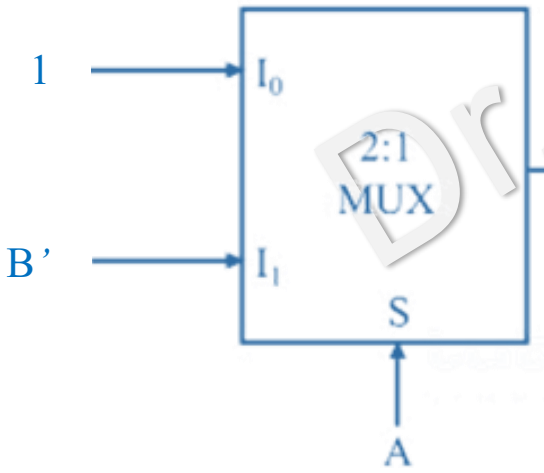
Taking A as the select line and B as the input variable:



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Y = 1 when A = 0

Y = B ' when A = 1



$Y = A'.1 + A \cdot B' = A' + AB' = A' + AB' + B'$ (Consensus)

$= A' + (A+1)\,B'$ (Distributive)

$= A' + B'$ (Null)

$= (AB)'$ (De-Morgan's)

Dr. Arindam Dutta

Problem

Example: Design an OR gate using 2:1 MUX.

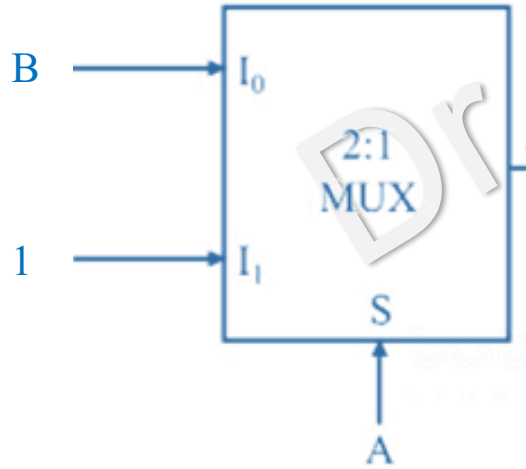
Taking A as the select line and B as the input variable:



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Y = B when A = 0

Y = 1 when A = 1



$= A'.B + A \cdot 1 = A'B + A = A'B + A + B$ (Consensus)

$= A + (A'+1)\,B$ (Distributive)

$= A + B$ (Null)

Dr. Arindam Dutta

Problem

Example: Design a NOR gate using 2:1 MUX.

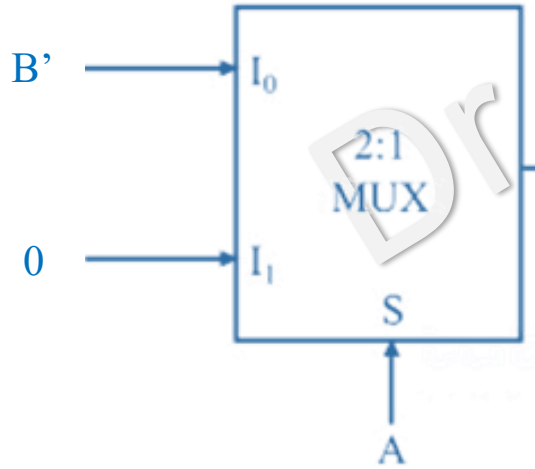
Taking A as the select line and B as the input variable:



A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Y = B' when A = 0

Y = 0 when A = 1



$Y = A'.B' + A \cdot 0 = A'B' = (A + B)'$ (De-Morgan's)

Dr. Arindam Dutta

Problem

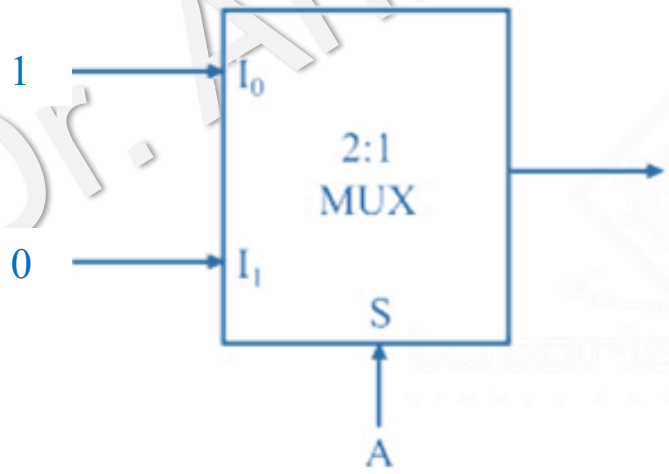
Example: **Design a NOT gate using 2:1 MUX.**



A	Y
0	1
1	0

Y = 1 when A = 0

Y = 0 when A = 1



$= A'.1 + A \cdot 0 = A'$

Dr. Arindam Dutta

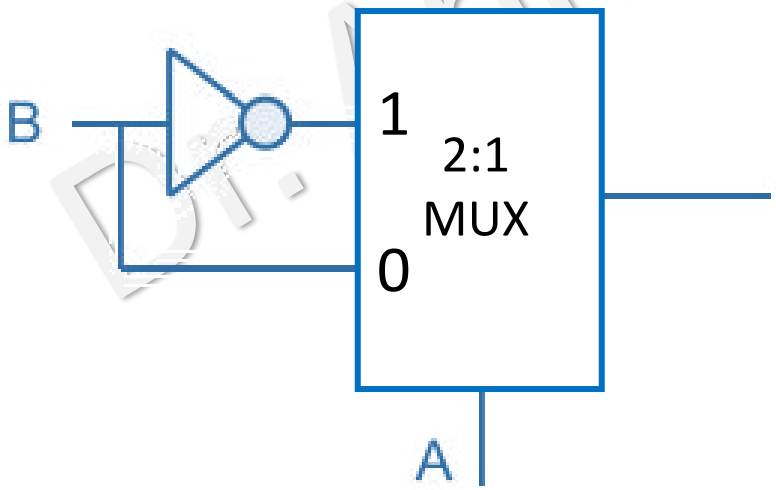
Problem

Example: Design an XOR gate using 2:1 MUX.

Taking A as the select line and B as the input variable:



A	B	Y	
0	0	0	Y = B when A = 0
0	1	1	
1	0	1	Y = $\overline{B}$ when A = 1
1	1	0	



$Y = \overline{A}\,B + A\overline{B} = A \oplus B$

Dr. Arindam Dutta

Problem

Example: Design an XNOR gate using 2:1 MUX.

Taking A as the select line and B as the input variable:

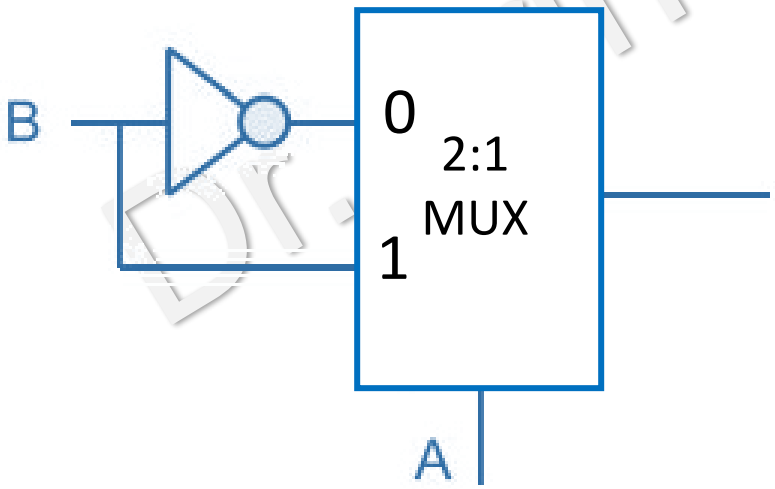


A	B	OUT
0	0	1
0	1	0
1	0	0
1	1	1

Y

$Y = \overline{B}$ when $A = 0$

$Y = B$ when $A = 1$

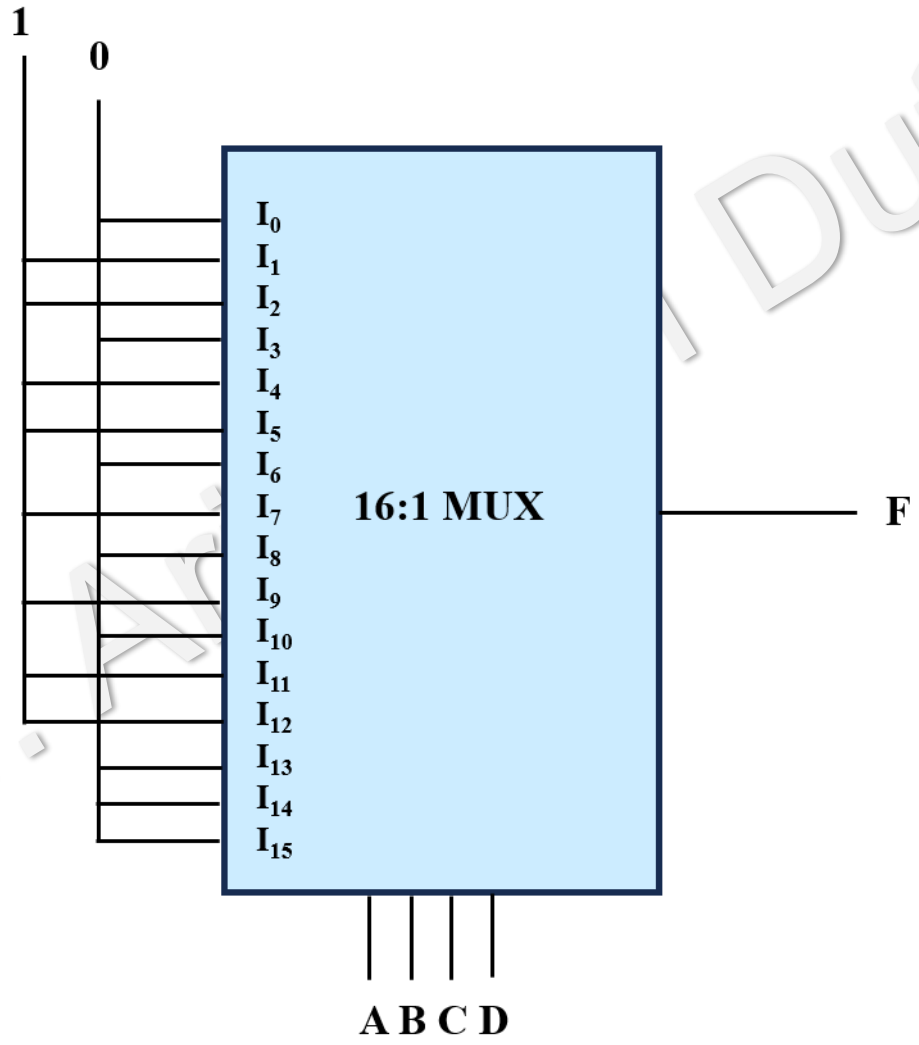


$$Y = AB + \overline{A}\,\overline{B} = Y = A \odot B$$

Dr. Arindam Dutta

Problem

Example: **Implement the Boolean function F(A,B,C,D) = Σ (1,2,4,5,7,9,11,12) using a MUX.**



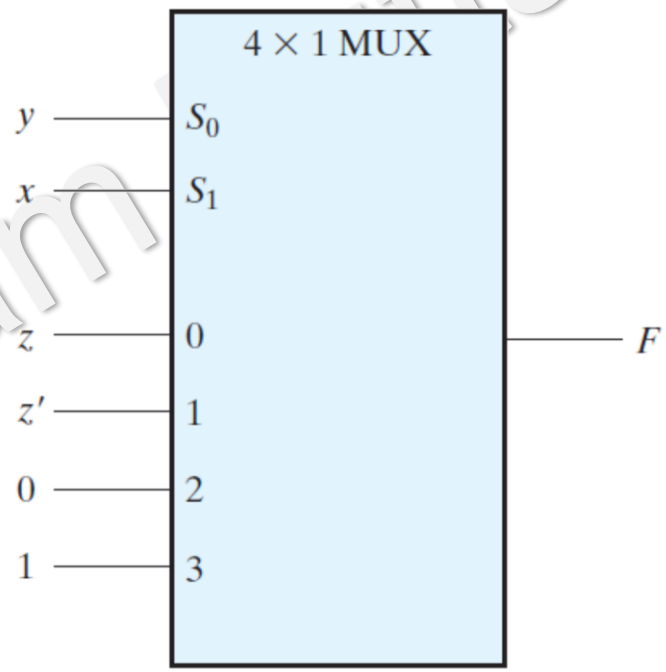
Dr. Ari Dutta

Problem

Example: Implement $f(x, y, z) = \Sigma(1,2,6,7)$ using 4:1 MUX.

Taking x and y as the select lines and z as the input variable:

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	



Dr. Arindam Dutta

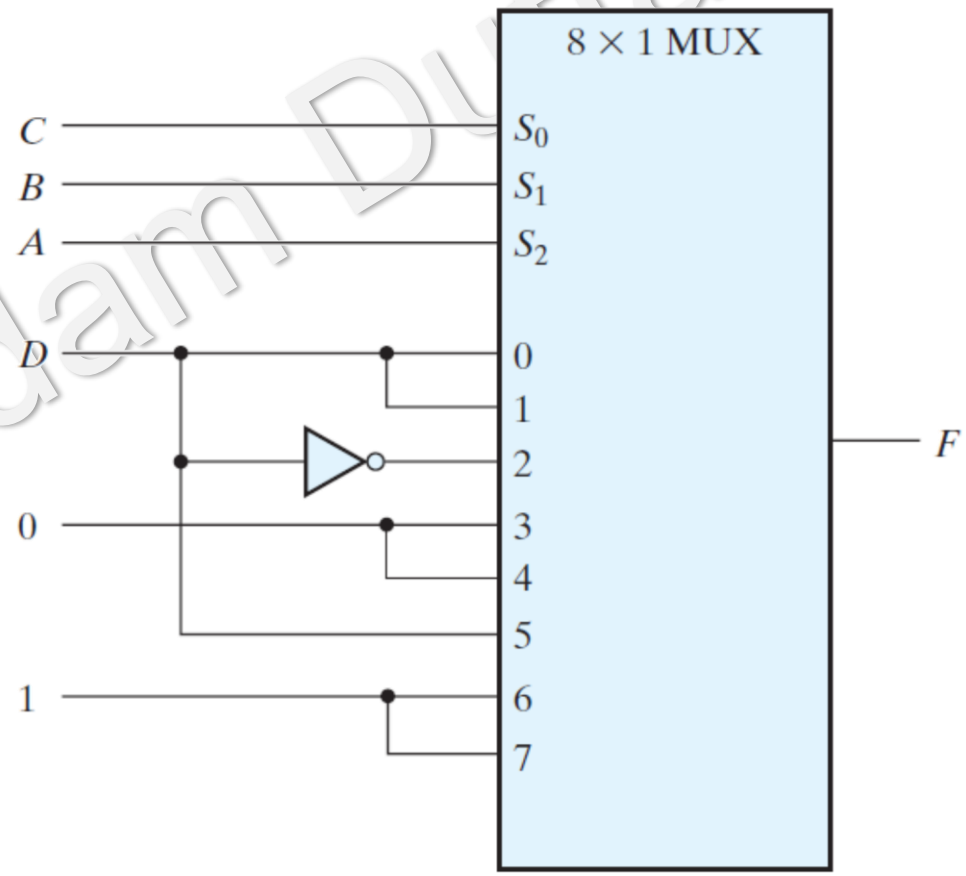
N. B. *We have the freedom to choose which variables would represent the input and the select lines (unless specifically mentioned in the question).*

Problem

Example: Implement $F(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$ using 8:1 MUX.

Taking A, B, and C as the select lines and D as the input variable:

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



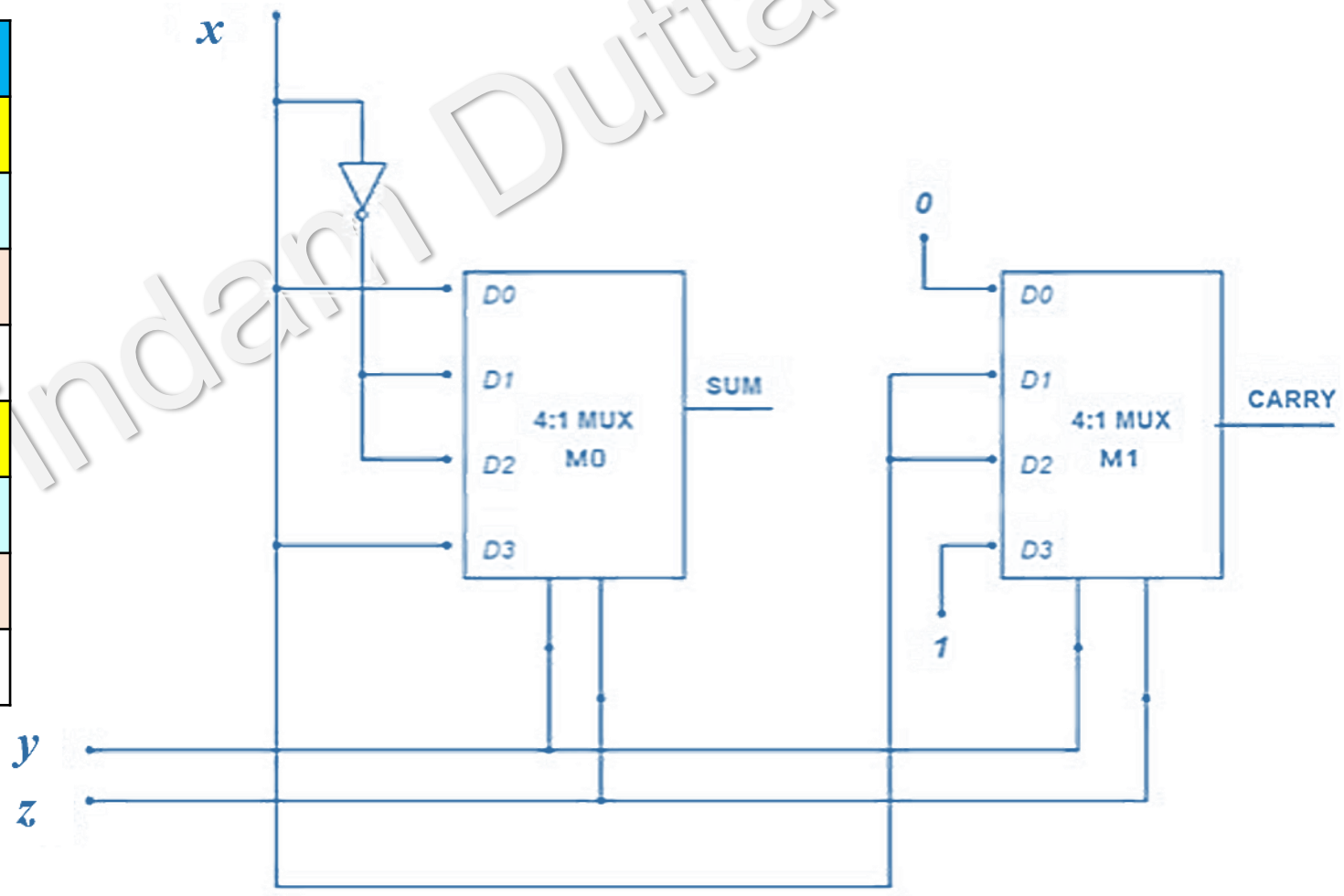
Dr. Arindam Dutta

Problem

Example: Implement full adder using 4:1 MUX.

Taking *y* and *z* as the select lines and *x* as the input variable:

x	*y*	*z*	*S*	*C*		*D*
0	0	0	0	0	$S = x, C = 0$	*D0*
0	0	1	1	0	$S = x', C = x$	*D1*
0	1	0	1	0	$S = x', C = x$	*D2*
0	1	1	0	1	$S = x, C = 1$	*D3*
1	0	0	1	0	$S = x, C = 0$	*D0*
1	0	1	0	1	$S = x', C = x$	*D1*
1	1	0	0	1	$S = x', C = x$	*D2*
1	1	1	1	1	$S = x, C = 1$	*D3*



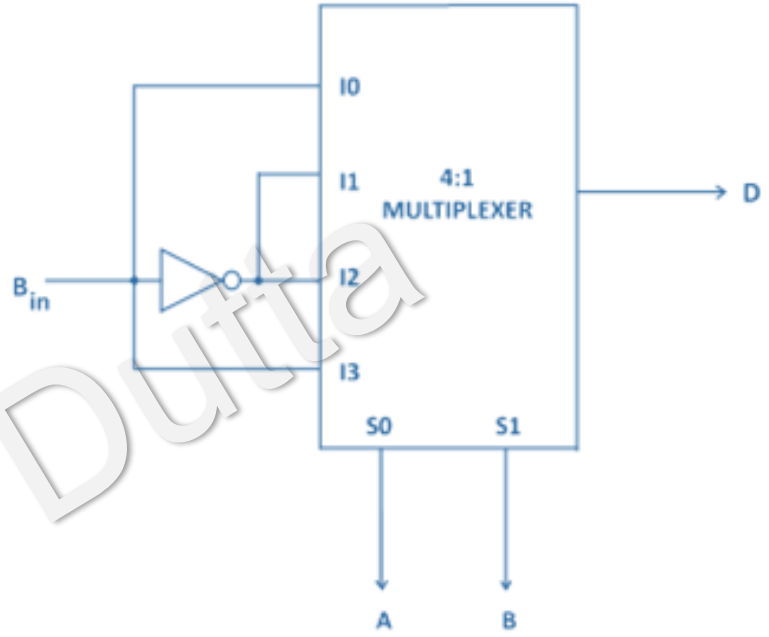
Dr. Jayanta Dutta

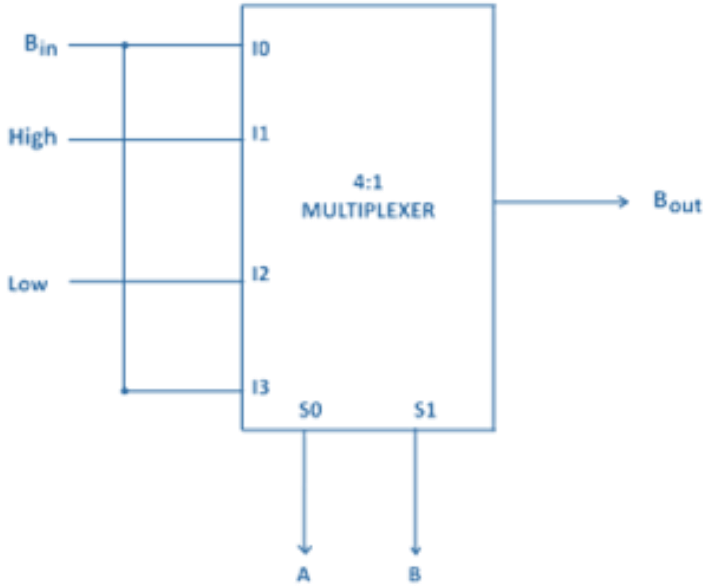
Problem

Example: Implement full subtractor using 4:1 MUX.

Taking A and B as the select lines and B_{in} as the input variable:

A	B	B_{in}	D	B_{out}	
0	0	0	0	0	$D = B_{in}$, $B_{out} = B_{in}$
0	0	1	1	1	
0	1	0	1	1	$D = B_{in}'$, $B_{out} = 1$
0	1	1	0	1	
1	0	0	1	0	$D = B_{in}'$, $B_{out} = 0$
1	0	1	0	0	
1	1	0	0	0	$D = B_{in}$, $B_{out} = B_{in}$
1	1	1	1	1	





Dr. Sam Dutta

ECE2002
DIGITAL LOGIC DESIGN

Module 2 – Lecture 5

Dr. Arindam Dutta

Assistant Professor
School of Electrical and Electronics Engineering
VIT Bhopal

Encoder and Decoder

Encoder

- digital circuit that takes multiple input signals and encodes them into smaller number of output signals
- only one input active at one time
- reduces number of data lines by converting information from one format to another, often by compressing multiple inputs into a binary code

Decoder

- reverse function of encoder: takes a binary input and decodes it into multiple outputs
- interprets the coded information back into a format that can be easily understood or used

Applications

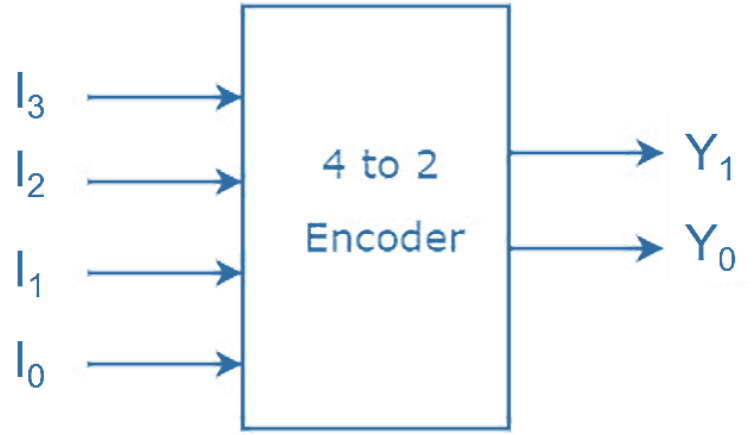
- **Encoder:** keyboards – encodes a keypress into binary code for the computer to process
- **Decoder:** digital displays – converts binary coded decimals (BCD) into signals for lighting up appropriate segments

Dr. Arindam Dutta

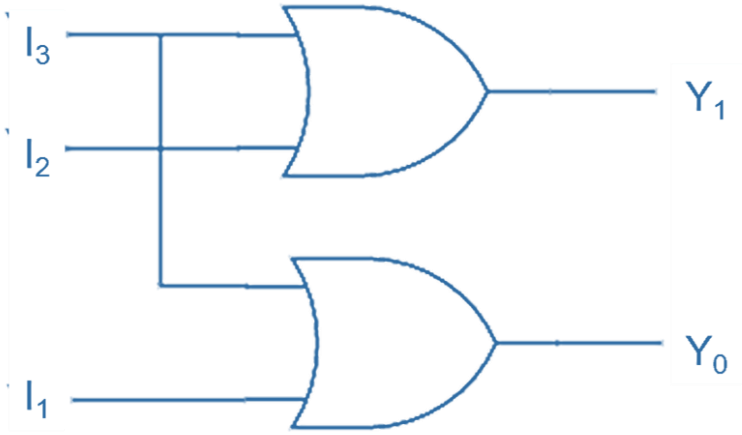
Encoder

- typically has 2^n input lines and n output lines

4:2 Encoder: has 4 inputs and 2 outputs. If the inputs are I_0, I_1, I_2, and I_3, then the two outputs will be Y_0 and Y_1



I_3	I_2	I_1	I_0	Y_1	Y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1



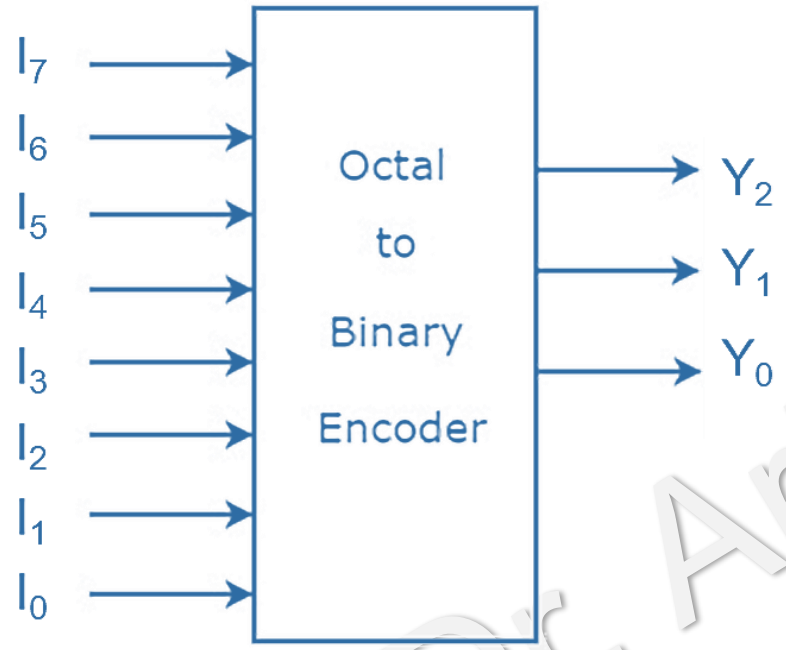
$$Y_0 = I_1 + I_3$$

$$Y_1 = I_2 + I_3$$

Dr. Arindam Dutta

Encoder

8:3 Encoder (Octal to Binary Encoder): has 8 inputs and 3 outputs



I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Dr. Arindam Dutta

$$Y_0 = I_1 + I_3 + I_5 + I_7$$

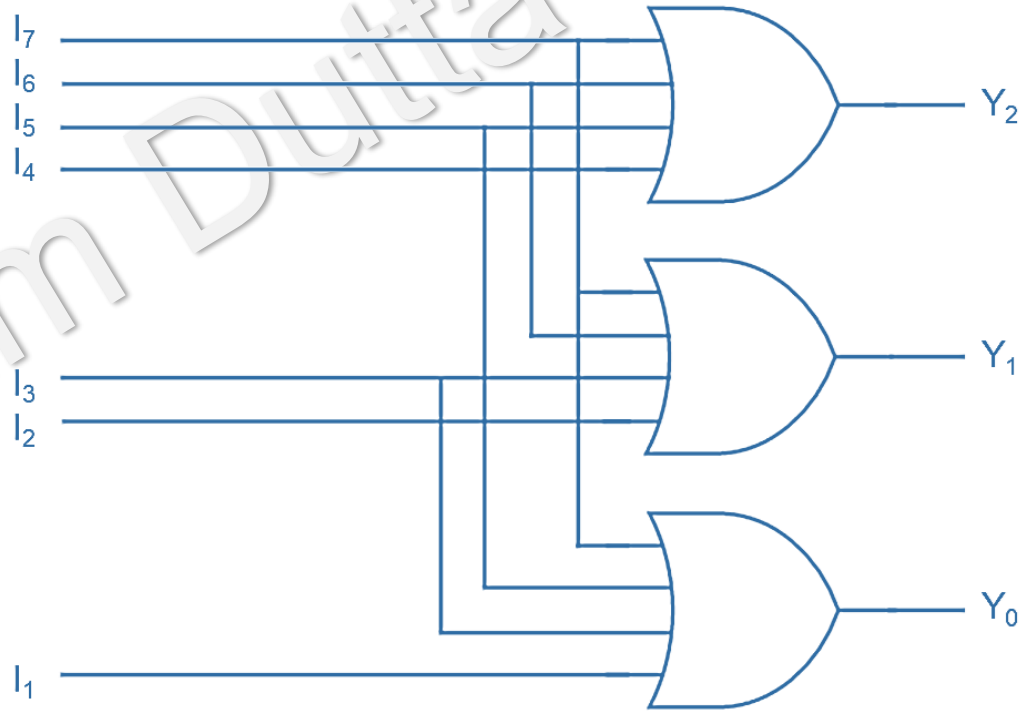
$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_2 = I_4 + I_5 + I_6 + I_7$$

Encoder

Logic Circuit of 8:3 Encoder

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1



Dr Arindam Dutta

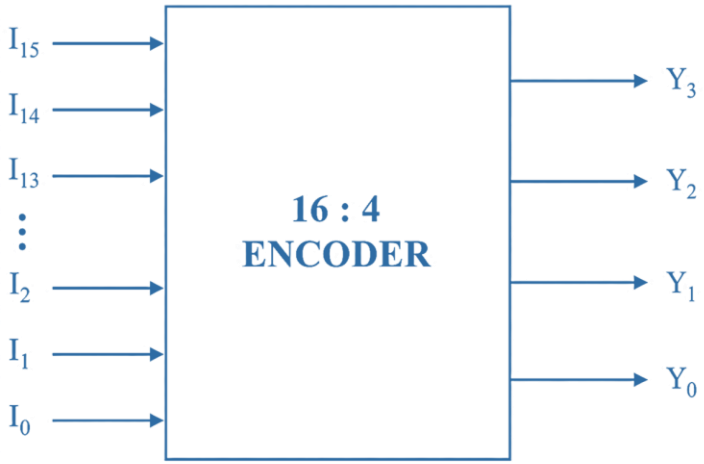
$$Y_0 = I_1 + I_3 + I_5 + I_7$$

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_2 = I_4 + I_5 + I_6 + I_7$$

Encoder

16:4 Encoder (Hexadecimal to Binary Encoder): has 16 inputs and 4 outputs



$$Y_0 = I_1 + I_3 + I_5 + I_7 + I_9 + I_{11} + I_{13} + I_{15}$$

$$Y_1 = I_2 + I_3 + I_6 + I_7 + I_{10} + I_{11} + I_{14} + I_{15}$$

$$Y_2 = I_4 + I_5 + I_6 + I_7 + I_{12} + I_{13} + I_{14} + I_{15}$$

$$Y_3 = I_8 + I_9 + I_{10} + I_{11} + I_{12} + I_{13} + I_{14} + I_{15}$$

I_{15}	I_{14}	I_{13}	I_{12}	I_{11}	I_{10}	I_9	I_8	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	1	0
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

Dr. Arindam Dutta

Encoder

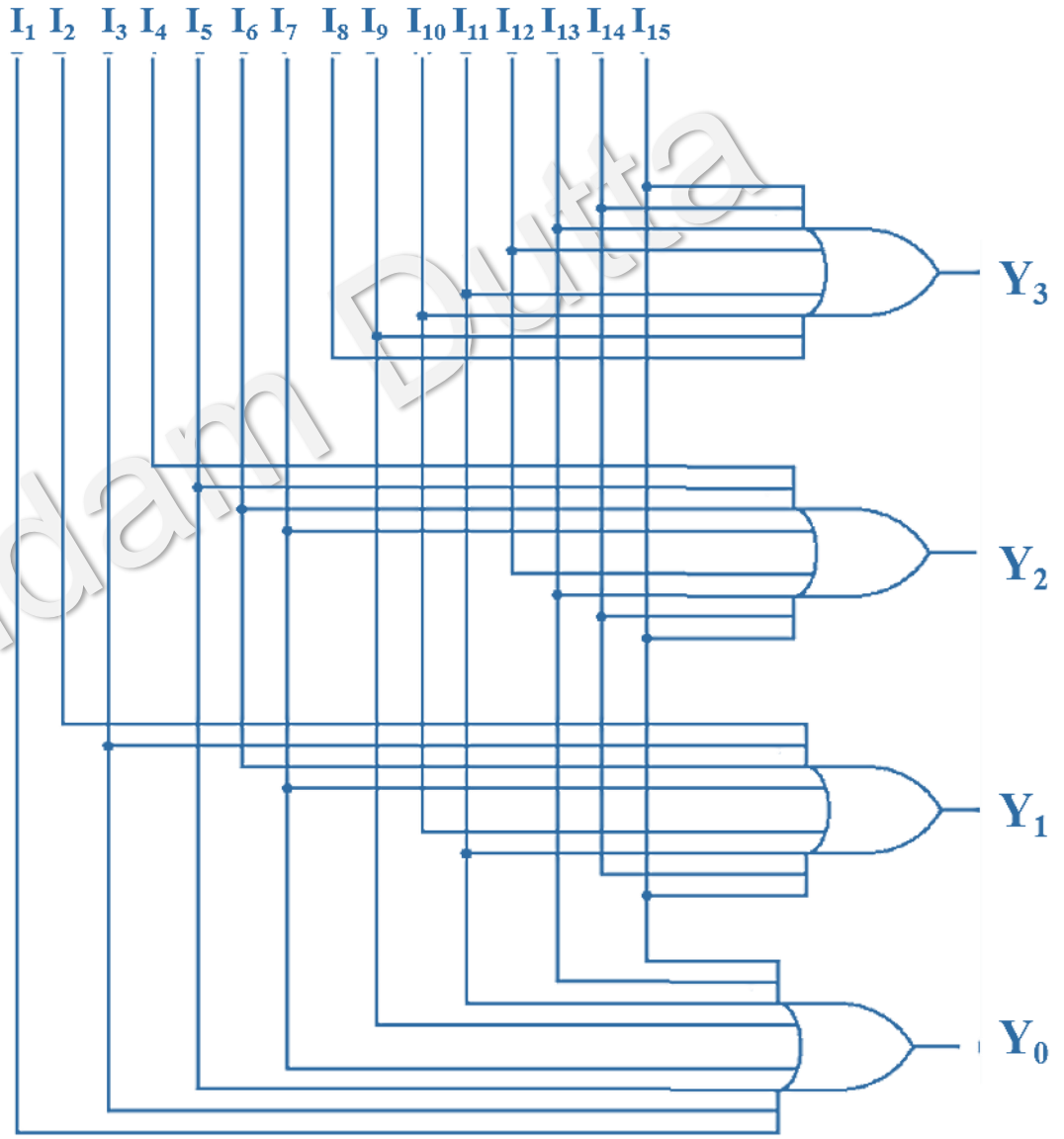
Logic Circuit of 16:4 Encoder

$$Y_0 = I_1 + I_3 + I_5 + I_7 + I_9 + I_{11} + I_{13} + I_{15}$$

$$Y_1 = I_2 + I_3 + I_6 + I_7 + I_{10} + I_{11} + I_{14} + I_{15}$$

$$Y_2 = I_4 + I_5 + I_6 + I_7 + I_{12} + I_{13} + I_{14} + I_{15}$$

$$Y_3 = I_8 + I_9 + I_{10} + I_{11} + I_{12} + I_{13} + I_{14} + I_{15}$$

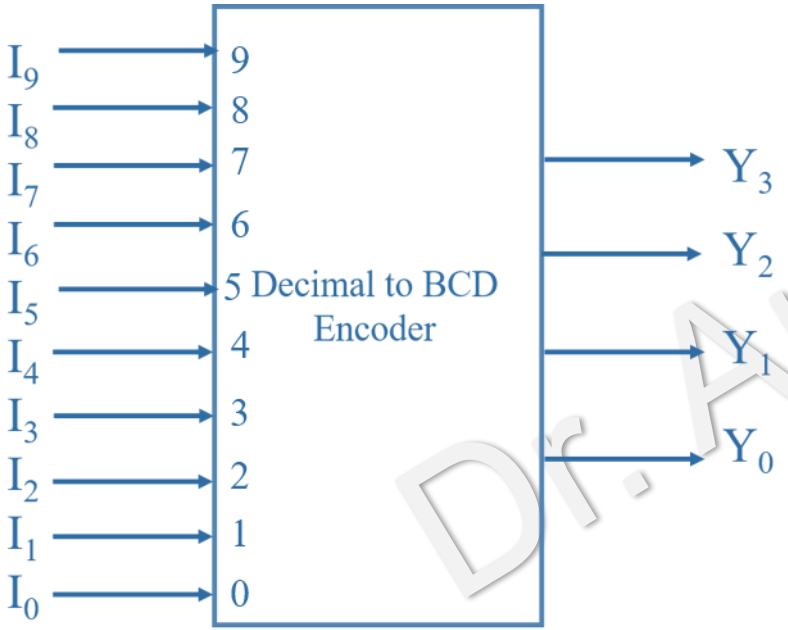


Dr. Arindam Dutta

Encoder

Decimal to BCD Encoder

- Ten inputs ($I_0 – I_9$) corresponding to 10 digits (0 – 9)
- Four outputs ($Y_0 – Y_3$) representing the BCD



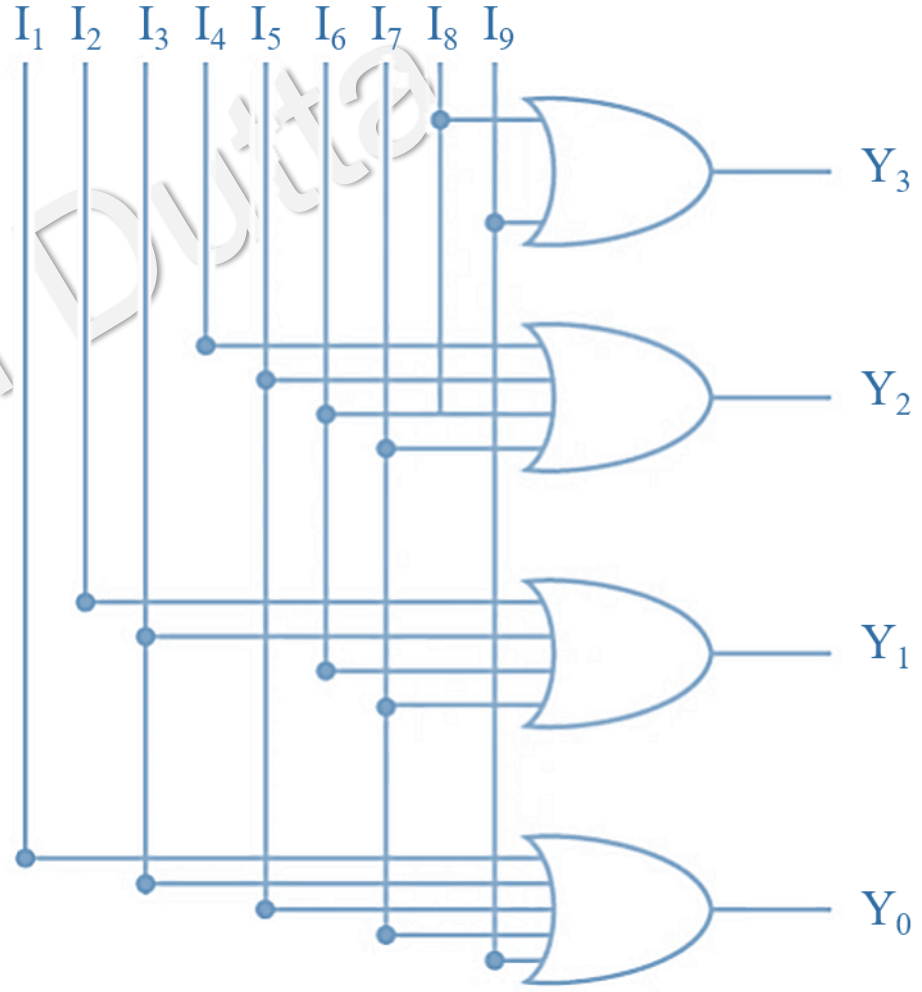
I_9	I_8	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	1	0	0	1

Dr. Arindam Dutta

Encoder

Logic Circuit of Decimal to BCD Encoder

I_9	I_8	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	1	0	0	1



$$Y_3 = I_8 + I_9$$

$$Y_2 = I_4 + I_5 + I_6 + I_7$$

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_0 = I_1 + I_3 + I_5 + I_7 + I_9$$

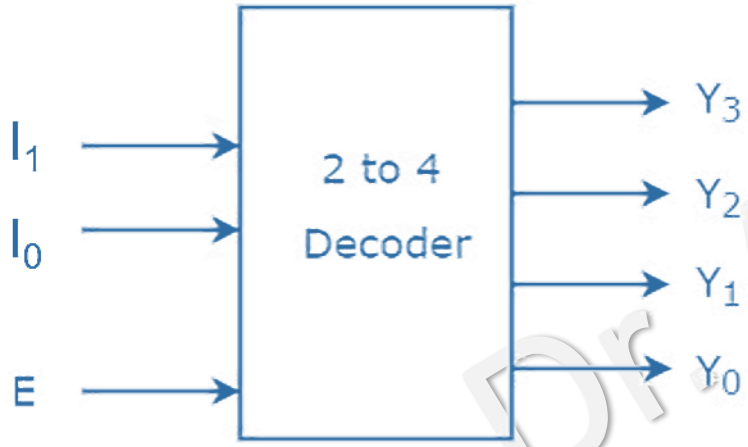
Dr Arindam Dutta

Decoder

- typically has 2^n output lines and n input lines

2:4 Decoder: has 2 inputs and 4 outputs

- if the inputs are I_0, I_1, then the four outputs will be Y_0, Y_1, Y_2, and Y_3
- one of these outputs will be 1 for each combination of inputs when enable, E is 1



E	I_1	I_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

$$Y_0 = E\,\bar{I_0}\,\bar{I_1}$$

$$Y_1 = E\,I_0\,\bar{I_1}$$

$$Y_2 = E\,\bar{I_0}\,I_1$$

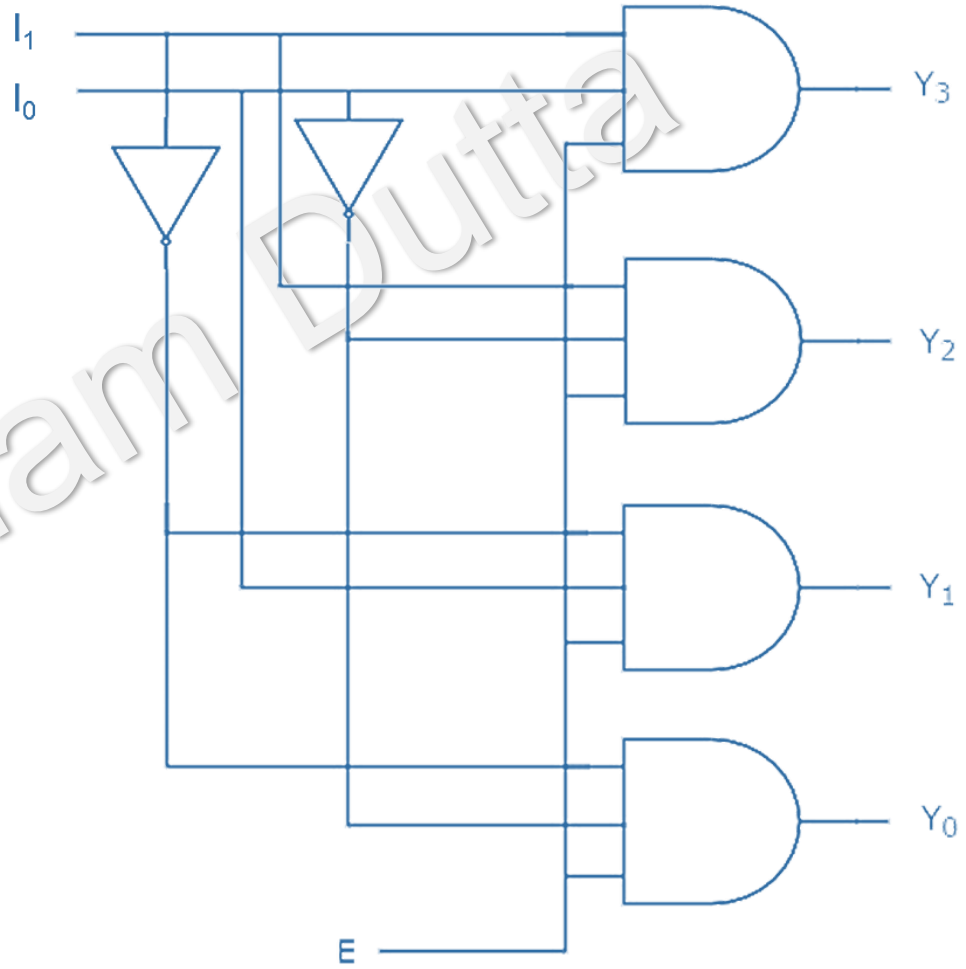
$$Y_3 = E\,I_0\,I_1$$

Dr Arindam Dutta

Decoder

Logic Circuit of 2:4 Decoder

E	I_1	I_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0



$$Y_0 = E\,\overline{I_0}\,\overline{I_1} \qquad Y_2 = E\,\overline{I_0}\,I_1$$

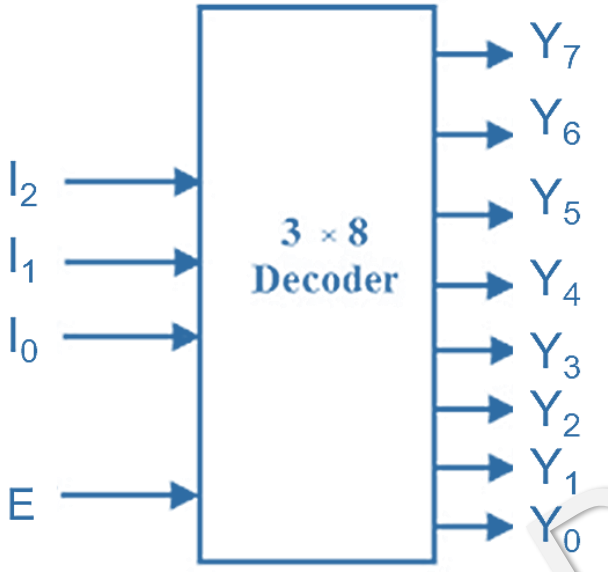
$$Y_1 = E\,I_0\,\overline{I_1} \qquad Y_3 = E\,I_0\,I_1$$

Dr. Arindam Dutta

Decoder

3:8 (Binary to Octal) Decoder:

- if the inputs are I_0, I_1, and I_2 then the eight outputs will be Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, and Y_7
- one of these outputs will be 1 for each combination of inputs when enable, E is 1



E	I_2	I_1	I_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	X	X	X	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

$$Y_0 = E\bar{I_2}\bar{I_1}\bar{I_0}$$

$$Y_1 = E\bar{I_2}\bar{I_1}I_0$$

$$Y_2 = E\bar{I_2}I_1\bar{I_0}$$

$$Y_3 = E\bar{I_2}I_1I_0$$

$$Y_4 = EI_2\bar{I_1}\bar{I_0}$$

$$Y_5 = EI_2\bar{I_1}I_0$$

$$Y_6 = EI_2I_1\bar{I_0}$$

$$Y_7 = EI_2I_1I_0$$

Dr. Arindam Dutta

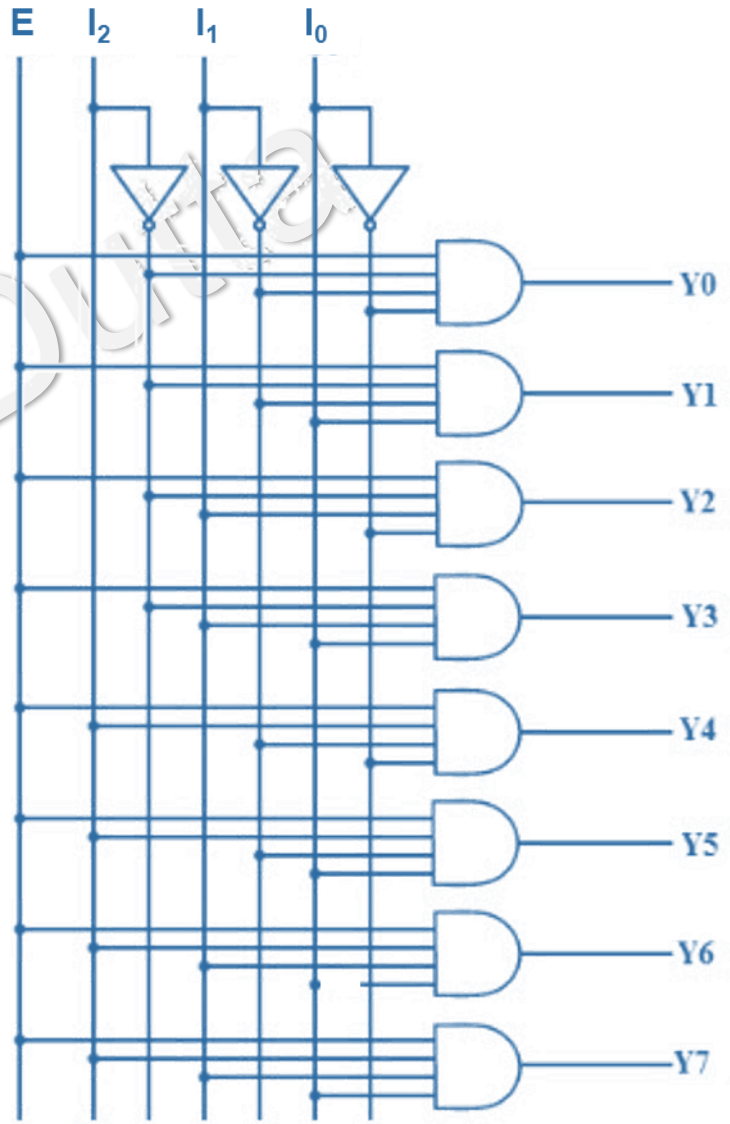
Decoder

Logic Circuit of 3:8 Decoder

E	I_2	I_1	I_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	X	X	X	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

$$Y_0 = E\bar{I_2}\bar{I_1}\bar{I_0} \quad Y_2 = E\bar{I_2}I_1\bar{I_0} \quad Y_4 = EI_2\bar{I_1}\bar{I_0} \quad Y_6 = EI_2I_1\bar{I_0}$$

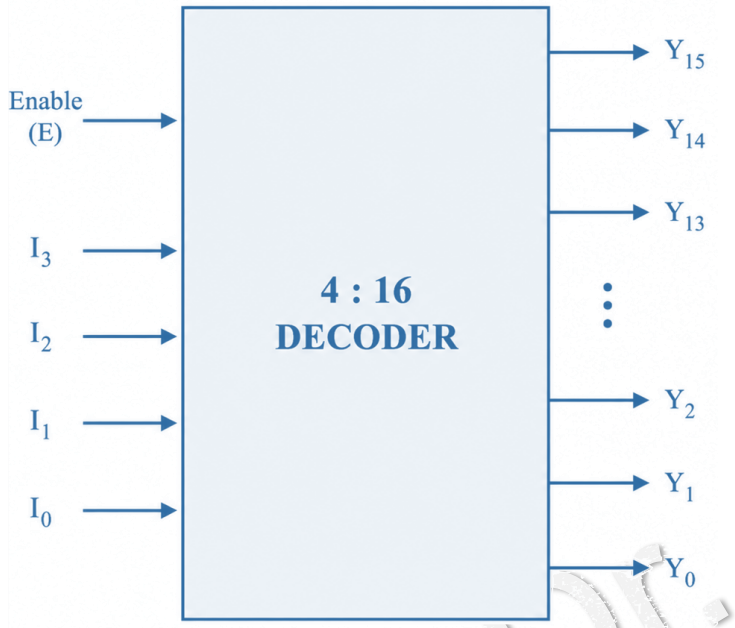
$$Y_1 = E\bar{I_2}\bar{I_1}I_0 \quad Y_3 = E\bar{I_2}I_1I_0 \quad Y_5 = EI_2\bar{I_1}I_0 \quad Y_7 = EI_2I_1I_0$$



Dr. Arindam Dutta

Decoder

4:16 (Binary to Hexadecimal) Decoder



E	I_0	I_1	I_2	I_3	Y_{15}	Y_{14}	Y_{13}	Y_{12}	Y_{11}	Y_{10}	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	X	X	X	X	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
1	0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Dr. Arindam Dutta

Decoder

Logic Circuit of 4:16 Decoder

$$Y_0 = E\bar{I}_0\bar{I}_1\bar{I}_2\bar{I}_3 \qquad Y_8 = EI_0\bar{I}_1\bar{I}_2\bar{I}_3$$

$$Y_1 = E\bar{I}_0\bar{I}_1\bar{I}_2I_3 \qquad Y_9 = EI_0\bar{I}_1\bar{I}_2I_3$$

$$Y_2 = E\bar{I}_0\bar{I}_1I_2\bar{I}_3 \qquad Y_{10} = EI_0\bar{I}_1I_2\bar{I}_3$$

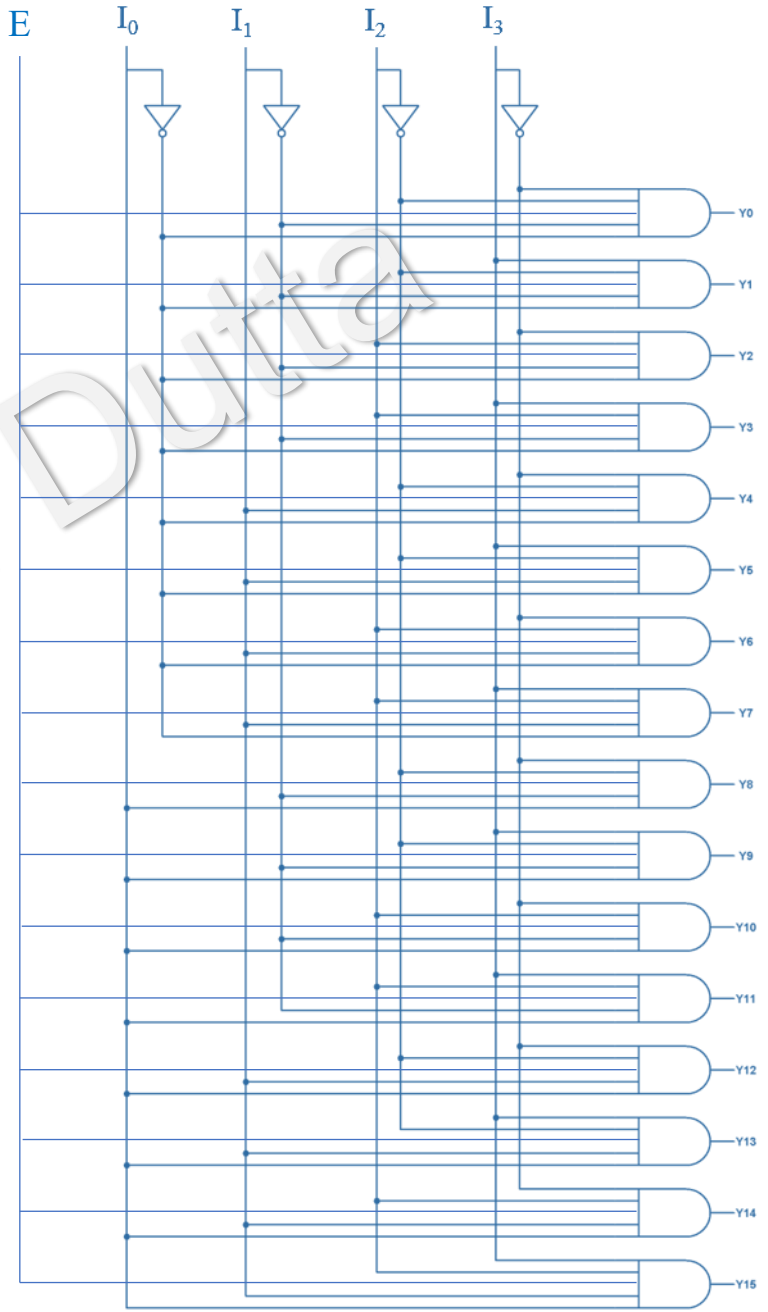
$$Y_3 = E\bar{I}_0\bar{I}_1I_2I_3 \qquad Y_{11} = EI_0\bar{I}_1I_2I_3$$

$$Y_4 = E\bar{I}_0I_1\bar{I}_2\bar{I}_3 \qquad Y_{12} = EI_0I_1\bar{I}_2\bar{I}_3$$

$$Y_5 = E\bar{I}_0I_1\bar{I}_2I_3 \qquad Y_{13} = EI_0I_1\bar{I}_2I_3$$

$$Y_6 = E\bar{I}_0I_1I_2\bar{I}_3 \qquad Y_{14} = EI_0I_1I_2\bar{I}_3$$

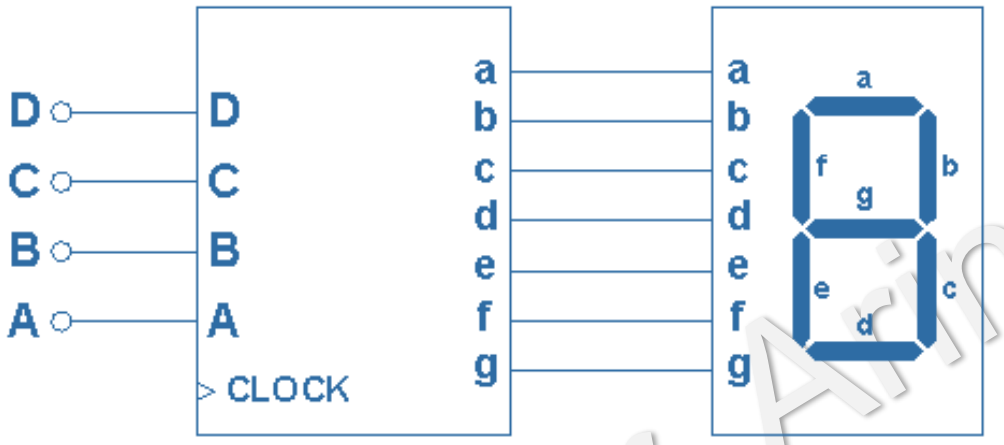
$$Y_7 = E\bar{I}_0I_1I_2I_3 \qquad Y_{15} = EI_0I_1I_2I_3$$



Dr. Arindam Dutta

Decoder

BCD to 7-Segment Decoder: code converter that converts a 4-bit BCD input (0–9) into 7 output signals to light up the correct segments of a digital display.



Digit	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

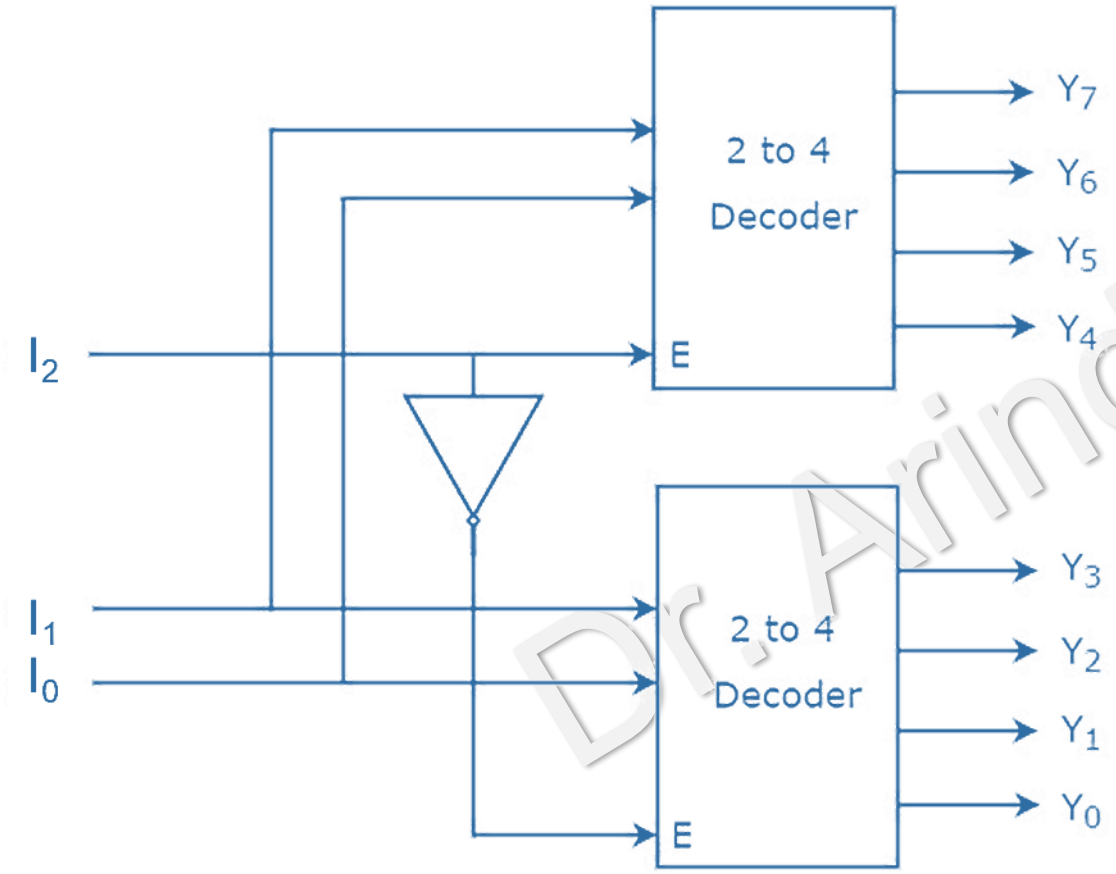
Dr. Arindam Dutta

Applications: digital clocks, calculators, counters and meters, etc.

Decoder

Construction of 3:8 Decoder from 2:4 Decoders

Required number of lower order decoders = 8 / 4 = 2



- When I_2 = 1, the top decoder is enabled and the other is disabled. The bottom decoder outputs are all 0's, and the top four outputs generate minterms 000 to 011.
- When I_2 = 0, the enable conditions are reversed: The bottom decoder outputs generate minterms 100 to 111, while the outputs of the top decoder are all 0's.

Dr. Arindam Dutta

Decoder

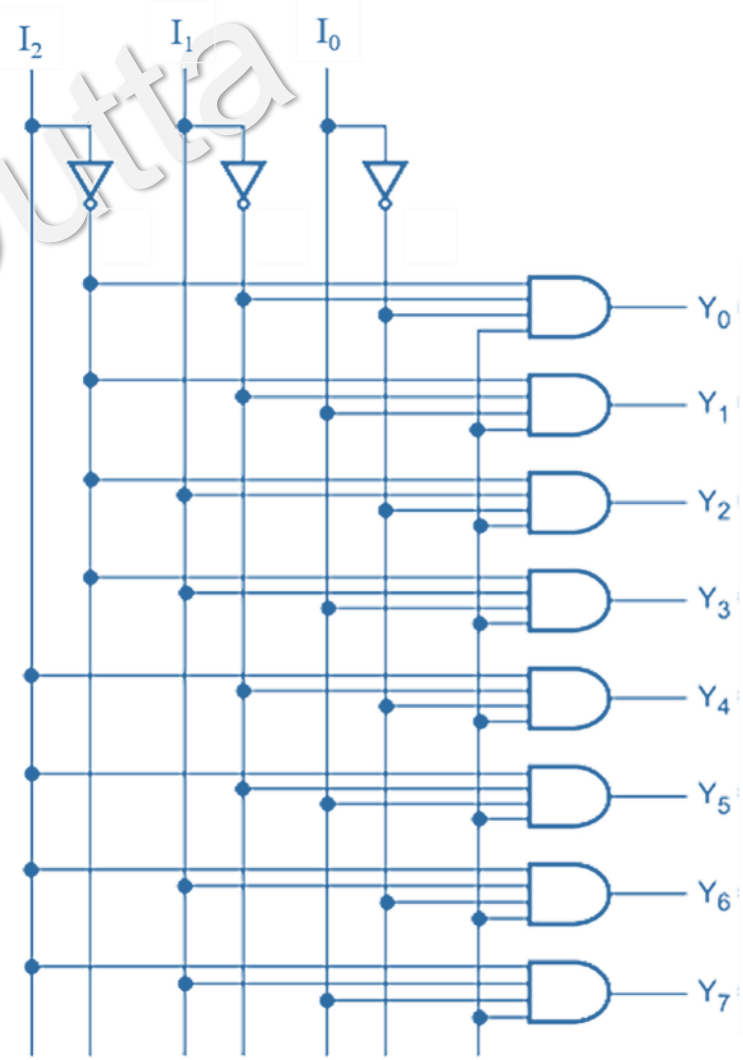
Logic Circuit of 3:8 Decoder Using 2:4 Decoders

The Enable (I_2) pin acts as one of the input pins for both 2:4 decoder circuits.

I_2	I_1	I_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

$$Y_0 = \overline{I_2}\,\overline{I_1}\,\overline{I_0} \qquad Y_2 = \overline{I_2} I_1 \overline{I_0} \qquad Y_4 = I_2 \overline{I_1}\,\overline{I_0} \qquad Y_6 = I_2 I_1 \overline{I_0}$$

$$Y_1 = \overline{I_2}\,\overline{I_1} I_0 \qquad Y_3 = \overline{I_2} I_1 I_0 \qquad Y_5 = I_2 \overline{I_1} I_0 \qquad Y_7 = I_2 I_1 I_0$$

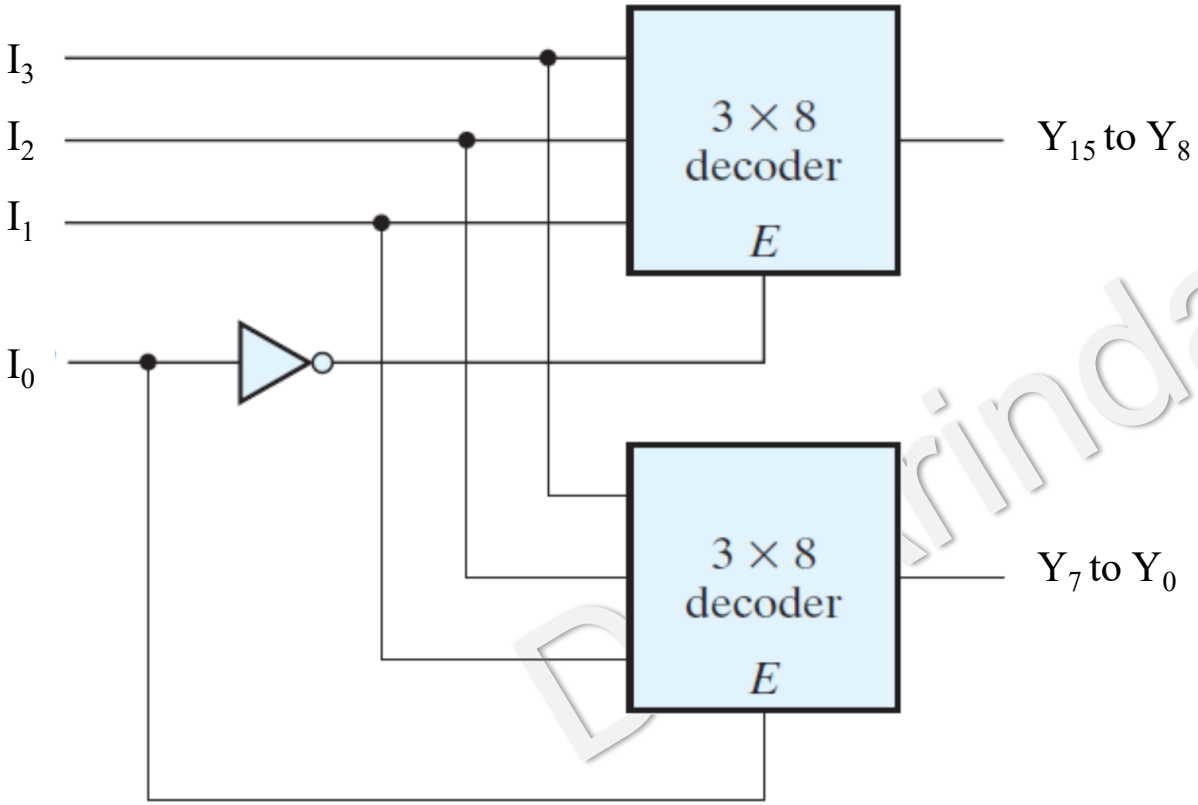


Dr. Arindam Dutta

Decoder

Construction of 4:16 Decoder from 3:8 Decoders

Required number of lower order decoders = 16 / 8 = 2



- When $I_0 = 0$, the top decoder is enabled and the other is disabled. The bottom decoder outputs are all 0's, and the top eight outputs generate minterms 0000 to 0111.
- When $I_0 = 1$, the enable conditions are reversed: The bottom decoder outputs generate minterms 1000 to 1111, while the outputs of the top decoder are all 0's.

Dhrindam Dutta

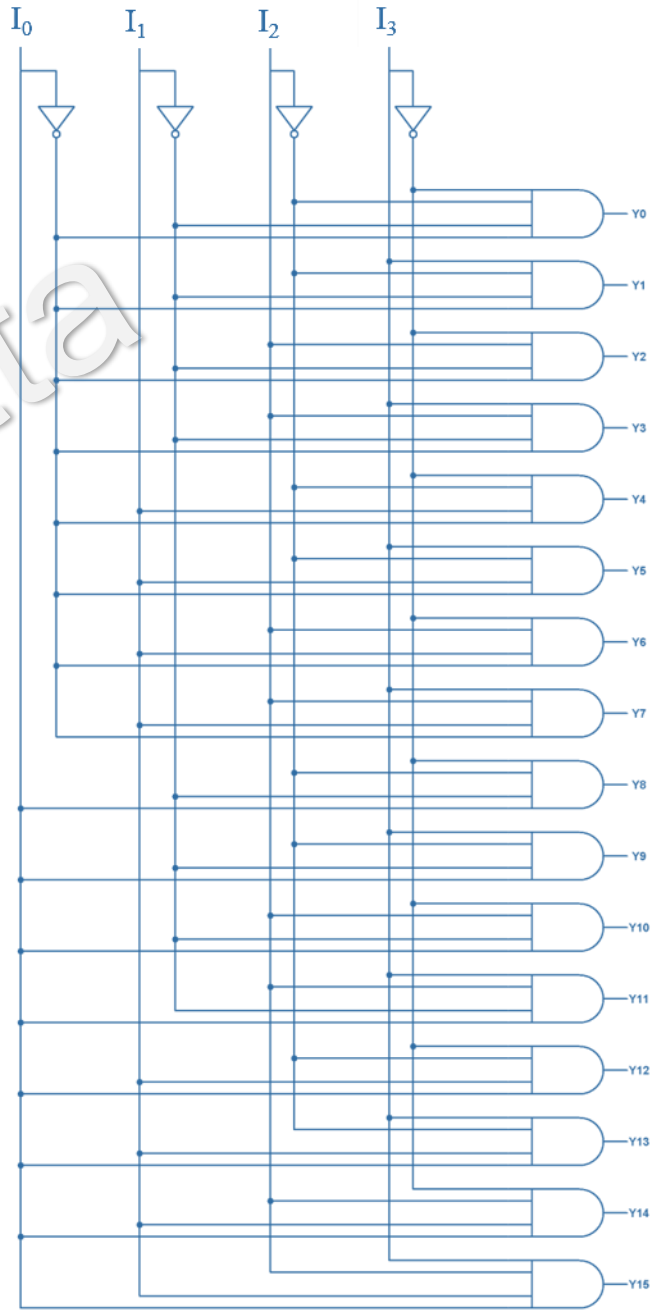
Decoder

Logic Circuit of 4:16 Decoder Using 3:8 Decoders

The Enable (I_0) pin acts as one of the input pins for both 3:8 decoder circuits.

I_0	I_1	I_2	I_3	Y_{15}	Y_{14}	Y_{13}	Y_{12}	Y_{11}	Y_{10}	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

$$Y_0 = \bar{I_0}\bar{I_1}\bar{I_2}\bar{I_3}$$
$$Y_1 = \bar{I_0}\bar{I_1}\bar{I_2}I_3$$
$$Y_2 = \bar{I_0}\bar{I_1}I_2\bar{I_3}$$
$$Y_3 = \bar{I_0}\bar{I_1}I_2I_3$$
$$Y_4 = \bar{I_0}I_1\bar{I_2}\bar{I_3}$$
$$Y_5 = \bar{I_0}I_1\bar{I_2}I_3$$
$$Y_6 = \bar{I_0}I_1I_2\bar{I_3}$$
$$Y_7 = \bar{I_0}I_1I_2I_3$$
$$Y_8 = I_0\bar{I_1}\bar{I_2}\bar{I_3}$$
$$Y_9 = I_0\bar{I_1}\bar{I_2}I_3$$
$$Y_{10} = I_0\bar{I_1}I_2\bar{I_3}$$
$$Y_{11} = I_0\bar{I_1}I_2I_3$$
$$Y_{12} = I_0I_1\bar{I_2}\bar{I_3}$$
$$Y_{13} = I_0I_1\bar{I_2}I_3$$
$$Y_{14} = I_0I_1I_2\bar{I_3}$$
$$Y_{15} = I_0I_1I_2I_3$$



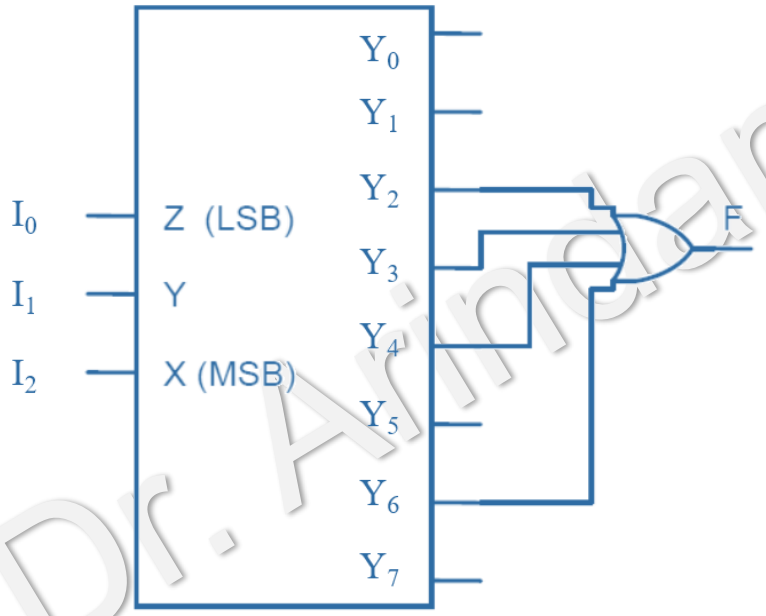
Dr. Arindam Dutta

Problem

Example: Implement the function F(x, y, z) = Σ (2, 3, 4, 6) using a 3:8 decoder.

Truth table of F(x, y, z)

x	y	z	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0



Truth table of 3:8 decoder

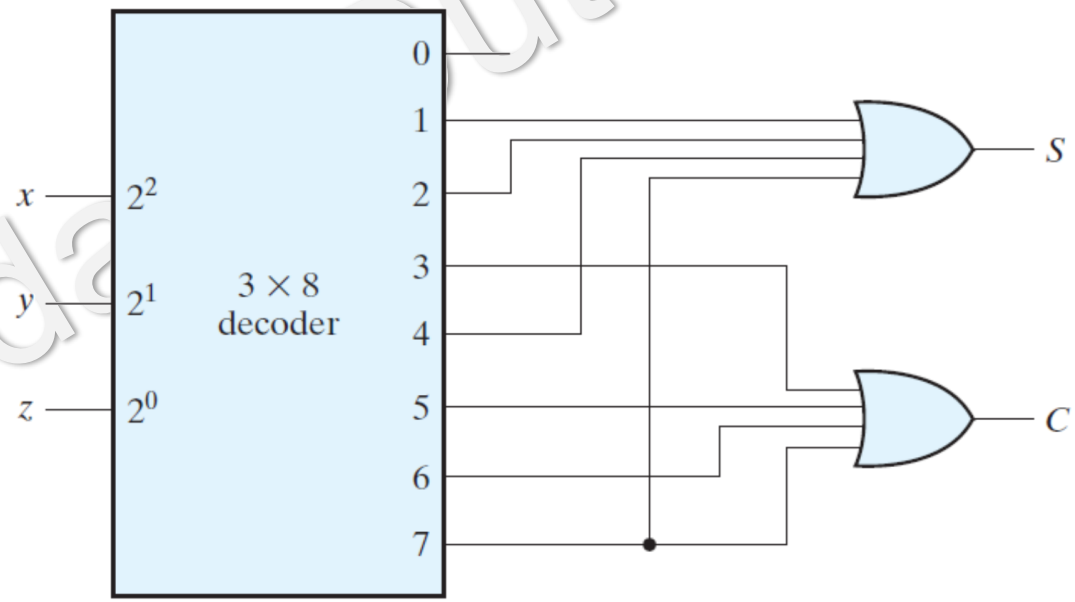
I_2	I_1	I_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

Dr. Arindam Dutta

Problem

Example: **Implement a full-adder circuit using a 3:8 decoder.**

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$S(x, y, z) = \Sigma(1, 2, 4, 7)$

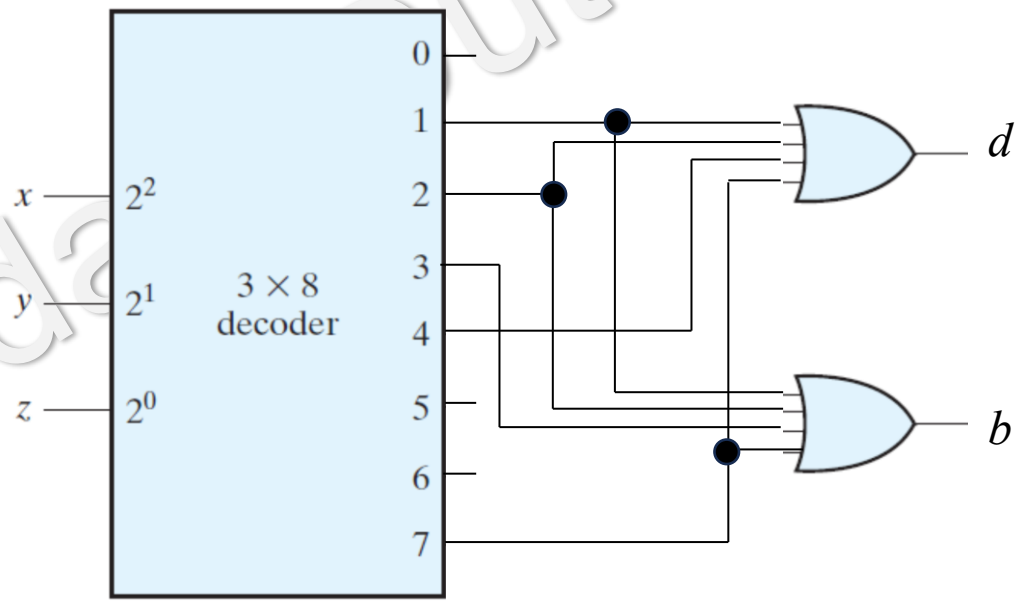
$C(x, y, z) = \Sigma(3, 5, 6, 7)$

Dr. Arinda Dutta

Problem

Example: **Implement a full-subtractor circuit using a 3:8 decoder.**

Inputs			Outputs	
x	*y*	*z*	*d* (Difference)	*b* (Borrow)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



$$d\,(x, y, z) = \Sigma\,(1, 2, 4, 7)$$

$$b\,(x, y, z) = \Sigma\,(1, 2, 3, 7)$$

Dr. Arinda Dutta